WEEK 3
C# INTERMEDIATE

29/09/2025 – 03/10/2025

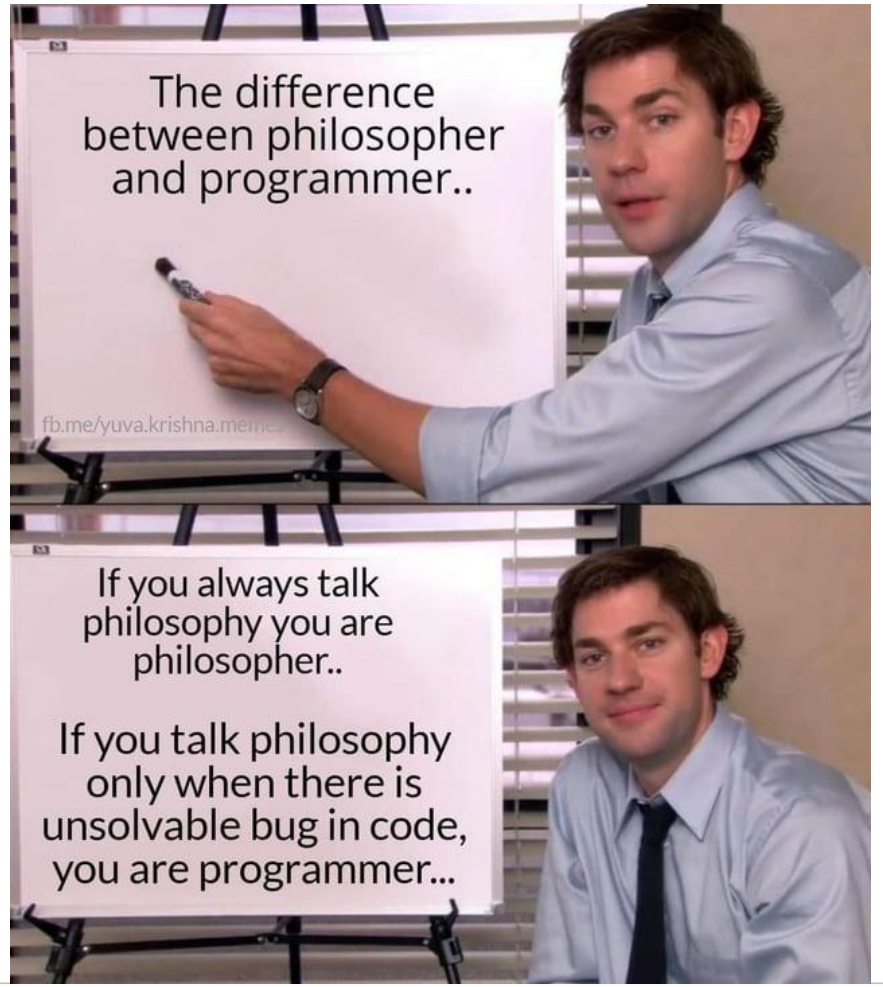
Alessandro Gargiulo
alessandro.gargiulo@msc.com

NICE TO MEET YOU

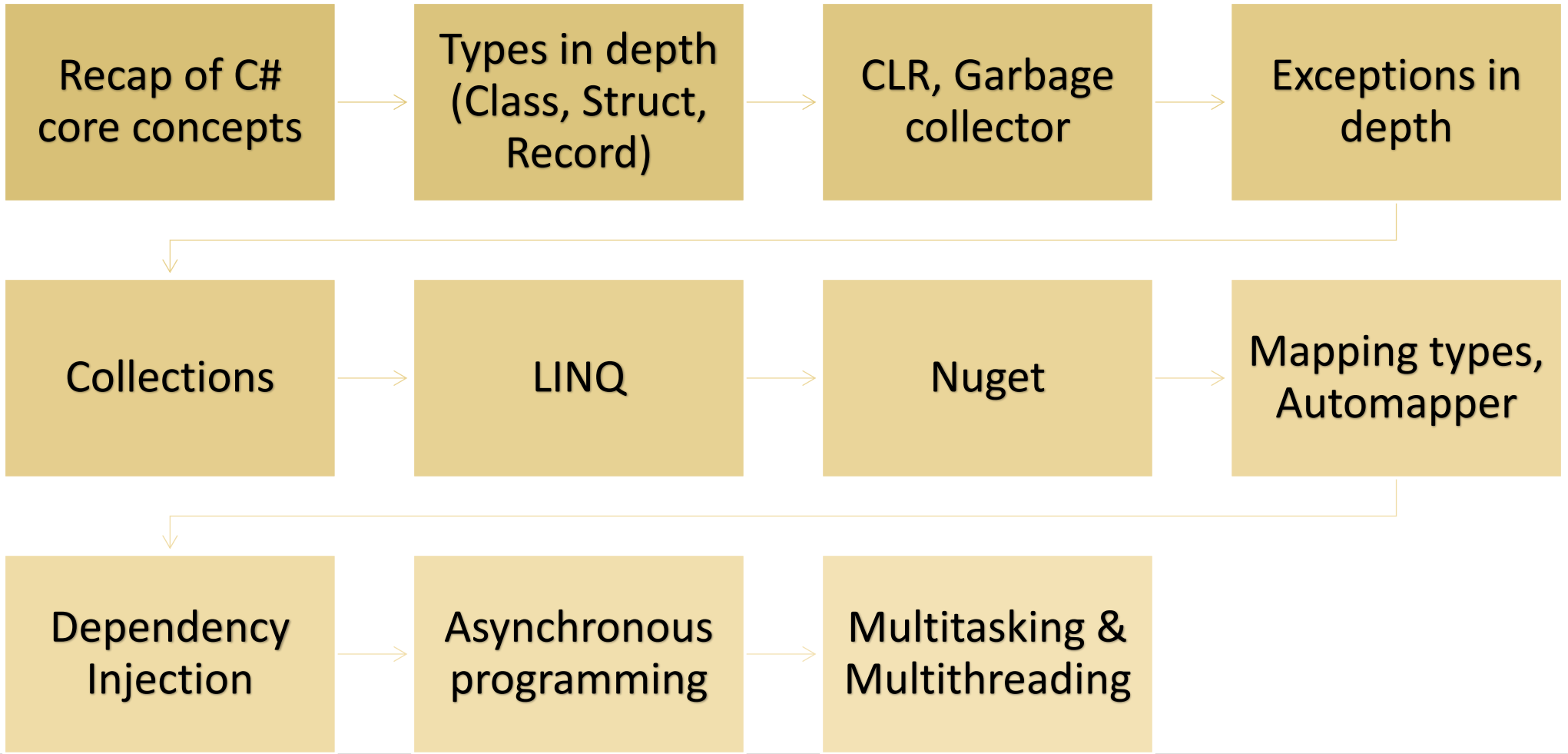
Alessandro Gargiulo

- Technical Team Lead @ MSC Technology Italy
- 3+ year on-board in MSC
- Teacher since academy 3rd edition
- 8+ years experience on .NET
- alessandro.gargiulo@msc.com
- https://www.linkedin.com/in/gargiulo-alessandro

WELCOME TO INTERMEDIATE C#



WEEK 3 - AGENDA



RECAP OF CORE C# CONCEPTS

- Understand value types versus reference types and their implications.
- Review object-oriented principles including classes, structs, interfaces, and inheritance.
- Explore exception handling and patterns for robust error management.
- Collections, generics.
- OOP pillars and SOLID principles for maintainable code.

TYPES IN DEPTH

CLASS

- A class is a data structure that may contain data members (constants and fields), function members (methods, properties, events, indexers, operators, instance constructors, finalizers, and static constructors), and nested types. Class types support inheritance, a mechanism whereby a derived class can extend and specialize a base class.
- A type that is defined as a class is a reference type. At run time, when you declare a variable of a reference type, the variable contains the value null until you explicitly create an instance of the class by using the new operator, or assign it an object of a compatible type created elsewhere

```C#
//Declaring an object of type MyClass.
MyClass mc = new MyClass();

//Declaring another object of the same type, assigning it the value of the first object.
MyClass mc2 = mc;
```

- When the object is created, enough memory is allocated on the managed heap for that specific object, and the variable holds only a reference to the location of said object. The memory used by an object is reclaimed by the automatic memory management functionality of the *CLR*, which is known as *garbage collection*.

CREATING OBJECTS

- Although they're sometimes used interchangeably, a class and an object are different things. A class defines a type of object, but it isn't an object itself. An object is a concrete entity based on a class, and is sometimes referred to as an instance of a class.
- When an instance of a class is created, a reference to the object is passed back to the programmer.
- We don't recommend creating object references that don't refer to an object because trying to access an object through such a reference fails at run time. A reference can refer to an object, either by creating a new object, or by assigning it an existing object.

```
C#

Customer object3 = new Customer();
Customer object4 = object3;
```

- This code creates two object references that both refer to the same object. Therefore, any changes to the object made through object3 are reflected in subsequent uses of object4. Because objects that are based on classes are referred to by reference, classes are known as reference types.

STRUCT

Keyword	Aliased type
sbyte	System.SByte
byte	System.Byte
short	System.Int16
ushort	System.UInt16
int	System.Int32
uint	System.UInt32
long	System.Int64
ulong	System.UInt64
char	System.Char
float	System.Single
double	System.Double
bool	System.Boolean
decimal	System.Decimal

- A struct type is a value type that can declare constants, fields, methods, properties, events, indexers, operators, instance constructors, static constructors, and nested types.
- Structs are similar to classes in that they represent data structures that can contain data members and function members. However, unlike classes, structs are value types and do not require heap allocation. A variable of a struct type directly contains the data of the struct, whereas a variable of a class type contains a reference to the data, the latter known as an object.

CLASS VS STRUCT

Classes are particularly useful when designing complex (and large) data structures which must be manipulated or inherited.

Structs are particularly useful for small data structures that have value semantics. Complex numbers, points in a coordinate system, or key-value pairs in a dictionary are all good examples of structs. Key to these data structures is that they have few data members, that they do not require use of inheritance or reference semantics, rather they can be conveniently implemented using value semantics where assignment copies the value instead of the reference.

Reference types are allocated on the heap and garbage-collected, whereas value types are allocated either on the stack or inline in containing types and deallocated when the stack unwinds or when their containing type gets deallocated.

Reference type assignments copy the reference, whereas value type assignments copy the entire value. Therefore, assignments of large reference types are cheaper than assignments of large value types.

MUTABLE VS IMMUTABLE OBJECTS

- Mutable classes in C# are those whose state can be modified after they are instantiated. This means that the internal data of a mutable class can be changed, leading to potential side effects and challenges in maintaining the integrity of the object. Common examples of mutable classes include many collection types, where elements can be added, removed, or modified.
- In contrast, immutable classes are designed to be unchangeable once they are created. The state of an immutable class remains constant throughout its lifetime, promoting more predictable and thread-safe behavior. Strings in C# are a classic example of immutable classes.

```
public class MutablePerson
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

```
public class ImmutablePerson
{
    public string Name { get; }
    public int Age { get; }

    public ImmutablePerson (string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```

IMMUTABILITY

Benefits of Immutable Classes:

- **Thread Safety:** Immutable classes are inherently thread-safe, as their state cannot be modified after creation, eliminating the need for locks in concurrent programming scenarios.
- **Predictable Behavior:** With immutability, objects maintain a consistent state, reducing the chances of unexpected changes and bugs.
- **Easier Debugging:** Immutable classes simplify debugging since their state doesn't change, making it easier to trace and understand the program flow.

Drawbacks of Immutable Classes:

- **Performance Concerns:** Creating new instances for each modification can lead to performance overhead, especially when dealing with large objects.
- **Memory Usage:** Immutable classes may consume more memory due to the creation of multiple instances, potentially impacting the application's overall memory footprint.

TUPLE

The **tuples** feature provides concise syntax to group multiple data elements in a lightweight data structure.

You can't define methods in a tuple type, but you can use the methods provided by .NET.

Tuple types support equality operators **==** and **!=**.

Tuple types are **value** types; tuple elements are public fields. That makes tuples **mutable** value types.

One of the most common use cases of tuples is as a method return type. That is, instead of defining **out** method parameters, you can group method results in a **tuple** return type

TUPLE – DECLARATION AND INITIALIZATION

```
(double, int) t1 = (4.5, 3);
Console.WriteLine($"Tuple with elements {t1.Item1} and {t1.Item2}.");
// Output:
// Tuple with elements 4.5 and 3.

(double Sum, int Count) t2 = (4.5, 3);
Console.WriteLine($"Sum of {t2.Count} elements is {t2.Sum}.");
// Output:
// Sum of 3 elements is 4.5.
```

TUPLE – AS RETURN TYPE

```
(int min, int max) FindMinMax(int[] input)
{
    if (input is null || input.Length == 0)
    {
        throw new ArgumentException("Cannot find minimum and maximum of a null or empty array.");
    }

    // Initialize min to MaxValue so every value in the input
    // is less than this initial value.
    var min = int.MaxValue;
    // Initialize max to MinValue so every value in the input
    // is greater than this initial value.
    var max = int.MinValue;
    foreach (var i in input)
    {
        if (i < min)
        {
            min = i;
        }
        if (i > max)
        {
            max = i;
        }
    }
    return (min, max);
}
```

INTRODUCING RECORDS

- A record in C# is a class or struct that provides special syntax and behavior for working with data models. The record modifier instructs the compiler to synthesize members that are useful for types whose primary role is storing data. These members include an overload of ToString() and members that support value equality.
- It is available since C# v9.0 released in November 2020 alongside .NET5
- Consider using a record in place of a class or struct in the following scenarios:
  - You want to define a data model that depends on value equality.
  - You want to define a type for which objects are immutable.

```
public record Person(string FirstName, string LastName);

public static class Program
{
    public static void Main()
    {
        Person person = new("Nancy", "Davolio");
        Console.WriteLine(person);
        // output: Person { FirstName = Nancy, LastName = Davolio }
    }
}
```

RECORDS

Value equality

For records, value equality means that two variables of a record type are equal if the types match and all property and field values compare equal. For other reference types such as classes, equality means reference equality by default, unless value equality was implemented. That is, two variables of a class type are equal if they refer to the same object. Methods and operators that determine equality of two record instances use value equality.

Immutability

An immutable type is one that prevents you from changing any property or field values of an object after it's instantiated. Immutability can be useful when you need a type to be thread-safe or you're depending on a hash code remaining the same in a hash table. Records provide concise syntax for creating and working with immutable types.

RECORDS

Records are primarily intended for supporting immutable data models even if they can be mutable.

The record type offers the following features:

- Concise syntax for creating a reference type with immutable properties
- Built-in behavior useful for a data-centric reference type:
  - Value equality
  - Concise syntax for nondestructive mutation
  - Built-in formatting for display

RECORDS IMMUTABILITY

```C#
public record Person(string FirstName, string LastName, string[] PhoneNumbers);

public static void Main()
{
    Person person = new("Nancy", "Davolio", new string[1] { "555-1234" });
    Console.WriteLine(person.PhoneNumbers[0]); // output: 555-1234

    person.PhoneNumbers[0] = "555-6789";
    Console.WriteLine(person.PhoneNumbers[0]); // output: 555-6789
}
```

RECORDS VALUE EQUALITY

```C#
public record Person(string FirstName, string LastName, string[] PhoneNumbers);

public static void Main()
{
    var phoneNumbers = new string[2];
    Person person1 = new("Nancy", "Davolio", phoneNumbers);
    Person person2 = new("Nancy", "Davolio", phoneNumbers);
    Console.WriteLine(person1 == person2); // output: True

    person1.PhoneNumbers[0] = "555-1234";
    Console.WriteLine(person1 == person2); // output: True

    Console.WriteLine(ReferenceEquals(person1, person2)); // output: False
}
```

RECORDS NONDESTRUCTIVE MUTATION

```C#
public record Person(string FirstName, string LastName)
{
    public string[] PhoneNumbers { get; init; }
}

public static void Main()
{
    Person person1 = new("Nancy", "Davolio") { PhoneNumbers = new string[1] };
    Console.WriteLine(person1);
    // output: Person { FirstName = Nancy, LastName = Davolio, PhoneNumbers = System.String[] }

    Person person2 = person1 with { FirstName = "John" };
    Console.WriteLine(person2);
    // output: Person { FirstName = John, LastName = Davolio, PhoneNumbers = System.String[] }
    Console.WriteLine(person1 == person2); // output: False

    person2 = person1 with { PhoneNumbers = new string[1] };
    Console.WriteLine(person2);
    // output: Person { FirstName = Nancy, LastName = Davolio, PhoneNumbers = System.String[] }
    Console.WriteLine(person1 == person2); // output: False

    person2 = person1 with { };
    Console.WriteLine(person1 == person2); // output: True
}
```

RECORDS FORMATTING DISPLAY

```
Person { FirstName = Nancy, LastName = Davolio, ChildNames = System.String[] }
```

C#

```
public override string ToString()
{
    StringBuilder stringBuilder = new StringBuilder();
    stringBuilder.Append("Teacher"); // type name
    stringBuilder.Append(" { ");
    if (PrintMembers(stringBuilder))
    {
        stringBuilder.Append(" ");
    }
    stringBuilder.Append("}");
    return stringBuilder.ToString();
}
```

RECORDS: DEMO

CONSTANTS

- A constant is a special variable which value cannot be modified during the execution of the program.
- For the constants, declaration and initialization must be performed **one time** and **one shot**.
- Only the C# built-in types may be declared as const. Reference type constants other than **String** can only be initialized with a null value. User-defined types, including classes, structs, and arrays, cannot be const.
- The trick behind a constant: when the compiler encounters a constant identifier in C# source code, it substitutes the literal value directly into the intermediate language (IL) code that it produces

EXAMPLES

```
// Declaration #1
int exampleIntVar;

// Declaration #2
var exampleVar = 1;

// Initialization
int exampleIntVar2 = 2;

// Assignation
exampleVar = 3;

// Constant
const double PI = 3.14;
```

READONLY

- Use the readonly modifier to create a class, struct, or array that is initialized one time at run time (for example in a constructor) and thereafter cannot be changed.
- In a field declaration, readonly indicates that assignment to the field can only occur as part of the declaration or in a constructor in the same class. A readonly field can be assigned and reassigned multiple times within the field declaration and constructor.

```C#
class Age
{
    private readonly int _year;
    Age(int year)
    {
        _year = year;
    }
    void ChangeYear()
    {
        //_year = 1967; // Compile error if uncommented.
    }
}
```

QUESTIONS TIME

ATTRIBUTES

28

ATTRIBUTES

Attributes are used in C# to convey declarative information or metadata about various code elements such as methods, assemblies, properties, types, etc. Attributes are added to the code by using a declarative tag that is placed using square brackets ([ ]) on top of the required code element

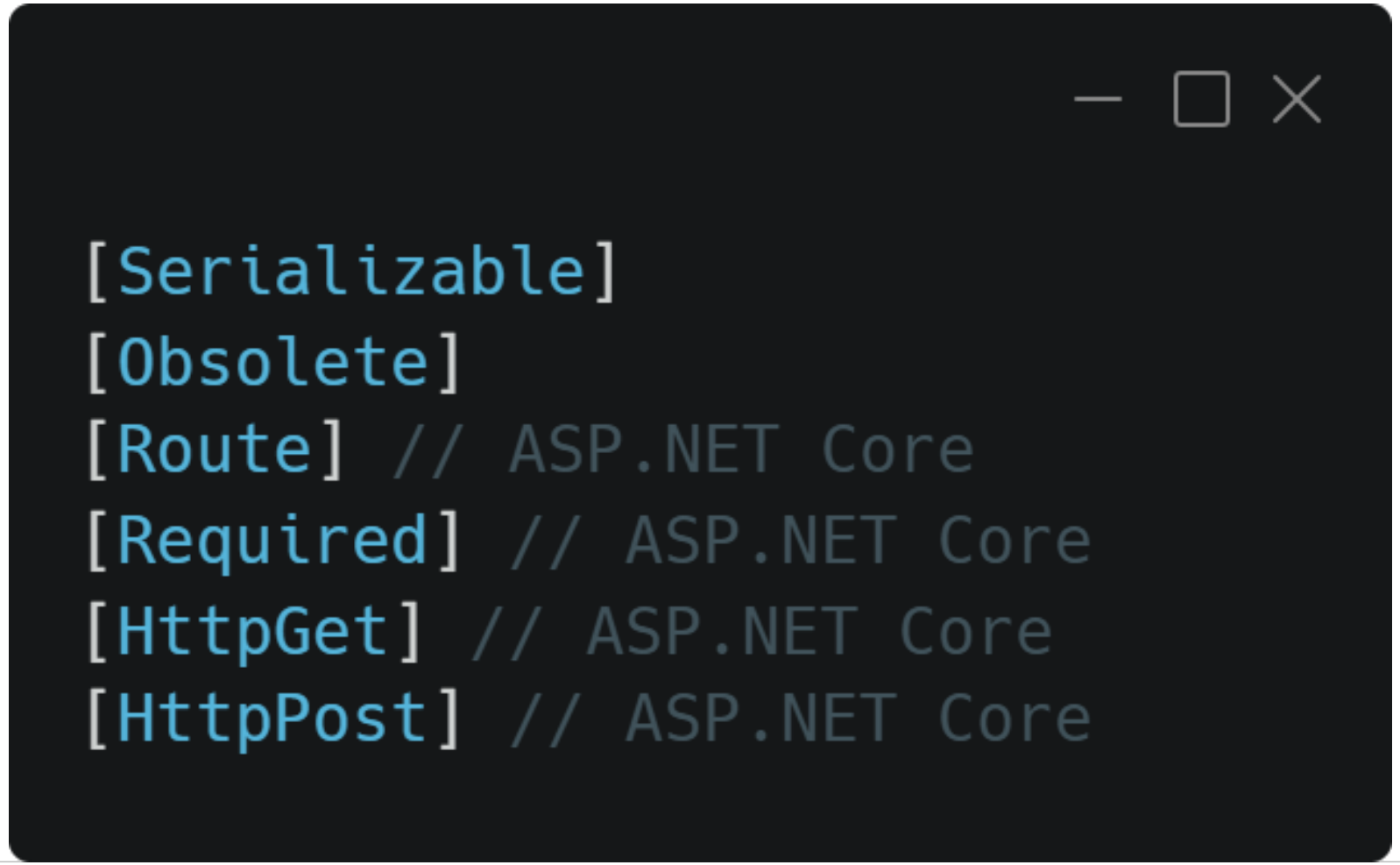
- Attributes can have arguments just like methods, properties, etc. can have arguments.
- Attributes can have zero or more parameters.
- Different code elements such as methods, assemblies, properties, types, etc. can have one or multiple attributes.
- Reflection can be used to obtain the metadata of the program by accessing the attributes at run-time.
- Attributes are generally derived from the **System. Attribute** Class.

Why using them?

- Provide additional information to the compiler or runtime.
- Control behavior in frameworks (e.g., ASP.NET, Entity Framework).
- Enable declarative programming, where behavior is defined by annotations rather than imperative code.
- You can create your own Attribute

Attributes are generally read by using reflection.

ATTRIBUTES



ATTRIBUTES

```
[Serializable]
public class Person(string FirstName, string LastName)
{
    [NonSerialized]
    private DateTime dateOfBirth;

    [Obsolete("This method is depricated, use GetInitialsWithConcat instead")]
    public string GetInitials()
    {
        return FirstName[0] + "" + LastName[0];
    }

    public string GetInitialsWithConcat()
    {
        return string.Concat(FirstName[0], LastName[0]);
    }
}
```

QUESTIONS TIME

DELEGATES AND EVENTS

DELEGATE: THE CONCEPT

- Delegates provide a late binding mechanism in .NET. Late Binding means that you create an algorithm where the caller also supplies at least one method that implements part of the algorithm.
- For example, consider sorting a list of stars in an astronomy application. You may choose to sort those stars by their distance from the earth, or the magnitude of the star, or their perceived brightness.
- In all those cases, the Sort() method does essentially the same thing: arranges the items in the list based on some comparison. The code that compares two stars is different for each of the sort orderings.
- These kinds of solutions have been used in software for half a century. The C# language delegate concept provides first class language support, and type safety around the concept.
- As you'll see later in this series, the C# code you write for algorithms like this is type safe. The compiler ensures that the types match for arguments and return types.

DELEGATE: WHAT ARE AND HOW TO DEFINE THEM

- Think of a delegate as a way to store a reference to a method, similar to how you might store a reference to an object. Just as you can pass objects to methods, you can pass method references using delegates. This is useful when you want to write flexible code where different methods can be "plugged in" to provide different behaviors.
- For example, imagine you have a calculator that can perform operations on two numbers. Instead of hard-coding addition, subtraction, multiplication, and division into separate methods, you could use delegates to represent any operation that takes two numbers and returns a result.
- When you define a delegate type, you're essentially creating a template that describes what kind of methods can be stored in that delegate.
- You define a delegate type using syntax that looks similar to a method signature, but with the delegate keyword at the beginning

```
// Define a simple delegate that can point to methods taking two integers and returning an integer
public delegate int Calculator(int x, int y);
```

DELEGATE: USAGE

- After defining the delegate type, you can create instances (variables) of that type. Think of this as creating a "slot" where you can store a reference to a method.
- Once you have a delegate instance that points to a method, you can call (invoke) that method through the delegate. You invoke the methods that are in the invocation list of a delegate by calling that delegate as if it were a method.

```
// From the .NET Core library
public delegate int Comparison<in T>(T left, T right);

// Inside a class definition:
public Comparison<T> comparator;

// Concrete instance of the comparator;
private static int CompareLength(string left, string right) =>
    left.Length.CompareTo(right.Length);

// Invoke the delegate
int result = comparator(left, right);
```

STRONGLY TYPED DELEGATES TO GENERIC

- The abstract Delegate class provides the infrastructure for loose coupling and invocation. Concrete Delegate types become much more useful by embracing and enforcing type safety for the methods that are added to the invocation list for a delegate object. When you use the delegate keyword and define a concrete delegate type, the compiler generates those methods.
- In practice, this would lead to creating new delegate types whenever you need a different method signature. This work could get tedious after a time. Every new feature requires new delegate types.
- Thankfully, this isn't necessary. The .NET Core framework contains several types that you can reuse whenever you need delegate types. These are generic definitions so you can declare customizations when you need new method declarations.

GENERIC DELEGATES – ACTION<T1, … ,T16>(T1, … ,T16)

```
using System;

public delegate void ShowValue();

public class Name
{
   private string instanceName;

   public Name(string name)
   {
      this.instanceName = name;
   }

   public void DisplayToConsole()
   {
      Console.WriteLine(this.instanceName);
   }
}

public class testTestDelegate
{
   public static void Main()
   {
      Name testName = new Name("Koani");
      ShowValue showMethod = testName.DisplayToConsole;
      showMethod();
   }
}
```

GENERIC DELEGATES – ACTION<T1, … ,T16>(T1, … ,T16)

```
using System;

public class Name
{
   private string instanceName;

   public Name(string name)
   {
      this.instanceName = name;
   }

   public void DisplayToConsole()
   {
      Console.WriteLine(this.instanceName);
   }
}

public class LambdaExpression
{
   public static void Main()
   {
      Name testName = new Name("Koani");
      Action showMethod = () => testName.DisplayToConsole();
      showMethod();
   }
}
```

GENERIC DELEGATES – FUNC<T1, … ,T16, TRESULT>(T1, … ,T16)

```
double pow(double x) { return x * x; }

Func<double, double> func1 = pow;

double res = func1(4);
```

LAMBDA EXPRESSION

- You use a lambda expression to create an anonymous function. Use the lambda declaration operator => to separate the lambda's parameter list from its body.
- To create a lambda expression, you specify input parameters (if any) on the left side of the lambda operator and an expression or a statement block on the other side.
- Any lambda expression can be converted to a **delegate** type. The types of its parameters and return value define the delegate type to which a lambda expression can be converted. If a lambda expression doesn't return a value, it can be converted to one of the Action delegate types; otherwise, it can be converted to one of the Func delegate types. For example, a lambda expression that has two parameters and returns no value can be converted to an **Action<T1,T2>** delegate. A lambda expression that has one parameter and returns a value can be converted to a **Func<T,TResult>** delegate

```
Func<int, int> square = x => x * x;
Console.WriteLine(square(5));
// Output:
// 25
```

QUESTIONS TIME

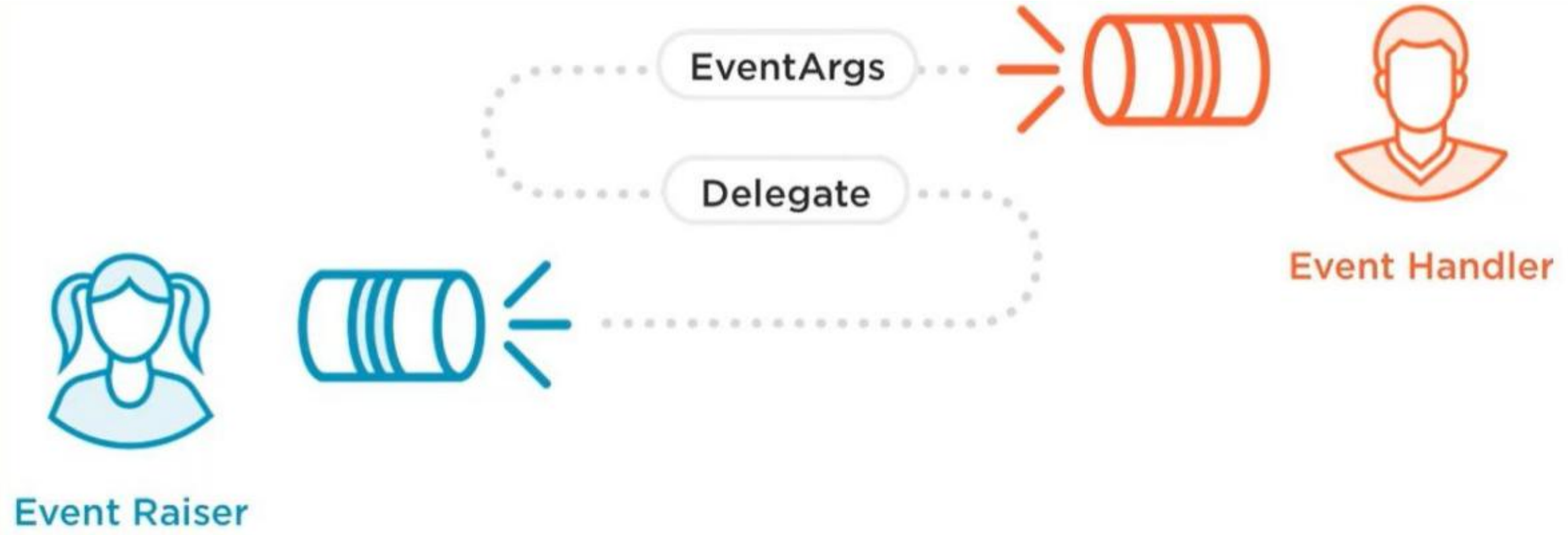
DELEGATE: DEMO

EVENTS

EVENTS

- Events are, like delegates, a *late binding* mechanism. In fact, events are built on the language support for delegates.
- Events are a way for an object to broadcast (to all interested components in the system) that something happened. Any other component can subscribe to the event and be notified when an event is raised.
- You probably used events in some of your programming. Many graphical systems have an event model to report user interaction. These events would report mouse movement, button presses, and similar interactions. That's one of the most common, but not the only scenario where events are used.
- You can define events that should be raised for your classes. One important consideration when working with events is that there might not be any object registered for a particular event. You must write your code so that it doesn't raise events when no listeners are configured.
- Subscribing to an event also creates a coupling between two objects (the event source, and the event sink). You need to ensure that the event sink unsubscribes from the event source when no longer interested in events.

EVENTS



EVENTS: HOW TO

- You use the event keyword to define an event
- The type of the event must be a delegate type. There are conventions that you should follow when declaring an event. Typically, the event delegate type has a void return. Event declarations should be a verb, or a verb phrase. Use past tense when the event reports something that happened. Use a present tense verb (for example, Closing) to report something that is about to happen. Often, using present tense indicates that your class supports some kind of customization behavior.
- When you want to raise the event, you call the event handlers using the delegate invocation syntax (using the ? Operator makes it easy to ensure that you don't attempt to raise the event when there are no subscribers to that event)

EVENTS – HOW TO

```
// Declaring the event
public event EventHandler<FileFoundArgs>? FileFound;

// Invokation syntax
FileFound?.Invoke(this, new FileFoundArgs(file));

// Event handling - Define your own custom behavior
EventHandler<FileFoundArgs> onFileFound = (sender, eventArgs) =>
{
    Console.WriteLine(eventArgs.FoundFile);
    filesFound++;
};

// Event subscription
fileLister.FileFound += onFileFound;

// To unsubscribe
fileLister.FileFound -= onFileFound;
```

EVENTS: DEMO

QUESTIONS TIME

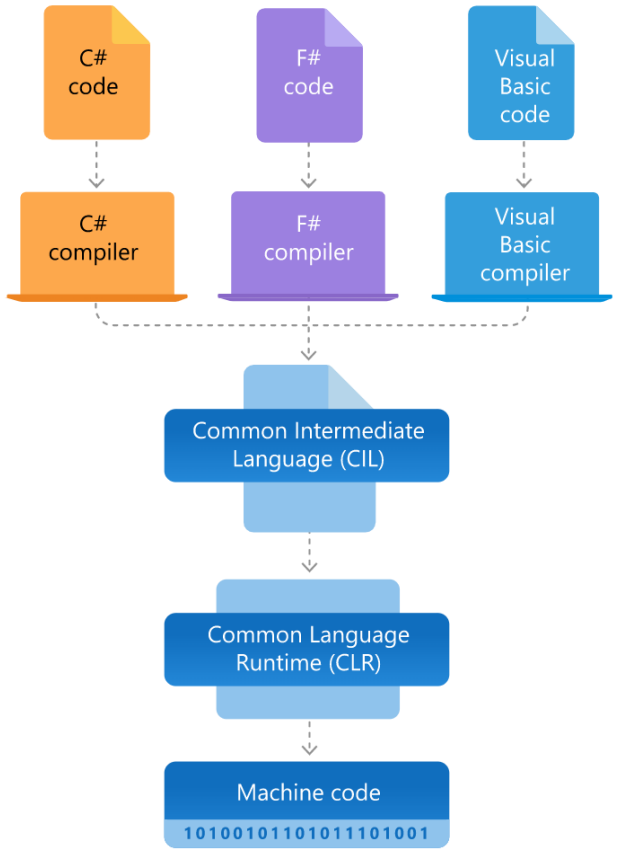
COMMON LANGUAGE RUNTIME

COMMON LANGUAGE RUNTIME

- .NET provides a run-time environment called the common language runtime that runs the code and provides services that make the development process easier.
- Compilers and tools expose the common language runtime's functionality and enable you to write code that benefits from the managed execution environment. Code that you develop with a language compiler that targets the runtime is called **managed code**. Managed code benefits from features such as **cross-language integration, cross-language exception handling, enhanced security, versioning and deployment support**, a simplified model for component interaction, and debugging and profiling services.
- The common language runtime makes it easy to design components and applications whose objects interact across **languages**. Objects written in different languages can communicate with each other, and their behaviors can be tightly integrated. For example, you can define a class and then use a different language to derive a class from your original class or call a method on the original class. You can also pass an instance of a class to a method of a class written in a different language. This cross-language integration is possible because language compilers and tools that target the runtime use a common type system defined by the runtime. They follow the runtime's rules for defining new types and for creating, using, persisting, and binding to types.

.NET COMPILATION AND EXECUTION (1/2)

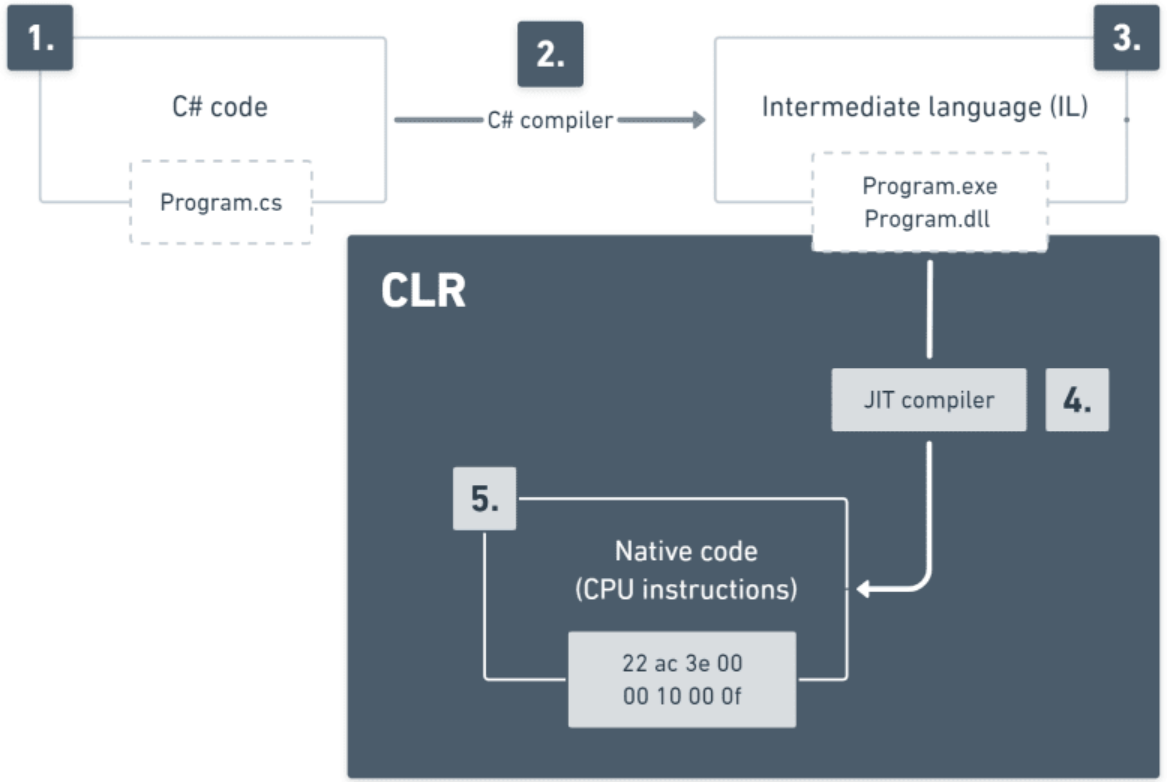
- **Common Language Infrastructure (CLI)**: an open specification of the run time environment (virtual machine component) of .NET/.NET Framework
- **Common Intermediate Language (CIL)**: an intermediate language that can be executed using any implementation of the CLI
- **Common Language Runtime (CLR)**: the virtual machine implementation of the CLI made by Microsoft



.NET COMPILATION AND EXECUTION (2/2)

1. A developer writes C# code
2. C# compiler checks the syntax and analyzes the source code. Before compilation starts, a **NuGet** restore took place.
3. Microsoft intermediate languages (MSIL) is generated as a result (EXE, DLL). Common output directories are Bin/Debug or Bin/Release
4. CLR gets initialized inside of a process and runs entry point method (Main)
5. MSIL gets converted to native code by the JIT compiler

At the end, native code can be executed by the host machine.



JIT (JUST-IN-TIME) VS AOT (AHEAD-OF-TIME)

- Publishing your app as native AOT produces an app that is self-contained and that has been ahead-of-time (AOT) compiled to native code. Native AOT apps start up very quickly and use less memory. Users of the application can run it on a machine that doesn't have the .NET runtime installed.
- The native AOT deployment model uses an ahead of time compiler to compile IL to native code at the time of publish. Native AOT apps don't use a Just-In-Time (JIT) compiler when the application runs. Native AOT apps can run in restricted environments where a JIT is not allowed. Native AOT applications target a specific runtime environment, such as Linux x64 or Windows x64, just like publishing a self-contained app.
- There are some limitations:
  - No dynamic loading (for example, Assembly.LoadFile)
  - No runtime code generation (for example, System.Reflection.Emit)
  - No C++/CLI
  - No built-in COM (only applies to Windows)

CLR BENEFITS

- **Memory management**: it reserve for you the right amount of memory to store your data. This operation can be quite simple for simple data types, but think for dynamic data structures (e.g. lists)
- **Security boundaries**: the runtime defines the boundaries of your and other applications, meaning that other applications can access your data or “stole” your reserved resources
- **Type safety**: the objects stored are characterized by their type, which defines also the available ways to access them.
- **Exception handling**: if you write a statement which cause an error at runtime, the CLR grab the errors details for you and raise an exception. Your job is to handle these exception (or to ignore deliberately)
- **Garbage collection**: all the object which are not useful anymore for the execution are deallocated from the memory so that you can reserve more space for your task objects
- **Performance improvements**: CLR try to understand situations in which it can speed up the performance

GARBAGE COLLECTION

GARBAGE COLLECTION

GARBAGE COLLECTION

.NET's garbage collector manages the allocation and release of memory for your application. Each time you create a new object, the common language runtime allocates memory for the object from the managed heap. As long as address space is available in the managed heap, the runtime continues to allocate space for new objects. However, memory is not infinite. Eventually the garbage collector must perform a collection in order to free some memory. The garbage collector's optimizing engine determines the best time to perform a collection, based upon the allocations being made. When the garbage collector performs a collection, it checks for objects in the managed heap that are no longer being used by the application and performs the necessary operations to reclaim their memory.

GARBAGE COLLECTION

In the common language runtime (CLR), the garbage collector (GC) serves as an automatic memory manager. The garbage collector manages the allocation and release of memory for an application. Therefore, developers working with managed code don't have to write code to perform memory management tasks. Automatic memory management can eliminate common problems such as forgetting to free an object and causing a memory leak or attempting to access freed memory for an object that's already been freed. The garbage collector provides the following benefits:

- Frees developers from having to manually release memory.
- Allocates objects on the managed heap efficiently.
- Reclaims objects that are no longer being used, clears their memory, and keeps the memory available for future allocations.
- Provides memory safety by making sure that an object can't use for itself the memory allocated for another object.

GARBAGE COLLECTION

The garbage collector's optimizing engine determines the best time to perform a collection based on the allocations being made. When the garbage collector performs a collection, it releases the memory for objects that are no longer being used by the application. It determines which objects are no longer being used by examining the application's roots. An application's roots include static fields, local variables on a thread's stack, CPU registers, GC handles, and the finalize queue. Each root either refers to an object on the managed heap or is set to null. The garbage collector can ask the rest of the runtime for these roots. The garbage collector uses this list to create a graph that contains all the objects that are reachable from the roots.

Objects that aren't in the graph are unreachable from the application's roots. The garbage collector considers unreachable objects garbage and releases the memory allocated for them. During a collection, the garbage collector examines the managed heap, looking for the blocks of address space occupied by unreachable objects. As it discovers each unreachable object, it uses a memory-copying function to compact the reachable objects in memory, freeing up the blocks of address spaces allocated to unreachable objects.

GARBAGE COLLECTION: WHEN?

The system has low physical memory. The memory size is detected by either the low memory notification from the operating system or low memory as indicated by the host.

The memory that's used by allocated objects on the managed heap surpasses an acceptable threshold. This threshold is continuously adjusted as the process runs.

The GC.Collect method is called. In almost all cases, you don't have to call this method because the garbage collector runs continuously. This method is primarily used for unique situations and testing.

GARBAGE COLLECTION: GENERATIONS

Garbage collection primarily occurs with the reclamation of short-lived objects. To optimize the performance of the garbage collector, the managed heap is divided into three generations, 0, 1, and 2, so it can handle long-lived and short-lived objects separately. The garbage collector stores new objects in generation 0. Objects created early in the application's lifetime that survive collections are promoted and stored in generations 1 and 2. Because it's faster to compact a portion of the managed heap than the entire heap, this scheme allows the garbage collector to release the memory in a specific generation rather than release the memory for the entire managed heap each time it performs a collection.

- **Generation 0**: This generation is the youngest and contains short-lived objects. An example of a short-lived object is a temporary variable. Garbage collection occurs most frequently in this generation.
- **Generation 1**: This generation contains short-lived objects and serves as a buffer between short-lived objects and long-lived objects.
- **Generation 2**: This generation contains long-lived objects. An example of a long-lived object is an object in a server application that contains static data that's live for the duration of the process.
- **Generation 3 (LOH)**: Large objects go on the large object heap (LOH), which is sometimes referred to as generation 3. Generation 3 is a physical generation that's logically collected as part of generation 2

GARBAGE COLLECTION: GENERATIONS

Garbage collections occur in specific generations as conditions warrant. Collecting a generation means collecting objects in that generation and all its younger generations. A generation 2 garbage collection is also known as a full garbage collection because it reclaims objects in all generations (that is, all objects in the managed heap).

QUESTIONS TIME

DISPOSE

- The .NET garbage collector does not allocate or release unmanaged memory. The pattern for disposing an object, referred to as the dispose pattern, imposes order on the lifetime of an object. The dispose pattern is used for objects that implement the IDisposable interface and is common when interacting with file and registry handles, wait handles, or pointers to blocks of unmanaged memory. This is because the garbage collector is unable to reclaim unmanaged objects.
- To help ensure that resources are always cleaned up appropriately, a Dispose method should be idempotent, such that it is callable multiple times without throwing an exception. Furthermore, subsequent invocations of Dispose should do nothing.

DISPOSE

If your application is using an expensive external resource, we also recommend that you provide a way to explicitly release the resource before the garbage collector frees the object. To release the resource, implement a Dispose method from the IDisposable interface that performs the necessary cleanup for the object. This can considerably improve the performance of the application. Even with this explicit control over resources, the finalizer becomes a safeguard to clean up resources if the call to the Dispose method fails.

DISPOSE

Because the public, non-virtual, parameterless Dispose method is called when it is no longer needed (by a consumer of the type), its purpose is to free unmanaged resources, perform general cleanup, and to indicate that the finalizer, if one is present, doesn't have to run. Freeing the actual memory associated with a managed object is always the domain of the garbage collector. Because of this, it has a standard implementation

```
public void Dispose()
{
    // Dispose of unmanaged resources.
    Dispose(true);
    // Suppress finalization.
    GC.SuppressFinalize(this);
}
```

THE DISPOSE(BOOL) METHOD OVERLOAD

In the overload, the disposing parameter is a Boolean that indicates whether the method call comes from a Dispose method (its value is true) or from a finalizer (its value is false).

```
protected virtual void Dispose(bool disposing)
{
    if (_disposed)
    {
        return;
    }

    if (disposing)
    {
        // TODO: dispose managed state (managed objects).
    }

    // TODO: free unmanaged resources (unmanaged objects) and override a
finalizer below.
    // TODO: set large fields to null.

    _disposed = true;
}
```

THE STATEMENT “USING”

The **using** statement provides a convenient syntax that ensures the correct use of *IDisposable* objects. The **await using** statement ensures the correct use of *IAsyncDisposable* objects. The language supports asynchronous disposable types that implement the *System.IAsyncDisposable* interface.

```
using (FileStream writer = File.OpenRead("log.txt"))
{
    // Code
}

// From C# 8.0
using FileStream writer = File.OpenRead("log.txt");
// Code
```

IDISPOSABLE

```
// A base class that implements IDisposable.
// By implementing IDisposable, you are announcing that
// instances of this type allocate scarce resources.
public class MyResource: IDisposable
{
    // Pointer to an external unmanaged resource.
    private IntPtr handle;
    // Other managed resource this class uses.
    private Component component = new Component();
    // Track whether Dispose has been called.
    private bool disposed = false;

    // The class constructor.
    public MyResource(IntPtr handle)
    {
        this.handle = handle;
    }

    // Implement IDisposable.
    // Do not make this method virtual.
    // A derived class should not be able to override this method.
    public void Dispose()
    {
        Dispose(disposing: true);
        // This object will be cleaned up by the Dispose method.
        // Therefore, you should call GC.SuppressFinalize to
        // take this object off the finalization queue
        // and prevent finalization code for this object
        // from executing a second time.
        GC.SuppressFinalize(this);
    }
    ...
```

IDISPOSABLE

```
// Dispose(bool disposing) executes in two distinct scenarios.
// If disposing equals true, the method has been called directly
// or indirectly by a user's code. Managed and unmanaged resources
// can be disposed.
// If disposing equals false, the method has been called by the
// runtime from inside the finalizer and you should not reference
// other objects. Only unmanaged resources can be disposed.
protected virtual void Dispose(bool disposing)
{
    // Check to see if Dispose has already been called.
    if(!this.disposed)
    {
        // If disposing equals true, dispose all managed
        // and unmanaged resources.
        if(disposing)
        {
            // Dispose managed resources.
            component.Dispose();
        }

        // Call the appropriate methods to clean up
        // unmanaged resources here.
        // If disposing is false,
        // only the following code is executed.
        CloseHandle(handle);
        handle = IntPtr.Zero;

        // Note disposing has been done.
        disposed = true;
    }
}
```

THE TRICK BEHIND A USING STATEMENT

- The "trick" behind the using statement in C# is that it's syntactic sugar — the compiler transforms it into a try-finally block that ensures Dispose() is called even if an exception occurs.
- The "trick" is that using guarantees deterministic cleanup by wrapping the resource usage in a try-finally block, ensuring Dispose() is called no matter what — even if an exception is thrown inside the block.

```
using (var resource = new MyResource())
{
    // Use resource
}

////////// Compiled in //////////

var resource = new MyResource();
try
{
    // Use resource
}
finally
{
    if (resource != null)
        resource.Dispose();
}
```

QUESTIONS TIME

EXCEPTIONS

SOFTWARE ERRORS

What Is a Software Error?

- A software error—often called a bug—is a flaw in a program's code, design, or documentation that causes it to behave differently than expected. These errors can range from minor glitches to critical failures that crash entire systems or compromise security.

Why Do Software Errors Happen?

- Software errors can arise from many sources. Here are the most common causes:
  - *Human Mistakes*: most errors stem from simple human oversight—misunderstanding requirements, typos, or incorrect logic.
  - *Coding Errors*: these include syntax mistakes, incorrect variable usage, or failure to follow coding standards.
  - *Inadequate Testing*: skipping or rushing tests can leave bugs undetected, which then surface in production.
  - *External Dependencies*: reliance on third-party libraries or APIs can introduce errors if those components fail or change unexpectedly.

Why Should We Avoid Software Errors?

SOFTWARE ERRORS

...IN THE PAST (ARIANE 5)

On June 4, 1996, the European Space Agency (ESA) launched the Ariane 5 rocket from French Guiana. Just 37 seconds after liftoff, the rocket veered off course, broke apart, and self-destructed at an altitude of about 4 km. The failure destroyed a payload worth **$370 million** and delayed scientific research for years.

Root Cause: A Software Bug

- The failure was traced to a software error in the Inertial Reference System (IRS), which is responsible for determining the rocket's orientation and velocity.
- The IRS software attempted to convert a 64-bit floating-point number (representing horizontal velocity) into a 16-bit signed integer.
- This conversion caused an overflow because the value exceeded the maximum range of a 16-bit integer (±32,767).
- The overflow triggered a runtime exception that was not caught by the software.
- The system then shut down the primary IRS, and control was passed to the backup IRS, which was running the same software and also failed in the same way
- The software was reused from Ariane 4, where the velocity values never exceeded the 16-bit limit.

...IN THE PRESENT (GOOGLE CLOUD)

On June 12, 2025, a significant portion of the internet experienced a sudden outage. What started as intermittent failures on Gmail and Spotify soon escalated into a global infrastructure meltdown. For millions of users and hundreds of companies, critical apps simply stopped working.

Among consumer platforms, the outage took down:

- **Spotify** (approximately 46,000 users reported on Downdetector).
- **Snapchat, Discord, Twitch, and Fitbit**: users were unable to stream, chat, or sync their data.
- **Google Workspace apps** (including Gmail, Calendar, Meet, and Docs). These apps power daily workflows for hundreds of millions of users.

The Faulty Feature: On May 29, 2025, Google introduced a new feature into the Service Control system. This feature added support for more advanced quota policy checks, allowing finer-grained control over how quota limits are applied.

The feature was rolled out across regions in a staged manner. However, it contained a bug that introduced a null pointer vulnerability in a new code path that was never exercised during rollout. The feature relied on a specific type of policy input to activate. Because that input had not yet been introduced during testing, the bug went undetected.

EXCEPTIONS

The C# language's exception handling features help you deal with any unexpected or exceptional situations that occur when a program is running. Exception handling uses the try, catch, and finally keywords to try actions that may not succeed, to handle failures when you decide that it's reasonable to do so, and to clean up resources afterward. Exceptions can be generated by the common language runtime (CLR), by .NET or third-party libraries, or by application code. Exceptions are created by using the throw keyword.

Reminder: do not handle business faulty scenarios as an exception! Exceptions should be used for “unintentional behaviors”. For instance, your software prompts the user for inserting his fiscal code. The condition for a wrong fiscal code format should not be intercepted by using an exception: it is an expected behavior.

Consider these 2 scenarios that any experienced developer can anticipate:

1. Business Errors – Anything the user / data might do which the business flow does not permit. Like any kind of form validation, filling in text inside a phone number form field, checking out with an empty cart, etc.
2. System Errors – Anything you ask from the OS and it might say no, things that are out of your control. Like, trying to access a file you don’t have permissions for.

EXCEPTIONS

- Exceptions are types that all ultimately derive from *System.Exception*.
- Use a **try** block around the statements that might throw exceptions.
- Once an exception occurs in the **try** block, the flow of control jumps to the first associated exception handler that is present anywhere in the call stack. In C#, the **catch** keyword is used to define an exception handler.
- If no exception handler for a given exception is present, the program stops executing with an error message.
- Don't catch an exception unless you can handle it and leave the application in a known state. If you catch System.Exception, rethrow it using the throw keyword at the end of the catch block.
- If a catch block defines an exception variable, you can use it to obtain more information about the type of exception that occurred.
- Exceptions can be explicitly generated by a program by using the **throw** keyword.
- Exception objects contain detailed information about the error, such as the state of the call stack and a text description of the error.
- Code in a **finally** block is executed regardless of if an exception is thrown. Use a finally block to release resources, for example to close any streams or files that were opened in the try block.

EXCEPTIONS

```
try
{
    // Code to try goes here.
}
catch (SomeSpecificException ex)
{
    // Code to handle the exception goes here.
}
finally
{
    // Code to execute after the try (and possibly catch) blocks
    // goes here.
}
```

EXCEPTIONS

```
using System;
using System.IO;

namespace Exceptions
{
    public class CatchOrder
    {
        public static void Main()
        {
            try
            {
                using (var sw = new StreamWriter("./test.txt"))
                {
                    sw.WriteLine("Hello");
                }
            }

            // Put the more specific exceptions first.
            catch (DirectoryNotFoundException ex)
            {
                Console.WriteLine(ex);
            }
            catch (FileNotFoundException ex)
            {
                Console.WriteLine(ex);
            }
            // Put the least specific exception last.
            catch (IOException ex)
            {
                Console.WriteLine(ex);
            }

            Console.WriteLine("Done");
        }
    }
}
```

EXCEPTIONS – CATCH BLOCK

- Multiple **catch** blocks with different exception classes can be chained together. The **catch** blocks are evaluated from top to bottom in your code, but only one catch block is executed for each exception that is thrown. The first **catch** block that specifies the exact type or a base class of the thrown exception is executed. If no **catch** block specifies a matching exception class, a **catch** block that doesn't have any type is selected, if one is present in the statement. It's important to position catch blocks with the most specific (that is, the most derived) exception classes first.
- Use a try/catch block when you have a good understanding of why the exception might be thrown, and you can implement a specific recovery, such as prompting the user to enter a new file name when you catch a *FileNotFoundException* object.
- User-filtered exception handlers **catch** and handle exceptions based on requirements you define for the **exception**. These handlers use the **catch** statement ***with*** the when keyword

USER-FILTERED EXCEPTION

```
try
{
    //Try statements.
}
catch (Exception ex) when (ex.Message.Contains("404"))
{
    //Catch statements.
}
```

HANDLING EXCEPTIONS IN DEPTH

```
try
{
    // Try to access a file.
}
catch (FileNotFoundException e)
{
    // Handle file not found
}
```

```
if (File.Exists("PATH//TO//FILE.ext)"))
{
  // Try to access a file.
}
else
{
  // Handle file not found
}
```

EXCEPTIONS – CATCH BLOCK

If the exception variable is declared in the catch statement, then it will carry many properties which help the user to get information about the exception

- **Data**: This property helps to get the information about the arbitrary data which is held by the property in the key-value pairs.
- **TargetSite**: This property helps to get the name of the method where the exception will throw.
- **Message**: This property helps to provide the details about the main cause of the exception occurrence.
- **HelpLink**: This property helps to hold the URL for a particular exception.
- **StackTrace**: This property helps to provide the information about where the error occurred.
- **InnerException**: This property helps to provide the information about the series of exceptions that might have occurred.

STACK TRACE

- A stack trace is a report that provides a snapshot of the call stack at a specific point in time, usually when an exception is thrown. It shows the sequence of method calls that led to the exception, starting with the most recent method and working backward to the initial call.

```
System.NullReferenceException: Object reference not set to an instance of an object.
   at ConsoleApp.Program.MethodA() in C:\projects\ConsoleApp\Program.cs:line 20
   at ConsoleApp.Program.Main(String[] args) in C:\projects\ConsoleApp\Program.cs:line 10
```

```
System.ApplicationException: outer
 ---> System.FormatException: inner
   at ConsoleApp.C.Z() in C:\projects\ConsoleApp\Program.cs:line 54
   at ConsoleApp.B.Y() in C:\projects\ConsoleApp\Program.cs:line 35
   --- End of inner exception stack trace ---
   at ConsoleApp.B.Y() in C:\projects\ConsoleApp\Program.cs:line 41
   at ConsoleApp.A.X() in C:\projects\ConsoleApp\Program.cs:line 24
   at ConsoleApp.Program.Main(String[] args) in C:\projects\ConsoleApp\Program.cs:line 11
```

EXCEPTIONS – RE-THROWING

We don’t always want to handle an exception and let the program continue after catching it. Sometimes we want the exception to move on and perhaps be caught in the next catch clause, but we want to log something or do any other action related to this exception occurrence. This is what is referred to as re-throwing an exception. We can do it by either using “throw” or “throw ex”.

The difference between them is that:

- **throw** preserves the stack trace (the stack trace will point to the method that caused the exception in the first place)
- **throw ex** does not preserve the stack trace (we will lose the information about the method that caused the exception in the first place. It will seem like the exception was thrown from the place of its catching and re-throwing)

EXCEPTIONS – FINALLY BLOCK

- A **finally** block enables you to clean up actions that are performed in a try block. If present, the finally block executes last, after the try block and any matched catch block. A finally block always runs, whether an exception is thrown or a catch block matching the exception type is found.
- The **finally** block can be used to release resources such as file streams, database connections, and graphics handles without waiting for the garbage collector in the runtime to finalize the objects.
- In the following example, the finally block is used to close a file that is opened in the try block. Notice that the state of the file handle is checked before the file is closed. If the try block can't open the file, the file handle still has the value null and the finally block doesn't try to close it. Instead, if the file is opened successfully in the try block, the finally block closes the open file.

EXCEPTIONS – FINALLY BLOCK

```
FileStream? file = null;
FileInfo fileinfo = new System.IO.FileInfo("./file.txt");
try
{
    file = fileinfo.OpenWrite();
    file.WriteByte(0xF);
}
finally
{
    // Check for null because OpenWrite might have failed.
    file?.Close();
}
```

COMMON EXCEPTION

Exception type	Description	Example
Exception	Base class for all exceptions.	None (use a derived class of this exception).
IndexOutOfRangeException	Thrown by the runtime only when an array is indexed improperly.	Indexing an array outside its valid range: arr[arr.Length+1]
NullReferenceException	Thrown by the runtime only when a null object is referenced.	object o = null; o.ToString();
InvalidOperationException	Thrown by methods when in an invalid state.	Calling Enumerator.MoveNext() after removing an item from the underlying collection.
ArgumentException	Base class for all argument exceptions.	None (use a derived class of this exception).
ArgumentNullException	Thrown by methods that do not allow an argument to be null.	String s = null; "Calculate".IndexOf(s);
ArgumentOutOfRangeException	Thrown by methods that verify that arguments are in a given range.	String s = "string"; s.Substring(s.Length+1);

CUSTOM EXCEPTIONS

Programs can throw a predefined exception class in the **System** namespace (except where previously noted) or create their own exception classes by deriving from **Exception**. The derived classes should define at least three constructors: one parameterless constructor, one that sets the message property, and one that sets both the **Message** and **InnerException** properties. Add new properties to the exception class when the data they provide is useful to resolving the exception.

```
public class InvalidDepartmentException : Exception
{
    public InvalidDepartmentException() : base() { }
    public InvalidDepartmentException(string message) : base(message) { }
    public InvalidDepartmentException(string message, Exception inner) : base(message, inner) { }
}
```

EXCEPTION BUILDER METHODS

- It's common for a class to throw the same exception from different places in its implementation. To avoid excessive code, create a helper method that creates the exception and returns it.
- Some key .NET exception types have such static throw helper methods that allocate and throw the exception. You should call these methods instead of constructing and throwing the corresponding exception type:
  - ArgumentNullException.ThrowIfNull
  - ArgumentException.ThrowIfNullOrEmpty(String, String)
  - ArgumentException.ThrowIfNullOrWhiteSpace(String, String)
  - ArgumentOutOfRangeException.ThrowIfZero<T>(T, String)
  - ArgumentOutOfRangeException.ThrowIfNegative<T>(T, String)
  - ArgumentOutOfRangeException.ThrowIfEqual<T>(T, T, String)
  - ArgumentOutOfRangeException.ThrowIfLessThan<T>(T, T, String)
  - ArgumentOutOfRangeException.ThrowIfNotEqual<T>(T, T, String)
  - ArgumentOutOfRangeException.ThrowIfNegativeOrZero<T>(T, String)
  - ArgumentOutOfRangeException.ThrowIfGreaterThan<T>(T, T, String)
  - ArgumentOutOfRangeException.ThrowIfLessThanOrEqual<T>(T, T, String)
  - ArgumentOutOfRangeException.ThrowIfGreaterThanOrEqual<T>(T, T, String)
  - ObjectDisposedException.ThrowIf

EXCEPTION BUILDER METHODS

```
class FileReader
{
    private readonly string _fileName;

    public FileReader(string path)
    {
        _fileName = path;
    }

    public byte[] Read(int bytes)
    {
        byte[] results = FileUtils.ReadFromFile(_fileName, bytes) ?? throw NewFileIOException();
        return results;
    }

    static FileReaderException NewFileIOException()
    {
        string description = "My NewFileIOException Description";

        return new FileReaderException(description);
    }
}
```

EXCEPTIONS – BEST PRACTICES

- When your code can't recover from an exception, don't catch that exception. Enable methods further up the call stack to recover if possible.
- Clean up resources that are allocated with either using statements or finally blocks. Prefer using statements to automatically clean up resources when exceptions are thrown. Use finally blocks to clean up resources that don't implement *IDisposable*.
- Call Try* methods to avoid exceptions. Some .NET library methods provide alternative forms of error handling. For example, ***Int32.Parse*** throws an **OverflowException** if the value to be parsed is too large to be represented by Int32. However, **Int32.TryParse** doesn't throw this exception. Instead, it returns a Boolean and has an **out** parameter that contains the parsed valid integer upon success.
- Introduce a new exception class only when a predefined one doesn't apply. For example:
  - If a property set or method call isn't appropriate given the object's current state, throw an InvalidOperationException exception.
  - If invalid parameters are passed, throw an ArgumentException exception or one of the predefined classes that derive from ArgumentException.

QUESTIONS TIME

COLLECTIONS

MOSTLY USED COLLECTION TYPES

- In collections based on IList or directly on ICollection, every element contains only a value. These types include:
  - List<T>
  - ImmutableList<T>
  - Queue <T>
  - Stack<T>
  - Span<T>
- In collections based on the IDictionary interface, every element contains both a key and a value. These types include:
  - SortedList<TKey,TValue>
  - Dictionary<TKey,TValue>
  - SortedDictionary<TKey,TValue>
  - ImmutableDictionary<TKey, TValue>

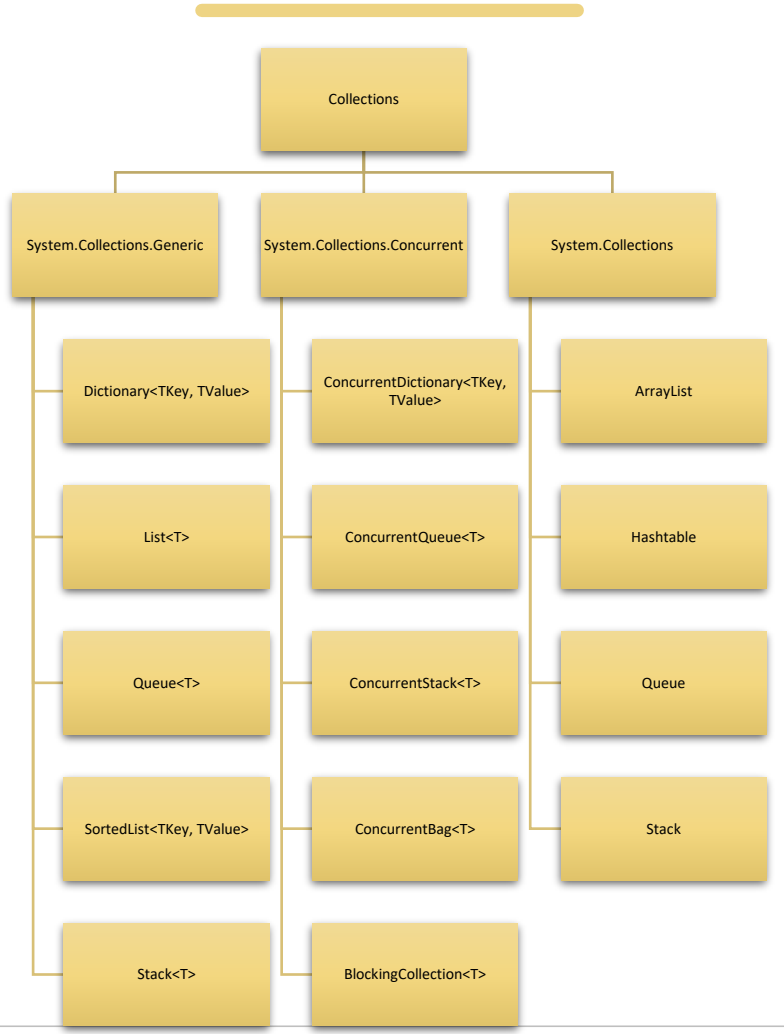
SELECTING A COLLECTION

- Do you need a sequential list where the element is typically discarded after its value is retrieved?
  - If yes, consider using the Queue<T> generic class if you need first-in, first-out (FIFO) behavior. Consider using the Stack<T> generic class if you need last-in, first-out (LIFO) behavior. For safe access from multiple threads, use the concurrent versions, ConcurrentQueue<T> and ConcurrentStack<T>. For immutability, consider the immutable versions, ImmutableQueue<T> and ImmutableStack<T>.
- Do you need to access each element by index?
  - The List<T> generic class offer access to their elements by the zero-based index of the element. For immutability, consider the immutable generic versions ImmutableList<T>.
  - The Dictionary<TKey,TValue> and SortedDictionary<TKey,TValue> generic classes offer access to their elements by the key of the element. Additionally, there are immutable versions of several corresponding types: ImmutableDictionary<TKey,TValue> and ImmutableSortedDictionary<TKey,TValue>.
- Do you need to sort the elements differently from how they were entered?
  - The Hashtable class sorts its elements by their hash codes.
  - The SortedList<TKey,TValue> and SortedDictionary<TKey,TValue> generic classes sort their elements by the key. The sort order is based on the implementation of the IComparer<T> generic interface for the SortedList<TKey,TValue> and SortedDictionary<TKey,TValue> generic classes. Of the two generic types, SortedDictionary<TKey,TValue> offers better performance than SortedList<TKey,TValue>, while SortedList<TKey,TValue> consumes less memory.

IMMUTABLE COLLECTIONS

- Belong to the System.Collections.Immutable Namespace
- Our well-know collection implemented as immutable
  - ImmutableList<T>
  - ImmutableQueue<T>
  - ImmutableStack<T>
  - ImmutableDictionary<T>
- With immutable collections, you can:
  - Share a collection in a way that its consumer can be assured that the collection never changes.
  - Provide implicit thread safety in multi-threaded applications (no locks required to access collections).
  - Modify a collection during enumeration, while ensuring that the original collection does not change.
  - The immutable collection classes are available as part of the core .NET libraries; however, they're not part of the core class library distributed with .NET Framework. For .NET Framework 4.6.2 and later apps, the classes are available through NuGet packages.

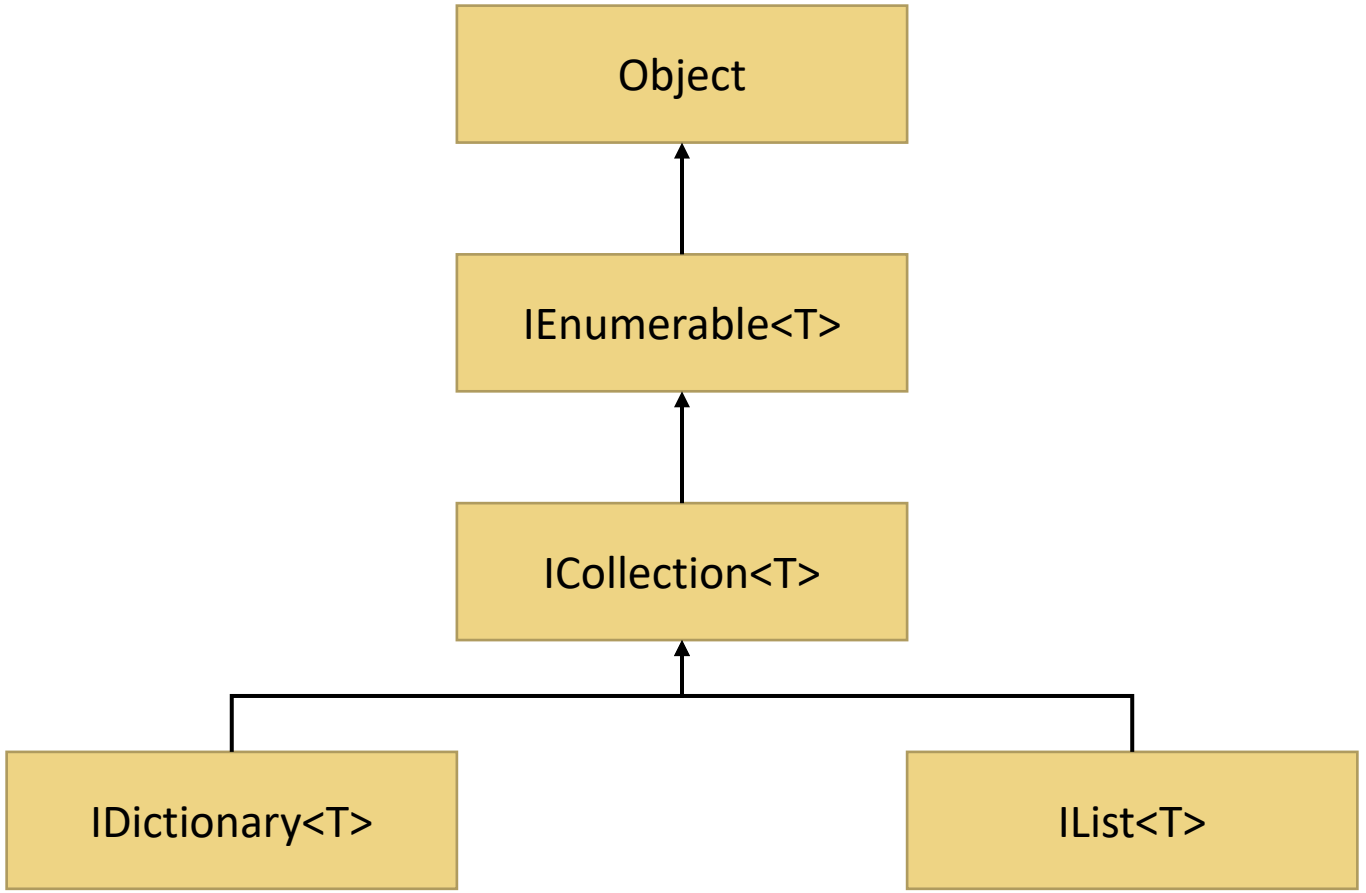
COLLECTION NAMESPACES



GENERIC COLLECTION INTERFACES

Interface	Description
IEnumerable<T>	Required by the foreach statement for iteration. This interface defines the method GetEnumerator, which returns an enumerator.
ICollection<T>	Implemented by generic collection classes. Allows to get the number of items, copy the entire collection, add and remove items.
IList<T>	Allows list to be accessed by the index and remove/add at specific positions.
IDictionary<TKey, TValue>	Implemented by collection that can be accessed by an indexer of type TKey.
IComparer<T>	Implemented by comparers and used to sort elements inside a collection.

GENERIC COLLECTION HIERARCHY



COLLECTIONS SUMMARY

Collection	Ordering	Contiguous Storage?	Direct Access?	Lookup Efficiency	Manipulate Efficiency	Notes
Dictionary	Unordered	Yes	Via Key	Key: O(1)	O(1)	Best for high performance lookups.
SortedDictionary	Sorted	No	Via Key	Key: O(log n)	O(log n)	Compromise of Dictionary speed and ordering, uses binary search tree.
SortedList	Sorted	Yes	Via Key	Key: O(log n)	O(n)	Very similar to SortedDictionary, except tree is implemented in an array, so has faster lookup on preloaded data, but slower loads.
List	User has precise control over element ordering	Yes	Via Index	Index: O(1) Value: O(n)	O(n)	Best for smaller lists where direct access required and no sorting.
LinkedList	User has precise control over element ordering	No	No	Value: O(n)	O(1)	Best for lists where inserting/deleting in middle is common and no direct access required.
HashSet	Unordered	Yes	Via Key	Key: O(1)	O(1)	Unique unordered collection, like a Dictionary except key and value are same object.
SortedSet	Sorted	No	Via Key	Key: O(log n)	O(log n)	Unique sorted collection, like SortedDictionary except key and value are same object.
Stack	LIFO	Yes	Only Top	Top: O(1)	O(1)*	Essentially same as List<T> except only process as LIFO
Queue	FIFO	Yes	Only Front	Front: O(1)	O(1)	Essentially same as List<T> except only process as FIFO

COMMONLY USED METHODS – LIST<T>

- Add(item) – Adds an item to the end.
- AddRange(collection) – Adds multiple items.
- Remove(item) – Removes the first occurrence.
- RemoveAt(index) – Removes item at a specific index.
- Insert(index, item) – Inserts item at index.
- Contains(item) – Checks if item exists.
- IndexOf(item) – Returns index of item.
- Sort() – Sorts the list.
- Reverse() – Reverses the list.
- Clear() – Removes all items.
- Find(predicate) – Finds first item matching condition.
- FindAll(predicate) – Returns all matching items.
- ForEach(action) – Executes action on each item.

COMMONLY USED METHODS – DICTIONARY<K,V>

- Add(key, value) – Adds a key-value pair.
- Remove(key) – Removes entry by key.
- ContainsKey(key) – Checks if key exists.
- ContainsValue(value) – Checks if value exists.
- TryGetValue(key, out value) – Safely gets value.
- Clear() – Removes all entries.
- Keys – Gets all keys.
- Values – Gets all values.

COMMONLY USED METHODS – HASHSET<T>

- Add(item) – Adds item if not present.
- Remove(item) – Removes item.
- Contains(item) – Checks if item exists.
- UnionWith(collection) – Adds elements from another set.
- IntersectWith(collection) – Keeps only common elements.
- ExceptWith(collection) – Removes elements in another set.
- Clear() – Removes all items.
- SetEquals(collection) – Checks if sets are equal.

COMMONLY USED METHODS – QUEUE<T>

- Enqueue(item) – Adds item to the end.
- Dequeue() – Removes and returns item from front.
- Peek() – Returns item at front without removing.
- Contains(item) – Checks if item exists.
- Clear() – Removes all items.

COMMONLY USED METHODS – STACK<T>

- Push(item) – Adds item to the top.
- Pop() – Removes and returns item from top.
- Peek() – Returns item at top without removing.
- Contains(item) – Checks if item exists.
- Clear() – Removes all items.

ICOMPARER<T> INTERFACE

The IComparer<T> interface in C# is part of the System.Collections.Generic namespace and is used to define a custom comparison logic for sorting or ordering generic collections like List<T>, SortedSet<T>, or Dictionary<TKey, TValue>.

IComparer<T> provides a way to compare two objects of type T. It is especially useful when you want to sort a collection based on custom rules (e.g., by using multiple fields or custom rules).

```
public interface IComparer<in T>
{
    int Compare(T x, T y);
}
```

Returns:

< 0 if x is less than y

0 if x equals y

> 0 if x is greater than y}

ICOMPARER<T> INTERFACE

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

```
public class AgeComparer : IComparer<Person>
{
    public int Compare(Person x, Person y)
    {
        if (x == null || y == null)
            throw new ArgumentException("Arguments cannot be null");

        return x.Age.CompareTo(y.Age);
    }
}
```

ICOMPARER<T> INTERFACE

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<Person> people = new List<Person>
        {
            new Person { Name = "Alice", Age = 30 },
            new Person { Name = "Bob", Age = 25 },
            new Person { Name = "Charlie", Age = 35 }
        };

        // Sort using AgeComparer
        people.Sort(new AgeComparer());

        foreach (var person in people)
        {
            Console.WriteLine($"{person.Name}, Age: {person.Age}");
        }
    }
}
```

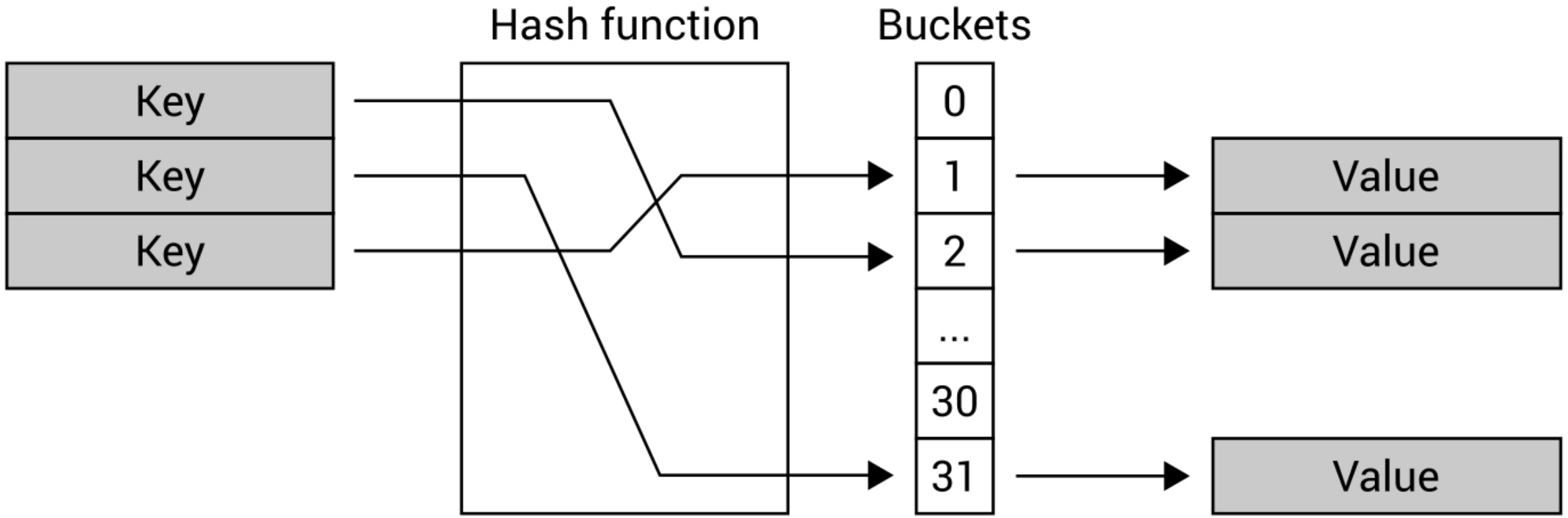
DICTIONARY<K,V> REMARKS

The **Dictionary<TKey,TValue>** generic class provides a mapping from a set of keys to a set of values. Each addition to the dictionary consists of a value and its associated key. Retrieving a value by using its key is very fast, close to O(1), because the Dictionary<TKey,TValue> class is implemented as a hash table.

As long as an object is used as a key in the Dictionary<TKey,TValue>, it must not change in any way that affects its hash value. Every key in a Dictionary<TKey,TValue> must be unique according to the dictionary's equality comparer. A key cannot be null, but a value can be, if its type TValue is a reference type.

Dictionary<TKey,TValue> requires an equality implementation to determine whether keys are equal. You can specify an implementation of the IEqualityComparer<T> generic interface by using a constructor that accepts a comparer parameter; if you do not specify an implementation, the default generic equality comparer EqualityComparer<T>.Default is used. If type TKey implements the System.IEquatable<T> generic interface, the default equality comparer uses that implementation.

DICTIONARY<K,V>



IEQUALITYCOMPARER<T>

This interface allows the implementation of customized equality comparison for collections. That is, you can create your own definition of equality for type **T**, and specify that this definition be used with a collection type that accepts the **IEqualityComparer<T>** generic interface. In the .NET Framework, constructors of the Dictionary<TKey,TValue> generic collection type accept this interface.

A default implementation of this interface is provided by the Default property of the EqualityComparer<T> generic class. T

This interface supports only equality comparisons. Customization of comparisons for sorting and ordering is provided by the IComparer<T> generic interface.

We recommend that you derive from the EqualityComparer<T> class instead of implementing the IEqualityComparer<T> interface, because the EqualityComparer<T> class tests for equality using the IEquatable<T>.Equals method instead of the Object.Equals method. This is consistent with the Contains, IndexOf, LastIndexOf, and Remove methods of the Dictionary<TKey,TValue> class and other generic collections.

IEQUALITYCOMPARER<T>

```
class Box
{
    public int Height { get; }
    public int Length { get; }
    public int Width { get; }

    public Box(int height, int length, int width)
    {
        Height = height;
        Length = length;
        Width = width;
    }

    public override string ToString() => $"({Height}, {Length}, {Width})";
}
```

IEQUALITYCOMPARER<T>

```
class BoxEqualityComparer : IEqualityComparer<Box>
{
    public bool Equals(Box? b1, Box? b2)
    {
        if (ReferenceEquals(b1, b2))
            return true;

        if (b2 is null || b1 is null)
            return false;

        return b1.Height == b2.Height
            && b1.Length == b2.Length
            && b1.Width == b2.Width;
    }

    public int GetHashCode(Box box) => box.Height ^ box.Length ^ box.Width;
}
```

IEQUALITYCOMPARER<T>

```
static void Main()
{
    BoxEqualityComparer comparer = new();

    Dictionary<Box, string> boxes = new(comparer);

    AddBox(new Box(4, 3, 4), "red");
    AddBox(new Box(4, 3, 4), "blue");
    AddBox(new Box(3, 4, 3), "green");

    Console.WriteLine($"The dictionary contains {boxes.Count} Box objects.");

    void AddBox(Box box, string name)
    {
        try
        {
            boxes.Add(box, name);
        }
        catch (ArgumentException e)
        {
            Console.WriteLine($"Unable to add {box}: {e.Message}");
        }
    }
}
```

SPAN<T>

Span<T> is a stack-only type that represents a contiguous region of arbitrary memory. It allows you to work with slices of arrays, strings, or unmanaged memory without allocating new memory. This makes it ideal for high-performance scenarios like parsing, serialization, and buffer manipulation.

Key Characteristics:

- No heap allocation: Span<T> lives on the stack, making it fast and memory-efficient.
- Safe slicing: You can create sub-spans without copying data.
- Type-safe: Works with any type T.
- Bounds-checked: Prevents out-of-range access.

SPAN<T> - EXAMPLE

```
int[] numbers = { 1, 2, 3, 4, 5 };
Span<int> slice = numbers.AsSpan(1, 3); // {2, 3, 4}
```

SPAN<T> - EXAMPLE

int[] numbers = { 1, 2, 3, 4, 5 };

- Allocated on the heap.
- Arrays in C# are reference types, so this array is stored in the managed heap.
- The variable numbers is a reference to that heap-allocated array.

Span<int> slice = numbers.AsSpan(1, 3);

- Allocated on the stack.
- Span<T> is a ref struct, which means it cannot be boxed or stored on the heap.

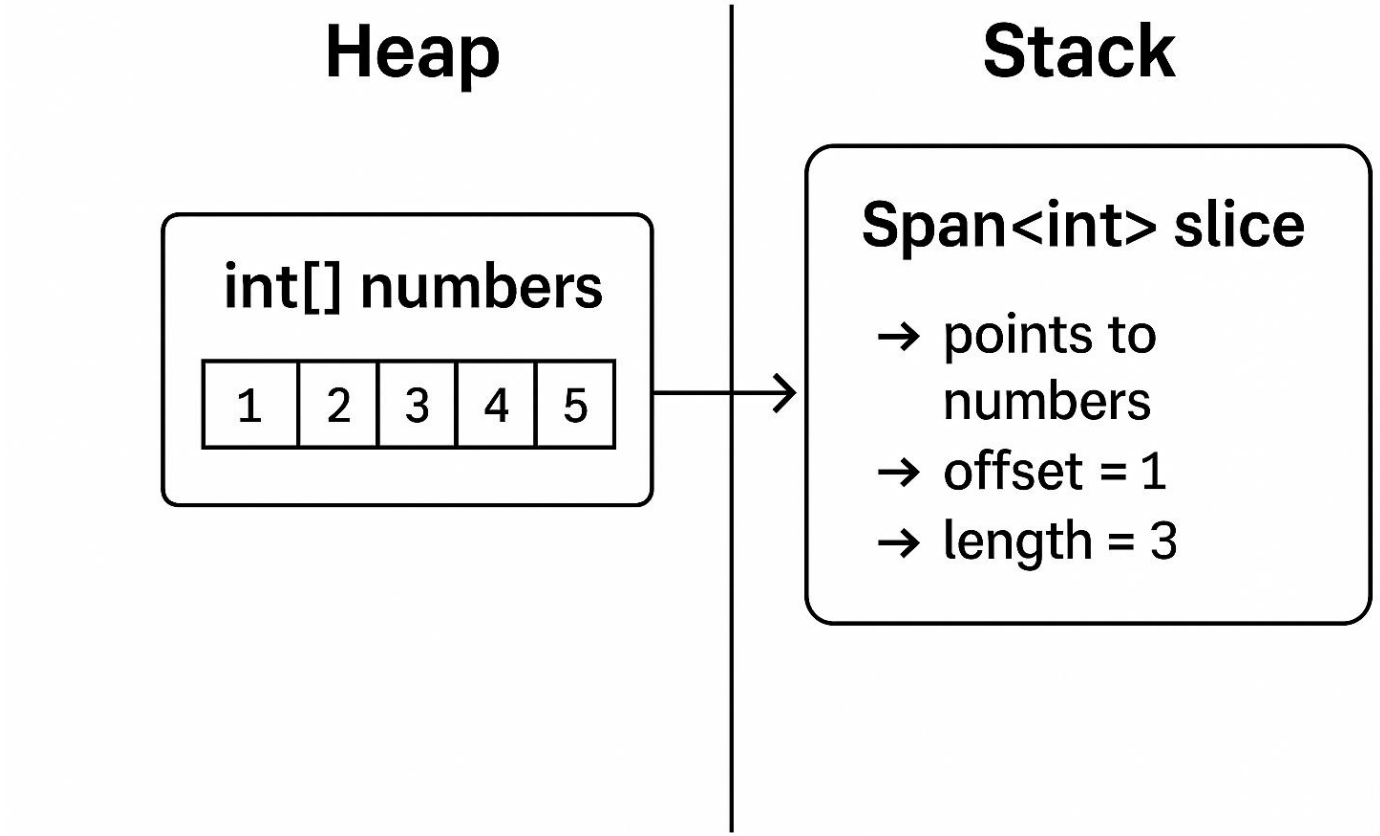
The slice variable contains:

1. A reference to the original array (numbers)
2. An offset (1)
3. A length (3)

Important Notes:

- No new array is created when slicing with Span<T>.
- The Span<T> is just a view into the existing array.
- The memory footprint is minimal and no garbage collection is involved for the span itself.

SPAN<T> - EXAMPLE



QUESTIONS TIME

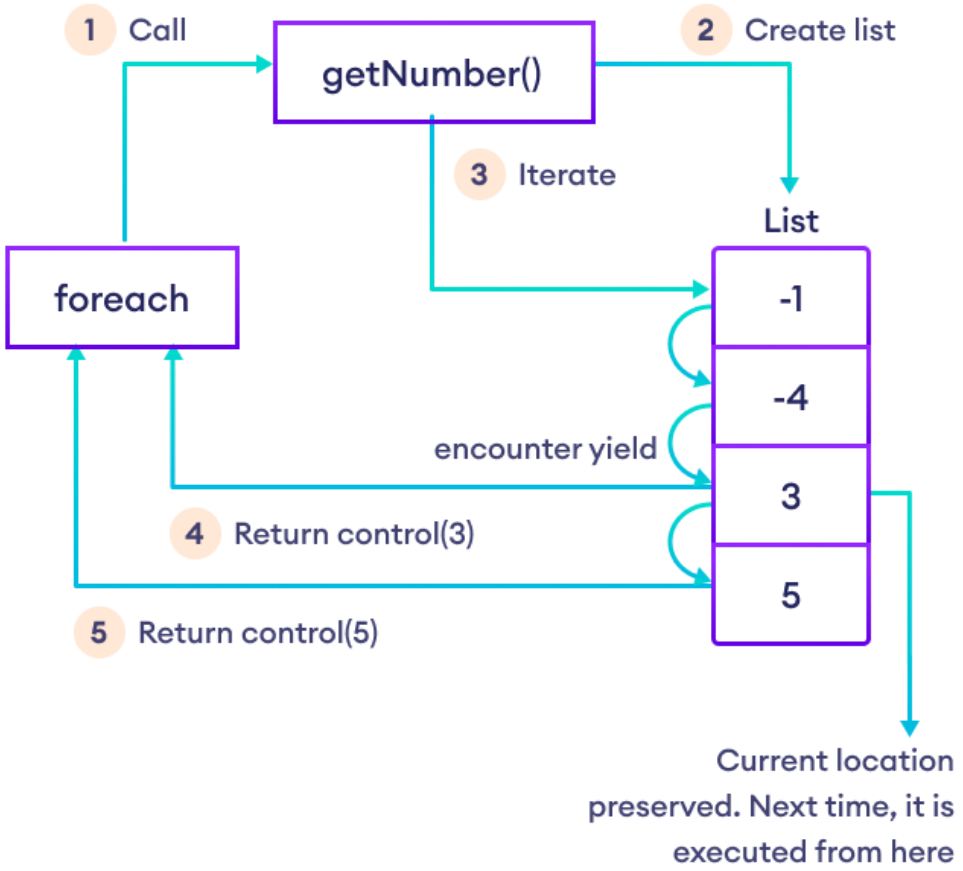
YIELD KEYWORD

- yield is used within **iterator blocks** to produce a sequence **lazily** (on demand), one element at a time.
- yield return <expr>; produces the next element.
- yield break; ends the sequence.
- The compiler transforms your method/property into a **state machine** that implements IEnumerable<T>/IEnumerator<T> (or IAsyncEnumerable<T> for async iterators).
- It’s great for streaming, composing pipelines, tree traversals, and potentially infinite sequences—without allocating large intermediate collections.

YIELD KEYWORD

- yield is used within **iterator blocks** to produce a sequence **lazily** (on demand), one element at a time.
- yield return <expr>; produces the next element.
- yield break; ends the sequence.
- It’s great for streaming, composing pipelines, tree traversals, and potentially infinite sequences—without allocating large intermediate collections.
- **Great for:**
  - Streaming large datasets without loading everything into memory.
  - Recursive traversal (trees/graphs) with clean code.
  - Potentially infinite sequences (as long as the consumer limits them).
- **Avoid or think twice if:**
  - You need **random access** or frequent re-iteration over a fixed snapshot → materialize to a List<T>/Array.
  - The sequence depends on **external mutable state** and you want deterministic results across runs.
  - You must not hold external resources open for the duration of iteration—then consider materializing quickly or moving resource access outside the iterator.

YIELD – HOW IT WORKS



YIELD KEYWORD (BONUS SLIDE)

The compiler **does not** run the whole method immediately. Instead, it generates **a hidden class** that implements the enumerator pattern (IEnumerable<int> + IEnumerator<int>), with a MoveNext() method that resumes from where it last yielded. Each yield return corresponds to a state transition. This is why you get **deferred execution** and **constant memory** usage independent of the total number of elements (aside from the iterator object itself).

Key implications:

- **Deferred execution**: No work is done until you start iterating (e.g., via foreach).
- **One element at a time**: Each MoveNext() call computes the next element only when needed.
- **Multiple enumerations**: Each call to the iterator method creates a new state machine (fresh enumeration).

YIELD - EXAMPLE

```
foreach (int i in ProduceEvenNumbers(9))
{
    Console.Write(i);
    Console.Write(" ");
}
// Output: 0 2 4 6 8

IEnumerable<int> ProduceEvenNumbers(int upto)
{
    for (int i = 0; i <= upto; i += 2)
    {
        yield return i;
    }
}
```

YIELD - EXAMPLE

```
Console.WriteLine(string.Join(" ", TakeWhilePositive(new int[] {2, 3, 4, 5, -1, 3, 4})));
// Output: 2 3 4 5

Console.WriteLine(string.Join(" ", TakeWhilePositive(new int[] {9, 8, 7})));
// Output: 9 8 7

IEnumerable<int> TakeWhilePositive(IEnumerable<int> numbers)
{
    foreach (int n in numbers)
    {
        if (n > 0)
        {
            yield return n;
        }
        else
        {
            yield break;
        }
    }
}
```

YIELD: DEMO

QUESTIONS TIME

LINQ (PRONOUNCED ‘LINK’)

LINQ – WHY?

- LINQ was designed by **Anders Hejlsberg**, the same legendary engineer behind *Turbo Pascal*, *Delphi*, and later *C#.*
- Many of the concepts that LINQ introduced were originally tested in Microsoft's Cω research project, formerly known by the codenames X# (X Sharp) and Xen. It was renamed to Cω after Polyphonic C# (another research language based on join calculus principles) was integrated into it. Cω attempts to make datastores (such as databases and XML documents) accessible with the same ease and type safety as traditional types like strings and arrays.
- First available in 2004 as a compiler preview, Cω's features were subsequently used by Microsoft in the creation of the LINQ features released in 2007 in .NET version 3.5
- **Hejlsberg** was fascinated by how developers had to constantly switch contexts between their programming language (like C#) and query languages (like SQL or XPath). This led to:
  - String-based queries that lacked compile-time checking.
  - Different type systems between code and data.
  - Poor integration between business logic and data access.
- LINQ was designed to unify querying across different data sources directly within the programming language, making queries type-safe, intellisense-enabled, and consistent.

LINQ

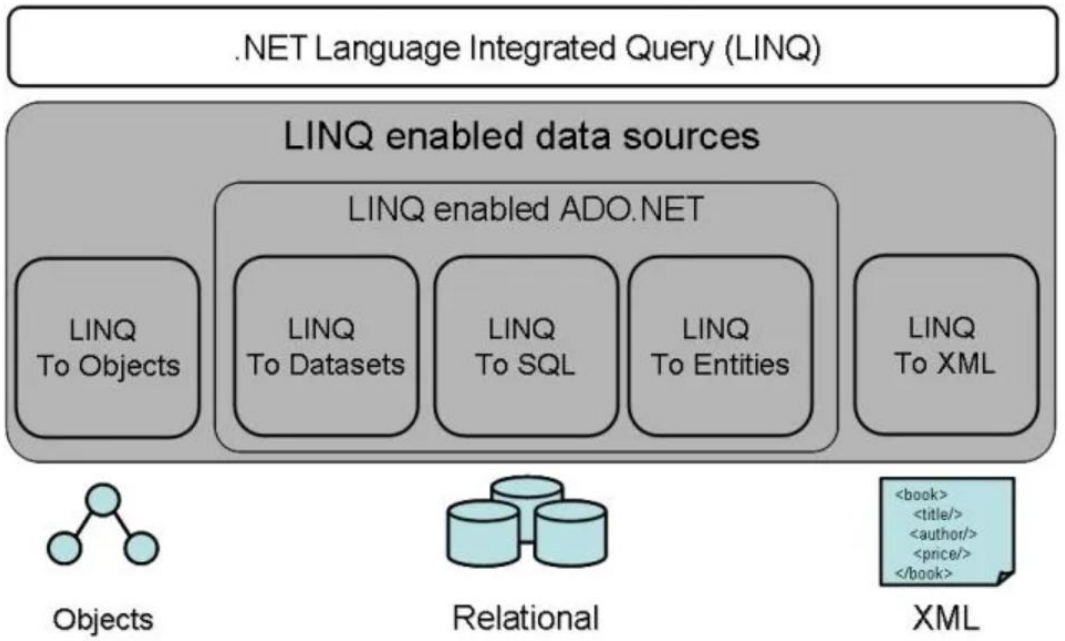
L
- Language

IN
- INtegrated

Q
- Query

- LINQ is the name of a set of technologies based on the integration of query capabilities directly into the C# language.
- LINQ provides an abstraction layer based on a common query syntax, which makes it possible to access different data sources without worrying about the implementation details of each one.
- Depending on the data source you want to access, a specific LINQ implementation will translate and execute the queries. These implementations are called LINQ providers.

LINQ



- **Key Features**:
  - *Declarative Syntax*: LINQ allows you to write queries in a declarative manner, specifying what you want to retrieve rather than how to retrieve it.
  - *Strongly Typed*: LINQ queries are checked at compile-time for syntax errors and type safety.
  - *Integration with C#*: LINQ is integrated into the C# language, allowing you to use familiar syntax and operators.
- **Abstractions**:
  - *LINQ to Objects* : Queries in-memory data using IEnumerable<T> from System.Collections.Generic.
  - *LINQ to Entities*: Queries data from the Entity Framework using IQueryable<T>. Found in System.Data.Entity
  - *LINQ to XML*: Simplifies in-memory XML programming with System.Xml.Linq

LINQ

All LINQ query operations consist of three distinct actions:

1. Obtain the data source: the data source in the example is an array, which supports the generic *IEnumerable<T>* interface. This fact means it can be queried with LINQ.
2. Create the query: the query specifies what information to retrieve from the data source or sources. Optionally, a query also specifies how that information should be sorted, grouped, and shaped before being returned. A query is stored in a query variable and initialized with a query expression
3. Execute the query: any attempt to enumerate or access the query results will bring to the execution of the query itself (deferred execution).

```
// 1. Data source.
int[] numbers = [ 0, 1, 2, 3, 4, 5, 6 ];

// 2. Query creation.
// numQuery is an IEnumerable<int>
var numQuery = from num in numbers
               where (num % 2) == 0
               select num;

// 3. Query execution.
foreach (int num in numQuery)
{
    Console.Write(num);
}
```

QUERY SYNTAX VS METHOD SYNTAX

```
List<int> numbers = [ 5, 4, 1, 3, 9, 8, 6, 7, 2, 0 ];

// Query #1.
IEnumerable<int> filteringQuery =
    from num in numbers
    where num is < 3 or > 7
    select num;

// Query #2.
IEnumerable<int> orderingQuery =
    from num in numbers
    where num is < 3 or > 7
    orderby num ascending
    select num;
```

```
List<int> numbers = [ 5, 4, 1, 3, 9, 8, 6, 7, 2, 0 ];

// Query #1.
double average = numbers.Average();

// Query #2
IEnumerable<int> largeNumbersQuery = numbers.Where(c => c > 15);

// Query #3
var numCount = numbers.Count(n => n is > 3 and < 7);
```

STANDARD QUERY OPERATORS

The standard query operators are the keywords and methods that form the LINQ pattern. The C# language defines LINQ query keywords (listed on the right side) that you use for the most common query expression. The compiler translates expressions using these keywords to the equivalent method calls. The two forms are synonymous. Other methods that are part of the **System.Linq** namespace don't have equivalent query keywords. In those cases, you must use the method syntax.

Clause	Description
from	Specifies a data source and a range variable (similar to an iteration variable).
where	Filters source elements based on one or more Boolean expressions separated by logical AND and OR operators (&& or ||).
select	Specifies the type and shape that the elements in the returned sequence will have when the query is executed.
group	Groups query results according to a specified key value.
into	Provides an identifier that can serve as a reference to the results of a join, group or select clause.
orderby	Sorts query results in ascending or descending order based on the default comparer for the element type.
join	Joins two data sources based on an equality comparison between two specified matching criteria.
let	Introduces a range variable to store subexpression results in a query expression.
in	Contextual keyword in a join clause.
on	Contextual keyword in a join clause.
equals	Contextual keyword in a join clause.
by	Contextual keyword in a group clause.
ascending	Contextual keyword in an orderby clause.
descending	Contextual keyword in an orderby clause.

FILTER DATA

Filtering refers to the operation of restricting the result set to contain only those elements that satisfy a specified condition. It's also referred to as selecting elements that match the specified condition.

Method Name	Description	C# Query Expression Syntax
OfType	Selects values, depending on their ability to be cast to a specified type.	Not applicable.
Where	Selects values that are based on a predicate function.	where

FILTER DATA

```
string[] words = ["the", "quick", "brown", "fox", "jumps"];

IEnumerable<string> query = from word in words
                            where word.Length == 3
                            select word;

foreach (string str in query)
{
    Console.WriteLine(str);
}

/* This code produces the following output:

    the
    fox
*/
```

```
string[] words = ["the", "quick", "brown", "fox", "jumps"];

IEnumerable<string> query =
    words.Where(word => word.Length == 3);

foreach (string str in query)
{
    Console.WriteLine(str);
}

/* This code produces the following output:

    the
    fox
*/
```

PROJECTION OPERATIONS

Projection refers to the operation of transforming an object into a new form that often consists only of those properties subsequently used. By using projection, you can construct a new type that is built from each object. You can project a property and perform a mathematical function on it. You can also project the original object without changing it.

Method names	Description	C# query expression syntax
Select	Projects values that are based on a transform function.	select
SelectMany	Projects sequences of values that are based on a transform function and then flattens them into one sequence.	Use multiple from clauses
Zip	Produces a sequence of tuples with elements from 2-3 specified sequences.	Not applicable.

PROJECTION: SELECT

```
List<string> words = ["an", "apple", "a", "day"];

var query = words.Select(word => word.Substring(0,

foreach (string s in query)
{
    Console.WriteLine(s);
}

/* This code produces the following output:

    a
    a
    a
    d
*/
```

```
List<string> words = ["an", "apple", "a", "day"];

var query = from word in words
            select word.Substring(0, 1);

foreach (string s in query)
{
    Console.WriteLine(s);
}

/* This code produces the following output:

    a
    a
    a
    d
*/
```

PROJECTION: SELECTMANY

```
List<string> phrases = ["an apple a day", "the quick brown fox"];

var query = from phrase in phrases
            from word in phrase.Split(' ')
            select word;

foreach (string s in query)
{
    Console.WriteLine(s);
}

/* This code produces the following output:

    an
    apple
    a
    day
    the
    quick
    brown
    fox
*/
```

```
List<string> phrases = ["an apple a day", "the quick brown fox"];

var query = phrases.SelectMany(phrase => phrase.Split(' '));

foreach (string s in query)
{
    Console.WriteLine(s);
}

/* This code produces the following output:

    an
    apple
    a
    day
    the
    quick
    brown
    fox
*/
```

PROJECTION: ZIP

```
// An int array with 7 elements.
IEnumerable<int> numbers = [1, 2, 3, 4, 5, 6, 7];
// A char array with 6 elements.
IEnumerable<char> letters = ['A', 'B', 'C', 'D', 'E', 'F'];

foreach ((int number, char letter) in numbers.Zip(letters))
{
    Console.WriteLine($"Number: {number} zipped with letter: '{letter}'");
}
// This code produces the following output:
//     Number: 1 zipped with letter: 'A'
//     Number: 2 zipped with letter: 'B'
//     Number: 3 zipped with letter: 'C'
//     Number: 4 zipped with letter: 'D'
//     Number: 5 zipped with letter: 'E'
//     Number: 6 zipped with letter: 'F'
```

SET OPERATIONS

Set operations in LINQ refer to query operations that produce a result set based on the presence or absence of equivalent elements within the same or separate collections.

Method names	Description	C# query expression syntax
Distinct or DistinctBy*	Removes duplicate values from a collection.	Not applicable.
Except or ExceptBy*	Returns the set difference, which means the elements of one collection that don't appear in a second collection.	Not applicable.
Intersect or IntersectBy*	Returns the set intersection, which means elements that appear in each of two collections.	Not applicable.

SET: DISTINCT

```
string[] words = ["the", "brown", "fox", "jumped", "over", "another", "brown", "fox"];

IEnumerable<string> distinctWords = words.Distinct();

foreach (var str in distinctWords)
{
    Console.WriteLine(str);
}

/* This code produces the following output:
 *
 * the
 * brown
 * fox
 * jumped
 * another
 */
```

SET: EXCEPT

```
string[] words1 = ["the", "quick", "brown", "fox"];
string[] words2 = ["jumped", "over", "the", "lazy", "dog"];

IEnumerable<string> query = words1.Except(words2);

foreach (var str in query)
{
    Console.WriteLine(str);
}

/* This code produces the following output:
 *
 * quick
 * brown
 * fox
 */
```

SET: INTERSECT

```
string[] words1 = ["the", "quick", "brown", "fox"];
string[] words2 = ["jumped", "over", "the", "lazy", "dog"];

IEnumerable<string> query = words1.Intersect(words2);

foreach (var str in query)
{
    Console.WriteLine(str);
}

/* This code produces the following output:
 *
 * the
 */
```

SET: UNION

```
string[] words1 = ["the", "quick", "brown", "fox"];
string[] words2 = ["jumped", "over", "the", "lazy", "dog"];

IEnumerable<string> query = words1.Union(words2);

foreach (var str in query)
{
    Console.WriteLine(str);
}

/* This code produces the following output:
 *
 * the
 * quick
 * brown
 * fox
 * jumped
 * over
 * lazy
 * dog
*/
```

SORTING DATA

A sorting operation orders the elements of a sequence based on one or more attributes. The first sort criterion performs a primary sort on the elements. By specifying a second sort criterion, you can sort the elements within each primary sort group.

Method Names	Description	C# Query Expression Syntax
OrderBy	Sorts values in ascending order.	orderby
OrderByDescending	Sorts values in descending order.	orderby … descending
ThenBy	Performs a secondary sort in ascending order.	orderby …, …
ThenByDescending	Performs a secondary sort in descending order.	orderby …, … descending
Reverse	Reverses the order of the elements in a collection.	Not applicable.

SORTING: ORDERBY ASCENDING

```
IEnumerable<(string, string)> query = from teacher in teachers
                                      orderby teacher.City, teacher.Last
                                      select (teacher.Last, teacher.City);

foreach ((string last, string city) in query)
{
    Console.WriteLine($"City: {city}, Last Name: {last}");
}
```

SORTING: ORDERBY ASCENDING

```
IEnumerable<string> query = teachers
    .OrderBy(teacher => teacher.Last)
    .Select(teacher => teacher.Last);

foreach (string str in query)
{
    Console.WriteLine(str);
}
```

```
IEnumerable<(string, string)> query = teachers
    .OrderBy(teacher => teacher.City)
    .ThenBy(teacher => teacher.Last)
    .Select(teacher => (teacher.Last, teacher.City));

foreach ((string last, string city) in query)
{
    Console.WriteLine($"City: {city}, Last Name: {last}");
}
```

SORTING: ORDERBY DESCENDING

```
IEnumerable<string> query = teachers
    .OrderByDescending(teacher => teacher.Last)
    .Select(teacher => teacher.Last);

foreach (string str in query)
{
    Console.WriteLine(str);
}
```

```
IEnumerable<(string, string)> query = teachers
    .OrderBy(teacher => teacher.City)
    .ThenByDescending(teacher => teacher.Last)
    .Select(teacher => (teacher.Last, teacher.City));

foreach ((string last, string city) in query)
{
    Console.WriteLine($"City: {city}, Last Name: {last}");
}
```

QUANTIFIER OPERATIONS

Quantifier operations return a Boolean value that indicates whether some or all of the elements in a sequence satisfy a condition.

Method Name	Description	C# Query Expression Syntax
All	Determines whether all the elements in a sequence satisfy a condition.	Not applicable.
Any	Determines whether any elements in a sequence satisfy a condition.	Not applicable.
Contains	Determines whether a sequence contains a specified element.	Not applicable.

QUANTIFIER : ALL

```
IEnumerable<Student> graduatedStudents =
  student.Scores.All(score => score > 70);

foreach (string student in graduatedStudents)
{
    Console.WriteLine($"{student.Name} {student.Surname}");
}
```

QUANTIFIER : ANY

```
IEnumerable<Student> retakeCandidates =
  student.Scores.Any(score => score <= 70);

foreach (string student in retakeCandidates)
{
    Console.WriteLine($"{student.Name} {student.Surname}");
}
```

QUANTIFIER : CONTAINS

```
IEnumerable<Student> greatCandidates =
  student.Scores.Contains(100);

foreach (string student in greatCandidates)
{
    Console.WriteLine($"{student.Name} {student.Surname}");
}
```

PARTITION DATA

Partitioning in LINQ refers to the operation of dividing an input sequence into sections, without rearranging the elements, and then returning one of the sections.

Method names	Description	C# query expression syntax
Skip	Skips elements up to a specified position in a sequence.	Not applicable.
SkipWhile	Skips elements based on a predicate function until an element doesn't satisfy the condition.	Not applicable.
Take	Takes elements up to a specified position in a sequence.	Not applicable.
TakeWhile	Takes elements based on a predicate function until an element doesn't satisfy the condition.	Not applicable.
Chunk	Splits the elements of a sequence into chunks of a specified maximum size.	Not applicable.

PARTITIONING : TAKE

```
foreach (int number in Enumerable.Range(0, 8).Take(3))
{
    Console.WriteLine(number);
}
// This code produces the following output:
// 0
// 1
// 2
```

PARTITIONING : SKIP

```
foreach (int number in Enumerable.Range(0, 8).Skip(3))
{
    Console.WriteLine(number);
}
// This code produces the following output:
// 3
// 4
// 5
// 6
// 7
```

PARTITIONING : TAKEWHILE

```
foreach (int number in Enumerable.Range(0, 8).TakeWhile(n => n < 5))
{
    Console.WriteLine(number);
}
// This code produces the following output:
// 0
// 1
// 2
// 3
// 4
```

PARTITIONING : SKIPWHILE

```
foreach (int number in Enumerable.Range(0, 8).SkipWhile(n => n < 5))
{
    Console.WriteLine(number);
}
// This code produces the following output:
// 5
// 6
// 7
```

PARTITIONING : CHUNK

```
int chunkNumber = 1;
foreach (int[] chunk in Enumerable.Range(0, 8).Chunk(3))
{
    Console.WriteLine($"Chunk {chunkNumber++}:");
    foreach (int item in chunk)
    {
        Console.WriteLine($"    {item}");
    }

    Console.WriteLine();
}
// This code produces the following output:
// Chunk 1:
//    0
//    1
//    2
//
// Chunk 2:
//    3
//    4
//    5
//
// Chunk 3:
//    6
//    7
```

CONVERTING DATA TYPES

Conversion methods change the type of input objects.

Method Name	Description	C# Query Expression Syntax
AsEnumerable	Returns the input typed as IEnumerable<T>.	Not applicable.
AsQueryable	Converts a (generic) IEnumerable to a (generic) IQueryable.	Not applicable.
Cast	Casts the elements of a collection to a specified type.	Use an explicitly typed range variable. For example: from string str in words
OfType	Filters values, depending on their ability to be cast to a specified type.	Not applicable.
ToArray	Converts a collection to an array. This method forces query execution.	Not applicable.
ToDictionary	Puts elements into a Dictionary<TKey,TValue> based on a key selector function. This method forces query execution.	Not applicable.
ToList	Converts a collection to a List<T>. This method forces query execution.	Not applicable.
ToLookup	Puts elements into a Lookup<TKey,TElement> (a one-to-many dictionary) based on a key selector function. This method forces query execution.	Not applicable.

CONVERTING: TODICTIONARY

```
public class Person
{
    public int Id { get; set; }
    public string Name { get; set; }
}

var people = new List<Person>
{
    new Person { Id = 1, Name = "Alice" },
    new Person { Id = 2, Name = "Bob" },
    new Person { Id = 3, Name = "Charlie" }
};

var peopleDict = people.ToDictionary(p => p.Id, p => p.Name);
```

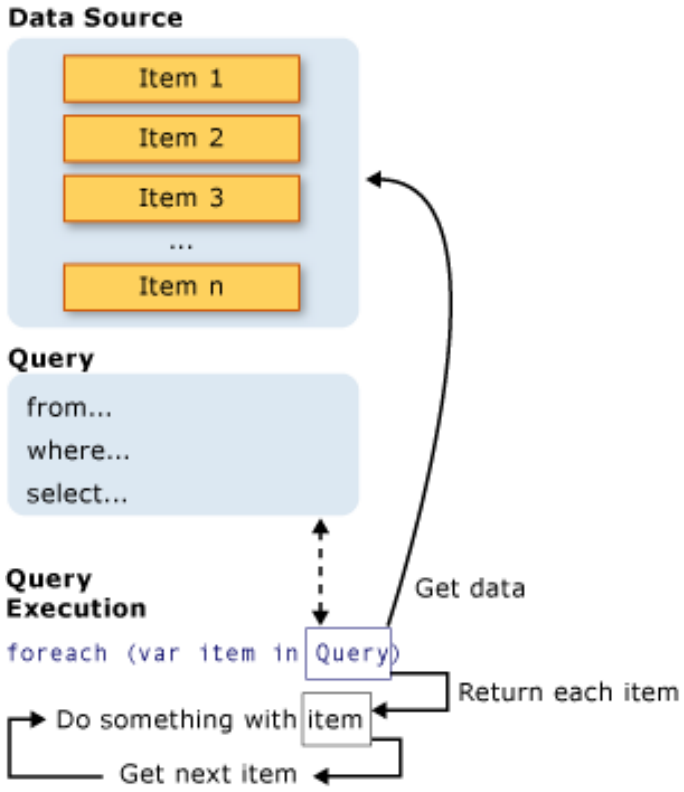
OTHER METHODS

Method Name	Description
Count	Returns the number of elements in a sequence
CountBy	Returns the count of elements in the source sequence grouped by key (From NET9)
First	Returns the first element of a sequence
FirstOrDefault	Returns the first element of a sequence, or a default value if no element is found
Last	Returns the last element of a sequence
LastOrDefault	Returns the last element of a sequence, or a default value if no element is found
Single	Returns the only element of a sequence and throws an exception if there is not exactly one element in the sequence.
SingleOrDefault	Returns the only element of a sequence, or a default value if the sequence is empty; this method throws an exception if there is more than one element in the sequence.

MANNER OF EXECUTIONS

MANNER OF EXECUTIONS

The LINQ to Objects implementations of the standard query operator methods execute in one of two main ways: **immediate** or **deferred**.



IMMEDIATE AND DEFERRED EXECUTION

- **Immediate execution** means that the data source is read and the operation is performed once. All the standard query operators that return a scalar result execute immediately. Examples of such queries are **Count**, **Max**, **Average**, and **First**. These methods execute without an explicit foreach statement because the query itself must use foreach in order to return a result. These queries return a single value, not an *IEnumerable* collection. You can force any query to execute immediately using the *Enumerable.ToList* or *Enumerable.ToArray* methods. Immediate execution provides reuse of query results, not query declaration. The results are retrieved once, then stored for future use.
- **Deferred execution** means that the operation isn't performed at the point in the code where the query is declared. The operation is performed only when the query variable is enumerated, for example by using a foreach statement. The results of executing the query depend on the contents of the data source when the query is executed rather than when the query is defined. If the query variable is enumerated multiple times, the results might differ every time. Almost all the standard query operators whose return type is IEnumerable<T> or IOrderedEnumerable<TElement> execute in a deferred manner. Deferred execution provides the facility of query reuse since the query fetches the updated data from the data source each time query results are iterated

IMMEDIATE OR DEFERRED?

Standard query operator	Return type	Immediate execution	Deferred execution
All	Boolean	✓	
Any	Boolean	✓	
Average	Single numeric value	✓	
Concat	IEnumerable<T>		✓
Contains	Boolean	✓	
Count	Int32	✓	
OrderBy	IOrderedEnumerable<T>		✓
Select	IEnumerable<T>		✓
ToList	IList<T>	✓	
Where	IEnumerable<T>		✓

DEMO: LINQ

QUESTIONS TIME

EXERCISE: LINQ

DEFINING THE MOVIE DATABASE

You’re building a tiny in-memory backend for a movie platform.

- **Movies** have a title, year, duration, genres, content ratings, country of production, a director, a cast (composed by actors' name) and optional tags.
- **Users** belong to countries and rate movies they watch. They are characterized by their name, surname, username, email, date of joining the platform, birthday.
- **Ratings** connect a user and a movie, with a star score (0.5–5.0 in 0.5 increments), optional text review, spoiler flag, and a watch date.

Some hint:

- Genres and content ratings are predefined in our platform.
- The database is just a class which holds the List of Movies, Users and Ratings.

QUERY THE DATABASE

Alphabetical list of titles: return all movie titles in ascending alphabetical order.

Filter by release year: rovies released after 2000, projected to { Title, Year }.

Ratings per user: for a given user name (e.g., "Alice"), count how many ratings they submitted.

Average rating for a given movie: given a title (e.g., "The Matrix"), compute its average stars.

Distinct genres in catalog: list all distinct genres present across all movies, alphabetically.

Top 3 longest movies: titles and durations of the three longest films.

Most recent reviews: get texts of non-null reviews sorted by WatchedOn descending.

QUERY THE DATABASE

User–Movie–Stars table: produce tuples (UserName, MovieTitle, Stars), ordered by UserName, then Stars desc.

Movie aggregates (avg & count): for each movie with at least 2 ratings, compute { Title, AvgStars, Count }, sorted by AvgStars desc then Count desc.

Harsh vs lenient raters: for each user, compute their mean rating; return users with average < 3.5.

Top tags (tag cloud): from movie Tags, find top 5 most common tags (case-insensitive), with counts.

Movies not rated by Italians: list movies that have no ratings from users where Country == "IT".

Per-genre winners: for each genre, return the top-1 movie by average rating (only consider movies with ≥ 2 ratings). Break ties by higher rating count, then title.

Pagination: return page N (0-based) of movies ordered by Year desc then Title, with page size P.

QUERY THE DATABASE

Movies with ≥ 2 ratings: Min, Max, Avg (sorted by Avg desc): returns all the movies having more than 2 ratings and listing min, max and average rating.

Unrated movies for a given user (by Year desc): returns all the unrated movies for a given user and order the results by Year descending.

Top 3 most active users (by rating count): list the 3 most active users (users having more ratings).

Average rating per user country (only countries with ≥ 2 ratings), sorted by Avg desc: Take all the countries with at least 2 ratings and show the average rating.

Movies per decade (e.g., 1990s, 2000s): count how many movies there are in each decade (90s, 00s, 10s, 20s)

Per-genre movie counts (sorted desc): count how many movies there are for each genre.

Case-insensitive search in titles or tags (by substring), ordered by Title: provides an extension method for the list of movies that allows the developer to search (case-insensitive) by title or by tags.

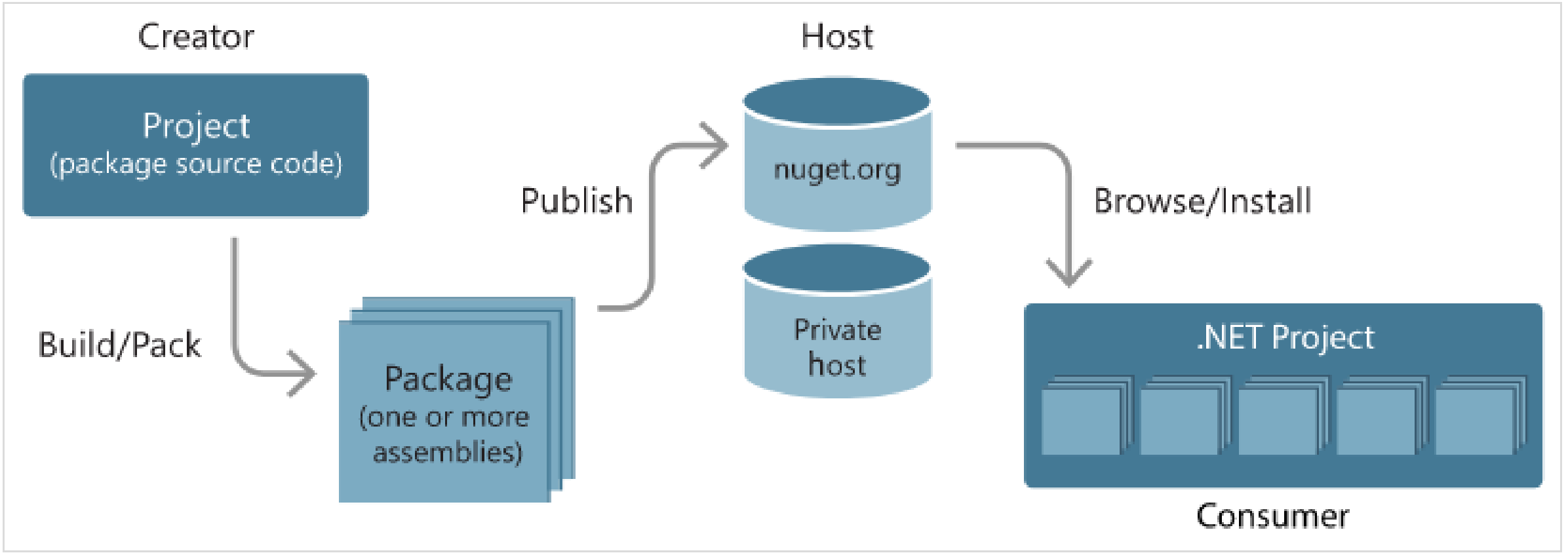
LINQ –THE END

NUGET

NUGET (1/4)

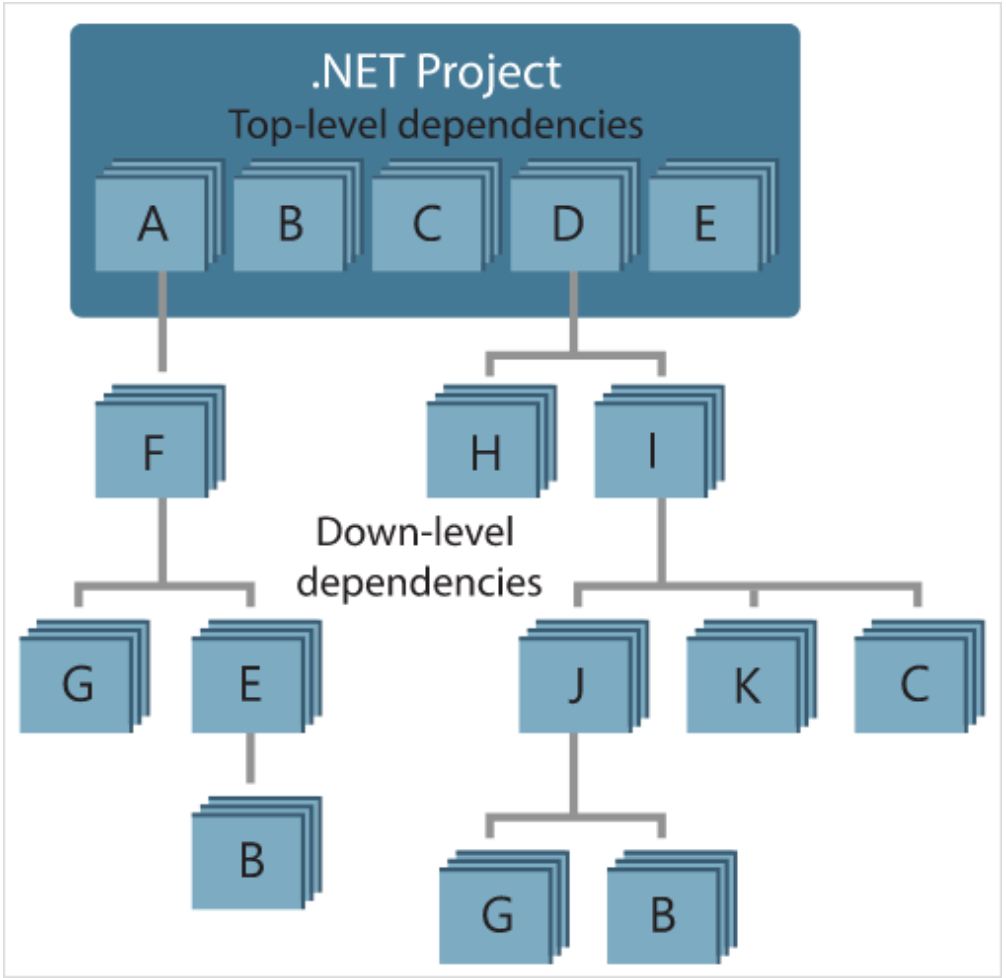
- An essential tool for any modern development platform is a mechanism through which developers can create, share, and consume useful code. Often such code is bundled into "packages" that contain compiled code (as DLLs) along with other content needed in the projects that consume these packages.
- For .NET (including .NET Core), the Microsoft-supported mechanism for sharing code is NuGet, which defines how packages for .NET are created, hosted, and consumed, and provides the tools for each of those roles.
- Put simply, a NuGet package is a single ZIP file with the .nupkg extension that contains compiled code (DLLs), other files related to that code, and a descriptive manifest that includes information like the package's version number. Developers with code to share create packages and publish them to a public or private host. Package consumers obtain those packages from suitable hosts, add them to their projects, and then call a package's functionality in their project code. NuGet itself then handles all of the intermediate details.
- Because NuGet supports private hosts alongside the public nuget.org host, you can use NuGet packages to share code that's exclusive to an organization or a work group.

NUGET (2/4)

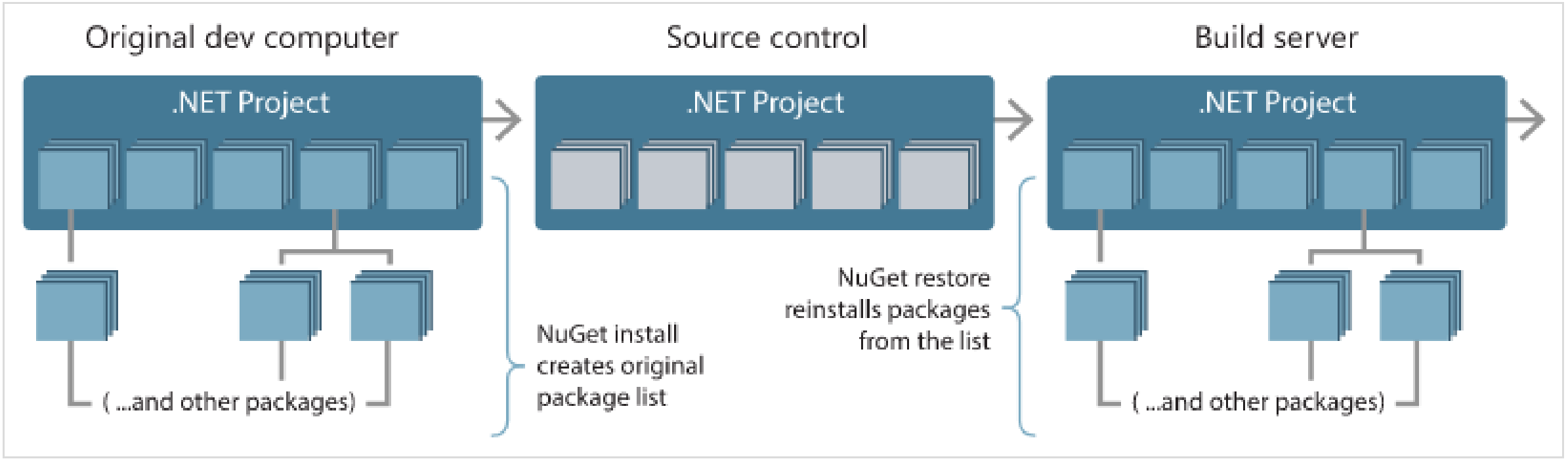


NUGET (3/4)

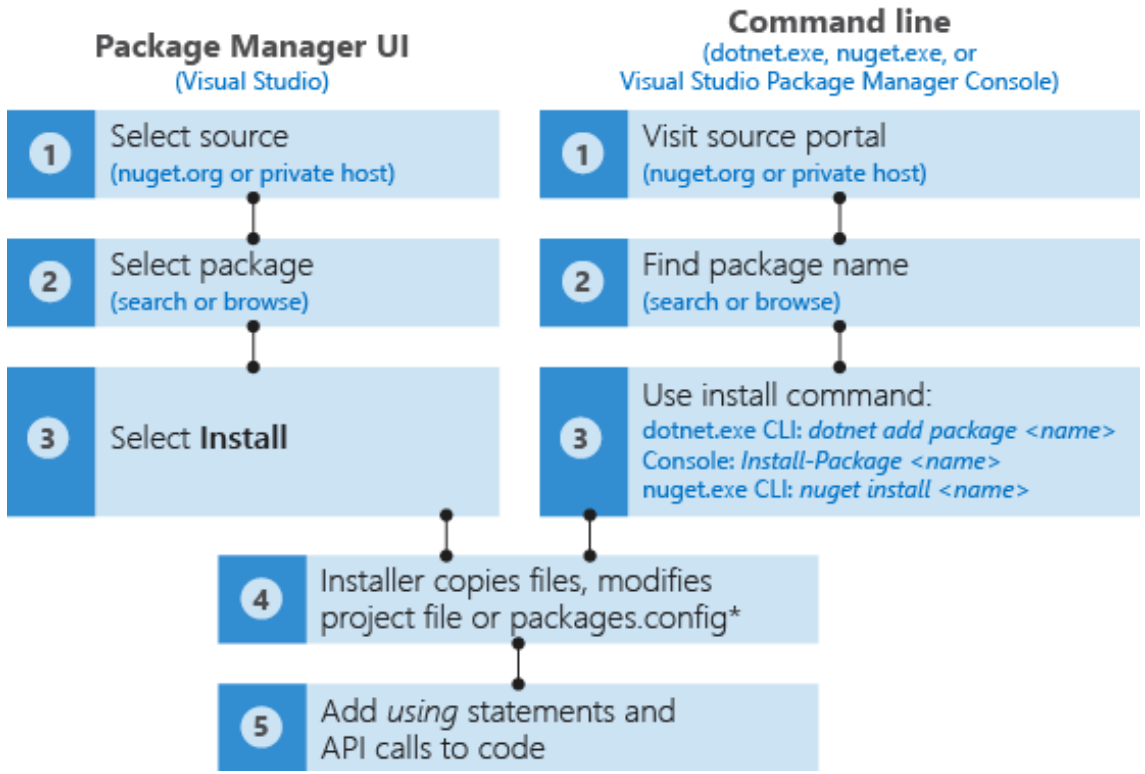
- The ability to easily build on the work of others is one of most powerful features of a package management system. Accordingly, much of what NuGet does is managing that dependency tree or "graph" on behalf of a project. Simply said, you need only concern yourself with those packages that you're directly using in a project. If any of those packages themselves consume other packages (which can, in turn, consume still others), NuGet takes care of all those down-level dependencies.
- Notice that some packages appear multiple times in the dependency graph. For example, there are three different consumers of package B, and each consumer might also specify a different version for that package



NUGET (4/4)

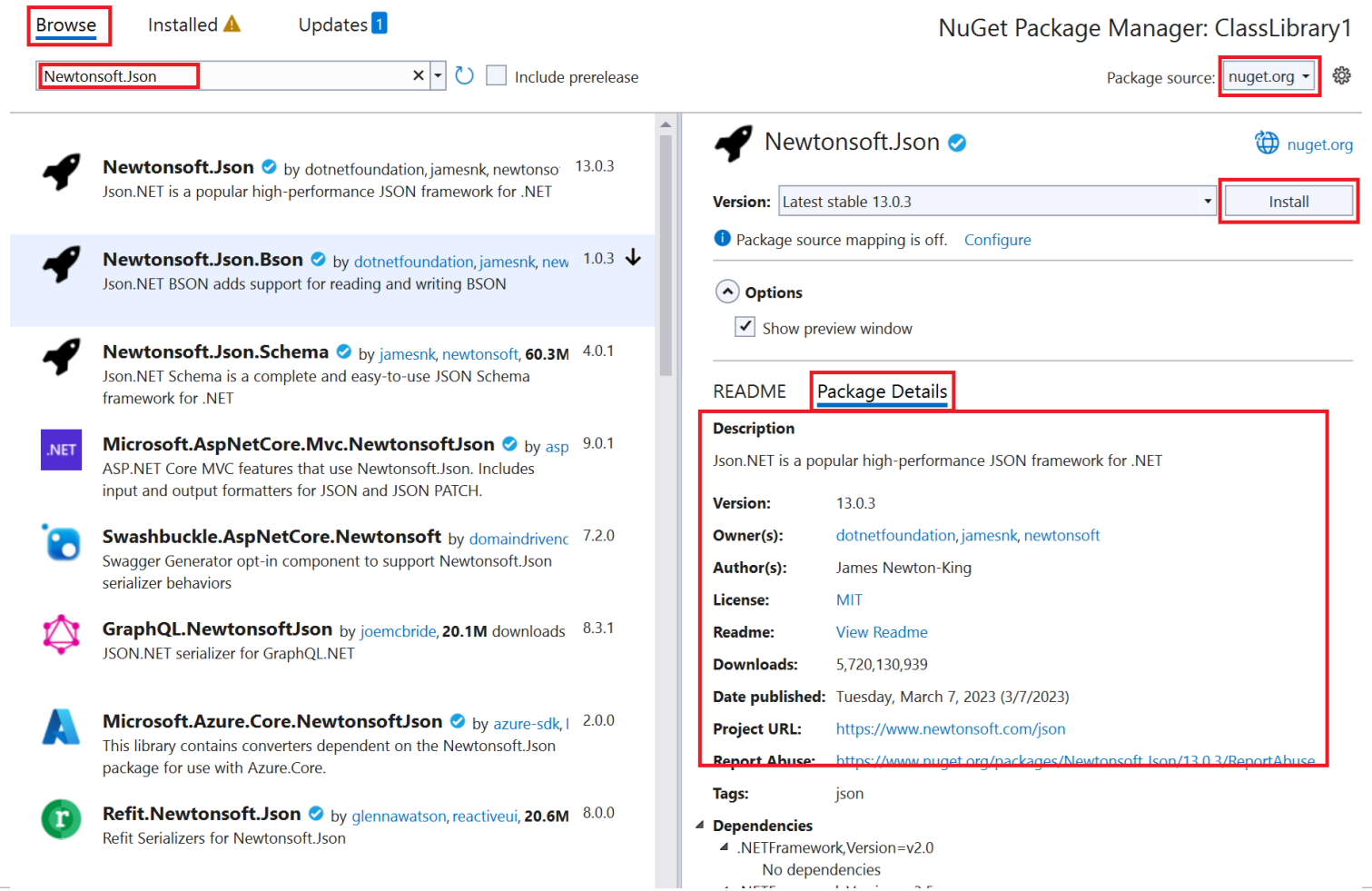


NUGET - HOW TO

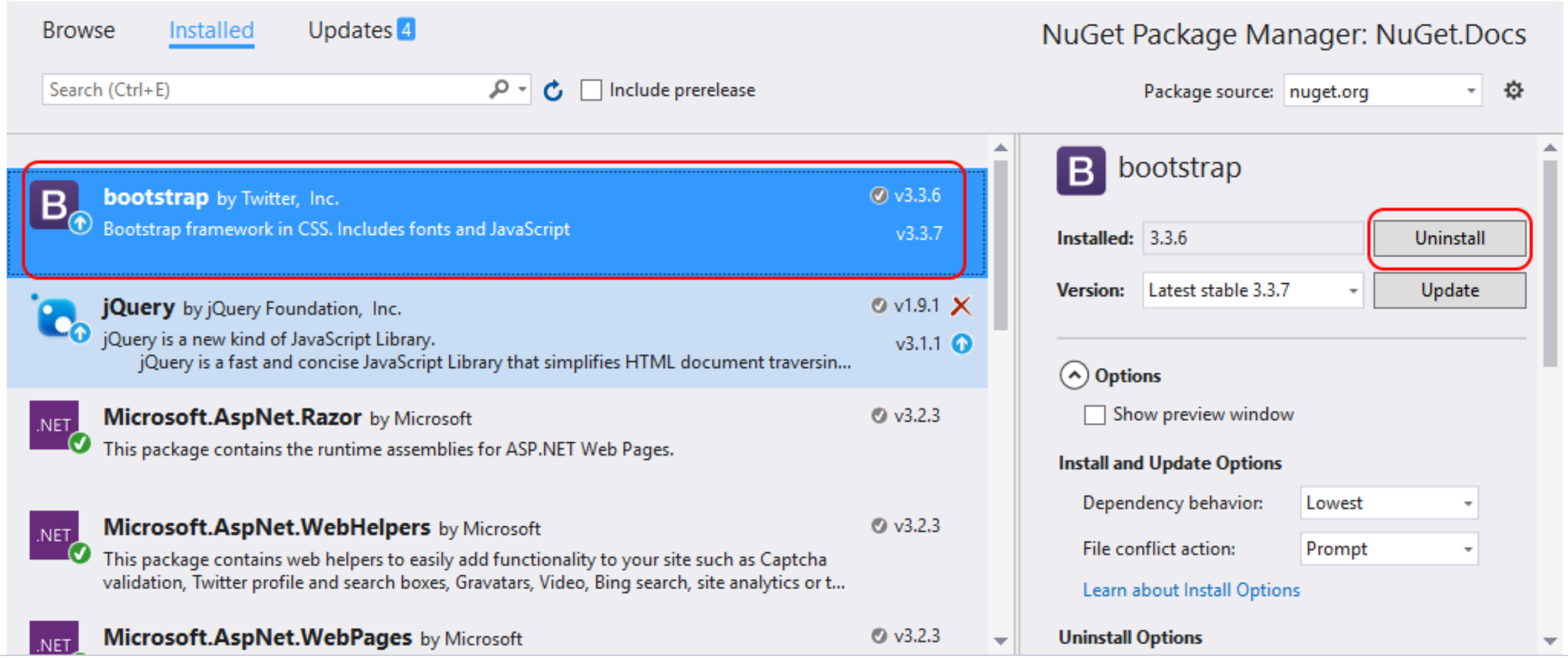


- NuGet remembers the identity and version number of each installed package, recording it in either the project file (using PackageReference) or packages.config, depending on project type and your version of NuGet. With NuGet 4.0+, PackageReference is preferred, although this is configurable in Visual Studio through the Package Manager UI. In any case, you can look in the appropriate file at any time to see the full list of dependencies for your project.
- It's prudent to always check the license for each package you intend to use in your software. On nuget.org, you find a **License Info** link on the right side of each package's description page.

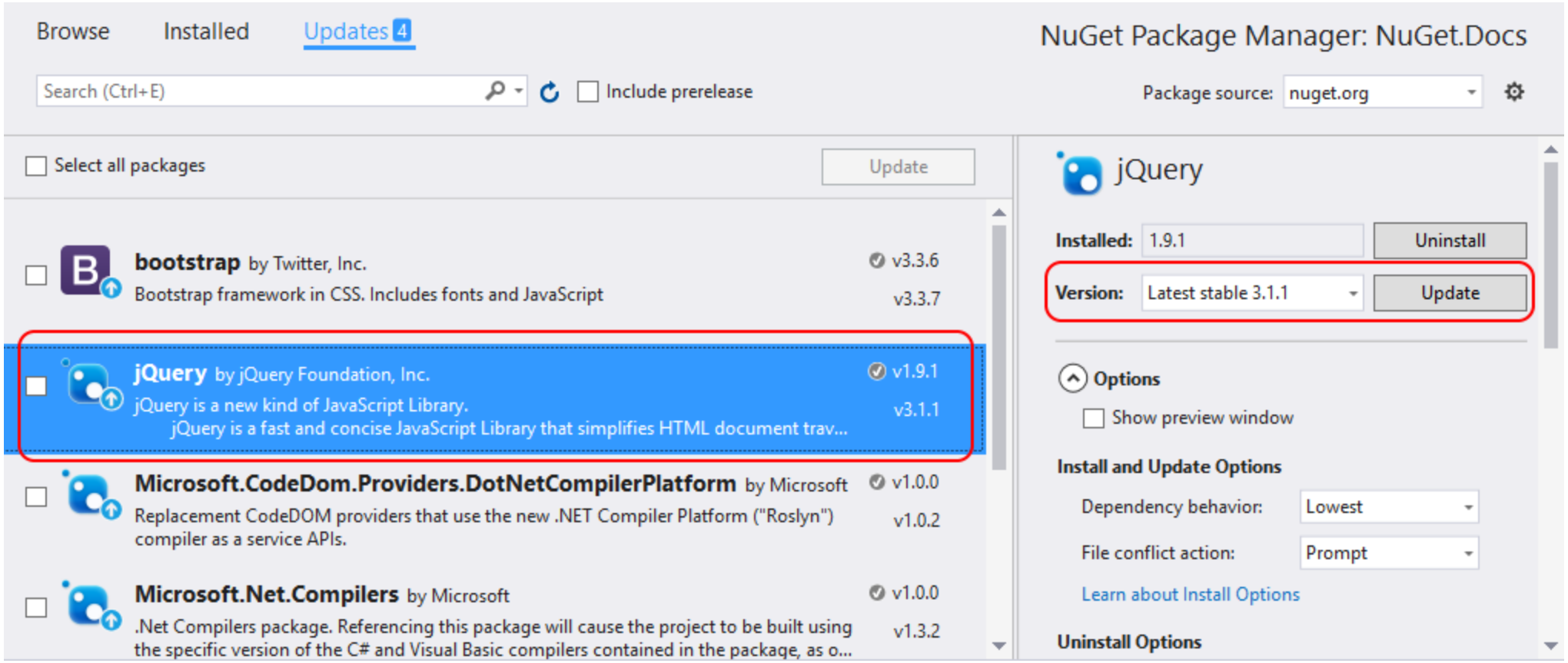
NUGET - HOW TO - INSTALL



NUGET - HOW TO - UNINSTALL



NUGET - HOW TO - UPDATE



POPULAR NUGET PACKAGES

- **Serilog**: A structured logging library that allows developers to log events in various formats such as JSON. It integrates seamlessly with external monitoring tools.
- **AutoMapper**: A library that automates object-to-object mapping, helping to keep your codebase clean and manageable.
- **Newtonsoft.Json (Json.NET)**: A popular library for handling JSON data in .NET applications.
- **NUnit**: A widely-used unit testing framework that helps developers catch bugs early.
- **FluentValidation**: A library that provides a fluent API for defining validation rules, improving code readability and maintainability.
- **Polly**: A resilience and fault-tolerance library that allows developers to define policies for handling API calls.
- **Hangfire**: A library for background job processing in .NET applications.
- **MediatR**: A library that implements the Mediator pattern to streamline communication between components in large-scale applications.
- **Swashbuckle.AspNetCore**: A library that simplifies Swagger integration for documenting APIs.
- **SignalR**: A library for real-time web communication

CREATE YOUR OWN PACKAGE

- Creating a package starts with the compiled code (typically .NET assemblies) that you want to package and share with others, either through the public nuget.org gallery or a private gallery within your organization. The package can also include additional files such as a readme that is displayed when the package is installed and can include transformations to certain project files.
- Whatever the case, creating a package begins with deciding its identifier, version number, license, copyright information, and any other necessary content. Once done, you can use the "pack" command to put everything together into a .nupkg file. This file can be published to a NuGet feed, like nuget.org
- You can create your own package by using
  - Dotnet CLI
  - Nuget.exe
  - MSBuild
  - Visual Studio

QUESTIONS TIME

MAPPING BETWEEN OBJECT

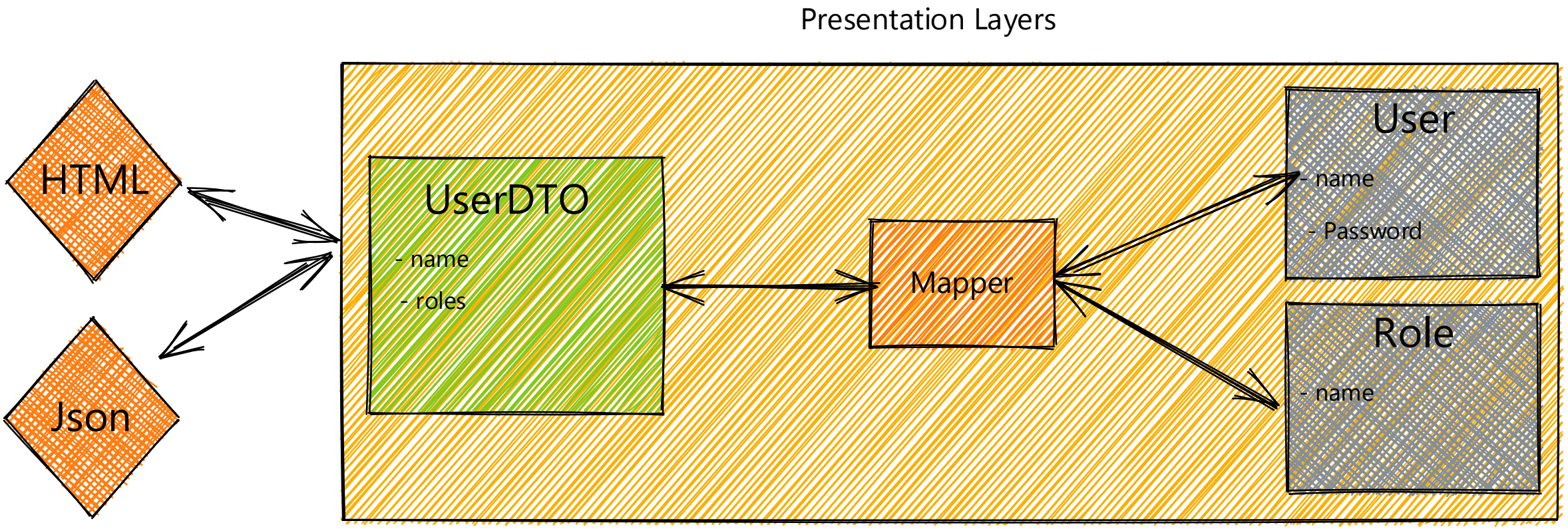
MAPPING BETWEEN OBJECT

In OOP, we often work with objects that represent real-world entities (like User, Order, Invoice). However, different layers of an application (e.g., database, business logic, API) have different needs and different representations of these objects.

This is where object mapping comes in: it’s the process of translating one object structure into another.

- Why Object Mapping Matters? Imagine a typical enterprise application with these layers:
  - **Database Layer**: Uses entities that match the database schema.
  - **Business Logic Layer**: Uses domain models with behavior and rules.
  - **API Layer**: Uses DTOs (Data Transfer Objects) to expose only necessary data.
- Why Do We Need Object Mappings?
  - **Separation of Concerns**: keeps your business logic, data access, and presentation layers clean and independent.
  - **Security**: prevents exposing sensitive internal data structures to external consumers (e.g., APIs).
  - **Flexibility**: allows you to evolve internal models without breaking external contracts.
  - **Maintainability**: easier to test, debug, and refactor when each layer has its own model.
  - **Performance Optimization**: you can shape the data to include only what’s needed, reducing payload size.

MAPPING BETWEEN OBJECT



MAPPING BETWEEN OBJECT

```
public class User
{
    public int Id { get; set; }
    public string FullName { get; set; }
    public string Email { get; set; }
    public string PasswordHash { get; set; } // Sensitive
}
```

MAPPING BETWEEN OBJECT

```
public class UserDto
{
    public string FullName { get; set; }
    public string Email { get; set; }
}
```

MAPPING BETWEEN OBJECT

```
public static class UserMapper
{
    public static UserDto ToDto(User user)
    {
        return new UserDto
        {
            FullName = user.FullName,
            Email = user.Email
        };
    }

    public static User FromDto(UserDto dto)
    {
        return new User
        {
            FullName = dto.FullName,
            Email = dto.Email
            // PasswordHash is not set here
        };
    }
}
```

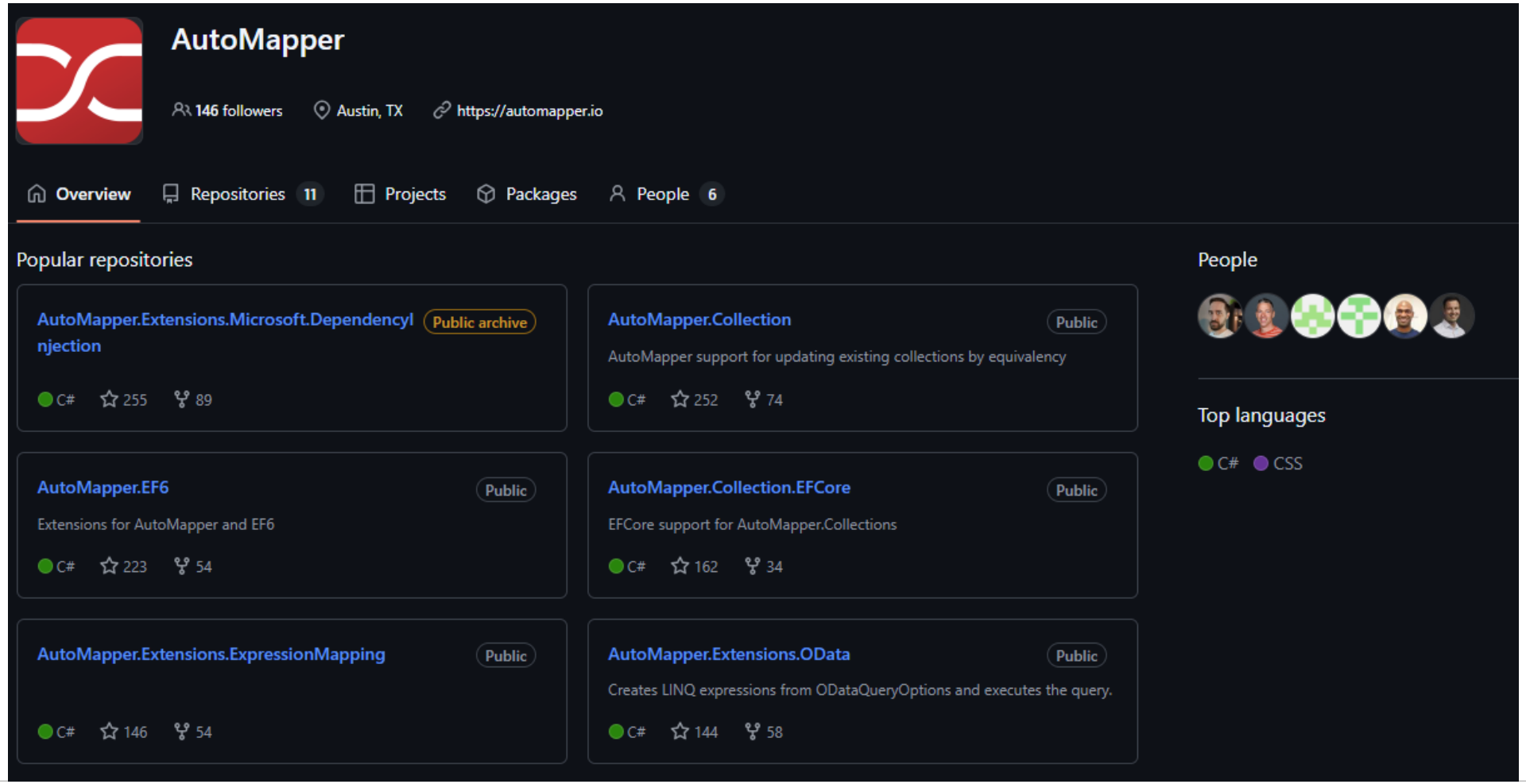
MAPPING BETWEEN OBJECT

```
User userFromDb = GetUserFromDatabase();
UserDto dto = UserMapper.ToDto(userFromDb);

// Send dto to API response
```

AUTOMAPPER

- AutoMapper is an object-object mapper. Object-object mapping works by transforming an input object of one type into an output object of a different type. What makes AutoMapper interesting is that it provides some interesting conventions to take the dirty work out of figuring out how to map type A to type B. As long as type B follows AutoMapper's established convention, almost zero configuration is needed to map two types.
- Mapping code is boring. Testing mapping code is even more boring. AutoMapper provides simple configuration of types, as well as simple testing of mappings. The real question may be "why use object-object mapping?" Mapping can occur in many places in an application, but mostly in the boundaries between layers, such as between the UI/Domain layers, or Service/Domain layers. Concerns of one layer often conflict with concerns in another, so object-object mapping leads to segregated models, where concerns for each layer can affect only types in that layer.
- https://github.com/automapper/
- It became commercial starting from version 15.0: https://www.jimmybogard.com/automapper-and-mediatr-commercial-editions-launch-today/

AUTOMAPPER



AUTOMAPPER: HOW TO

```
var config = new MapperConfiguration(cfg =>
{
    cfg.CreateMap<User, UserDto>();
});
var mapper = config.CreateMapper();

UserDto dto = mapper.Map<UserDto>(userFromDb);
```

QUESTIONS TIME

DEPENDENCY INJECTION

RECAP ON DEPENDENCY INVERSION PRINCIPLE

D of SOLID principle – Dependency Inversion Principle

High-level modules/classes should not depend on low-level modules/classes. Both should depend upon abstractions.

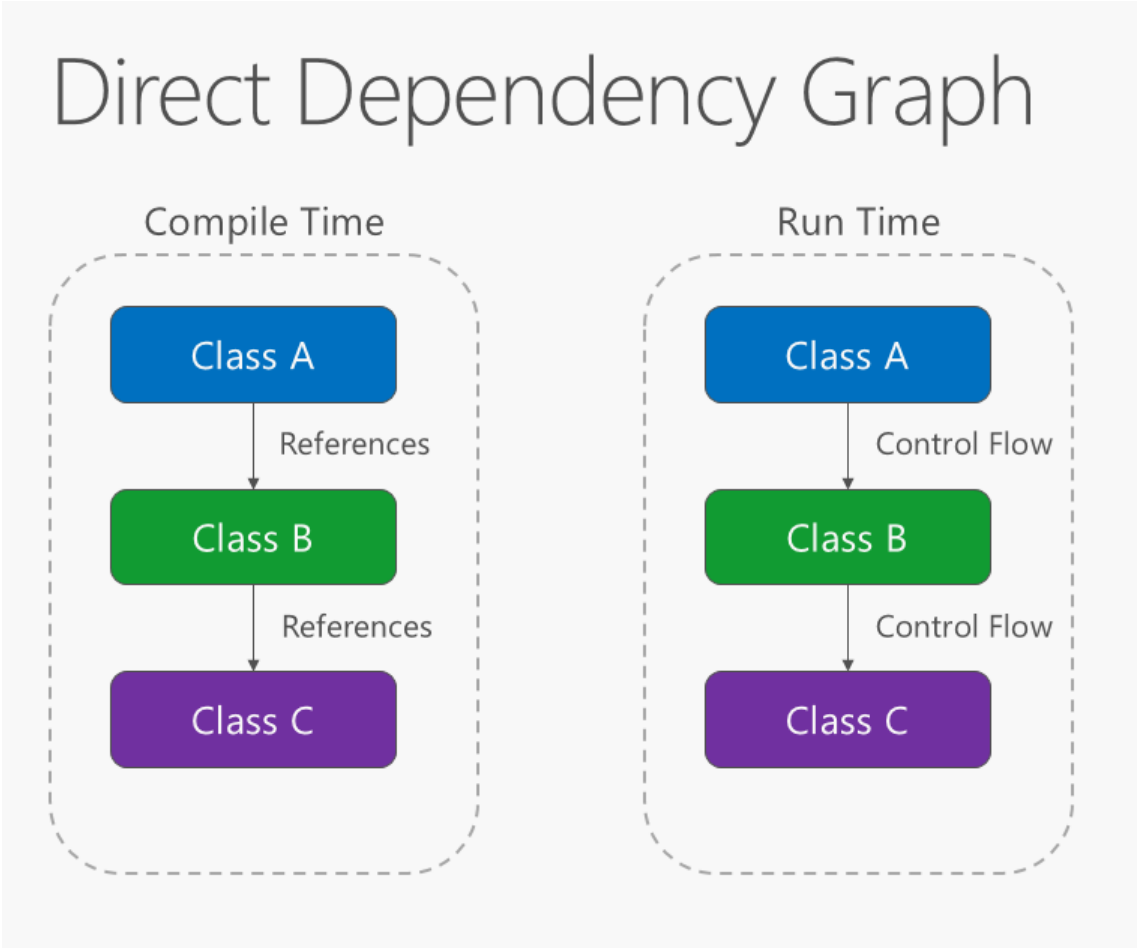
Abstractions should not depend upon details. Details should depend upon abstractions.

The above lines simply state that if a high module or class will be dependent more on low-level modules or class then your code would have tight coupling and if you will try to make a change in one class it can break another class which is risky at the production level. So always try to make classes loosely coupled as much as you can and you can achieve this through abstraction. The main motive of this principle is decoupling the dependencies so if class A changes the class B doesn't need to care or know about the changes.

You can consider the real-life example of a TV remote battery. Your remote needs a battery but it's not dependent on the battery brand. You can use any XYZ brand that you want and it will work. So, we can say that the TV remote is loosely coupled with the brand name.

INVERSION OF CONTROL (IOC)

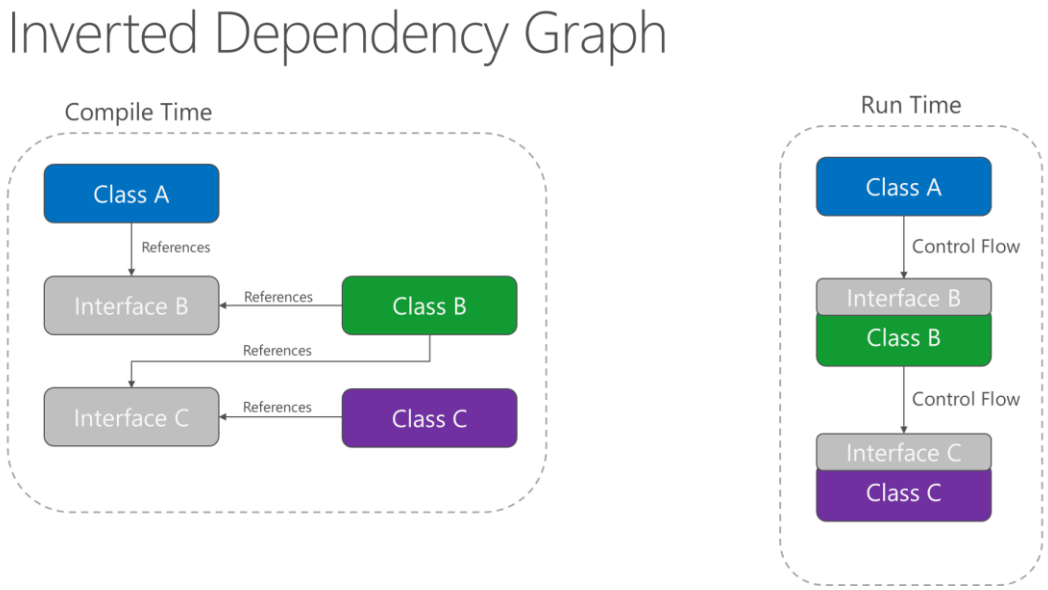
The direction of dependency within the application should be in the direction of abstraction, not implementation details. Most applications are written such that compile-time dependency flows in the direction of runtime execution, producing a direct dependency graph. That is, if class A calls a method of class B and class B calls a method of class C, then at compile time class A will depend on class B, and class B will depend on class C



INVERSION OF CONTROL (IOC)

Applying the **dependency inversion principle** allows A to call methods on an abstraction that B implements, making it possible for A to call B at run time, but for B to depend on an interface controlled by A at compile time (thus, inverting the typical compile-time dependency). At run time, the flow of program execution remains unchanged, but the introduction of interfaces means that different implementations of these interfaces can easily be plugged in.

Dependency inversion is a key part of building loosely coupled applications, since implementation details can be written to depend on and implement higher-level abstractions, rather than the other way around. The resulting applications are more testable, modular, and maintainable as a result. The practice of **dependency injection** is made possible by following the **dependency inversion principle**.



DEPENDENCY INVERSION PRINCIPLE

```
public interface ILogger
{
   public void Write(string message);
}

public class FileLogger : ILogger
{
   public void Write(string message) {
       // write to file
   }
}

public class StandardOutLogger : ILogger
{
   public void Write(string message) {
       // write to standard out
   }
}

public void DoStuff(ILogger logger)
{
   // do stuff
   logger.Write("Some message")
}
```

.NET DEPENDENCY INJECTION BASICS

- .NET supports the **dependency injection (DI)** software design pattern, which is a technique for achieving Inversion of Control (IoC) between classes and their dependencies. Dependency injection in .NET is a built-in part of the framework, along with configuration, logging, and the options pattern.
- A dependency is an object that another object depends on.
- Dependency injection addresses these problems through:
    1. The use of an interface or base class to abstract the dependency implementation.
    2. Registration of the dependency in a service container. .NET provides a built-in service container, ***IServiceProvider***. Services are typically registered at the app's start-up and appended to an ***IServiceCollection***. Once all services are added, you use *BuildServiceProvider()* to create the service container.
    3. Injection of the service into the constructor of the class where it's used. The framework takes on the responsibility of creating an instance of the dependency and disposing of it when it's no longer needed.

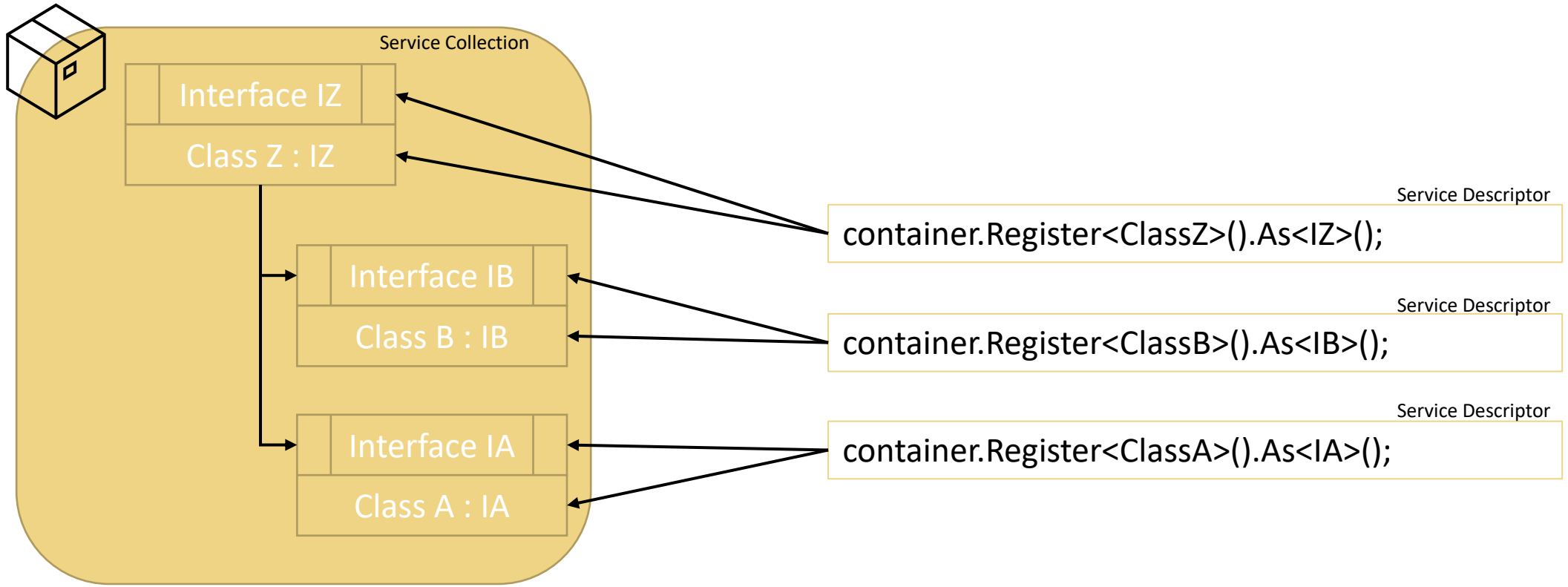
.NET DEPENDENCY INJECTION BASICS

The abstractions for DI in .NET are defined in the Microsoft.Extensions.DependencyInjection.Abstractions NuGet package:

- **IServiceCollection**: Defines a contract for a collection of service descriptors.
- **IServiceProvider**: Defines a mechanism for retrieving a service object.
- **ServiceDescriptor**: Describes a service with its service type, implementation, and lifetime.

In .NET, DI is managed by adding services and configuring them in an IServiceCollection. After services are registered, an IServiceProvider instance is built by calling the BuildServiceProvider method. The IServiceProvider acts as a container of all the registered services, and it's used to resolve services.

.NET DEPENDENCY INJECTION BASICS



SERVICES LIFETIMES

The most commonly used APIs for adding services to the ServiceCollection are lifetime-named generic extension methods, such as:

- AddTransient<TService>: **transient** lifetime services are created each time they're requested from the service container. This lifetime works best for lightweight, stateless services.
- AddScoped<TService>: for web applications, a **scoped** lifetime indicates that services are created once per client request (connection).
- AddSingleton<TService>: **singleton** lifetime services are created either the first time they're requested or by the developer, when providing an implementation instance directly to the container. Every subsequent request of the service implementation from the dependency injection container uses the same instance. If the app requires singleton behavior, allow the service container to manage the service's lifetime.

By default, in the development environment, resolving a service from another service with a longer lifetime throws an exception.

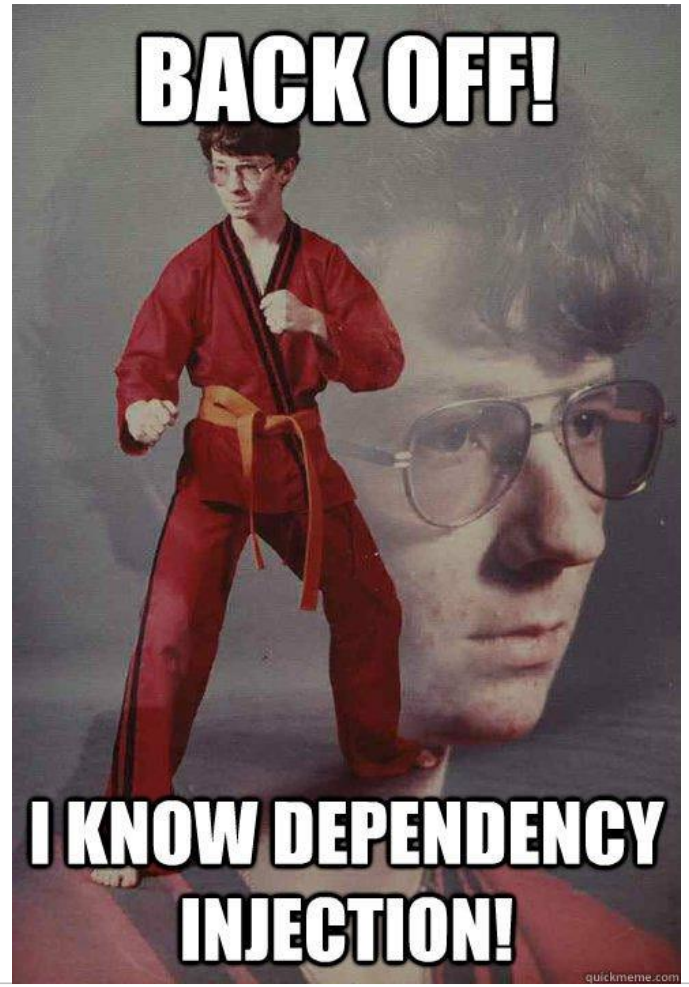
- Resolve a singleton service from a scoped, transient or singleton service.
- Resolve a scoped service from another scoped or transient service.
- Resolve a transient service from another transient service.

DEPENDENCY INJECTION GUIDELINES

- Avoid direct instantiation of dependent classes within services. Direct instantiation couples the code to a particular implementation.
- Make services small, well-factored, and easily tested.
- Services resolved from the container should never be disposed by the developer. Instances created by the service provider are disposed by the provider itself by calling the Dispose of IDisposable interface.
- The built-in service container is designed to serve the needs of the framework and most consumer apps. We recommend using the built-in container unless you need a specific feature that it doesn't support.
- The following third-party containers can be used: Autofac, DryIoc, Grace, LightInject…

QUESTIONS TIME

DEPENDENCY INJECTION



DEMO: .NET DEPENDENCY INJECTION

EXERCISE: DEPENDENCY INJECTION

EXERCISE: DEPENDENCY INJECTION

Create a simple console application that demonstrates constructor-based dependency injection using interfaces and services.

Scenario: You are building a logging system. The application should be able to log messages using different logging mechanisms (e.g., Console logging, File logging).

You will use Dependency Injection to inject the desired logging service.

Steps

1. Define the Interface
    1. Create an interface ILogger with a method Log(string message).
2. Implement the Interface: create three classes that implement ILogger:
    1. DebugLogger: logs to Debug console
    2. ConsoleLogger: logs to the console.
    3. FileLogger: logs to a file (e.g., log.txt).
3. Create a Service that Depends on ILogger
    1. Create a class Application that uses ILogger to log messages.
4. Inject the Dependency in Main
5. Use constructor injection to pass the desired logger to the Application.

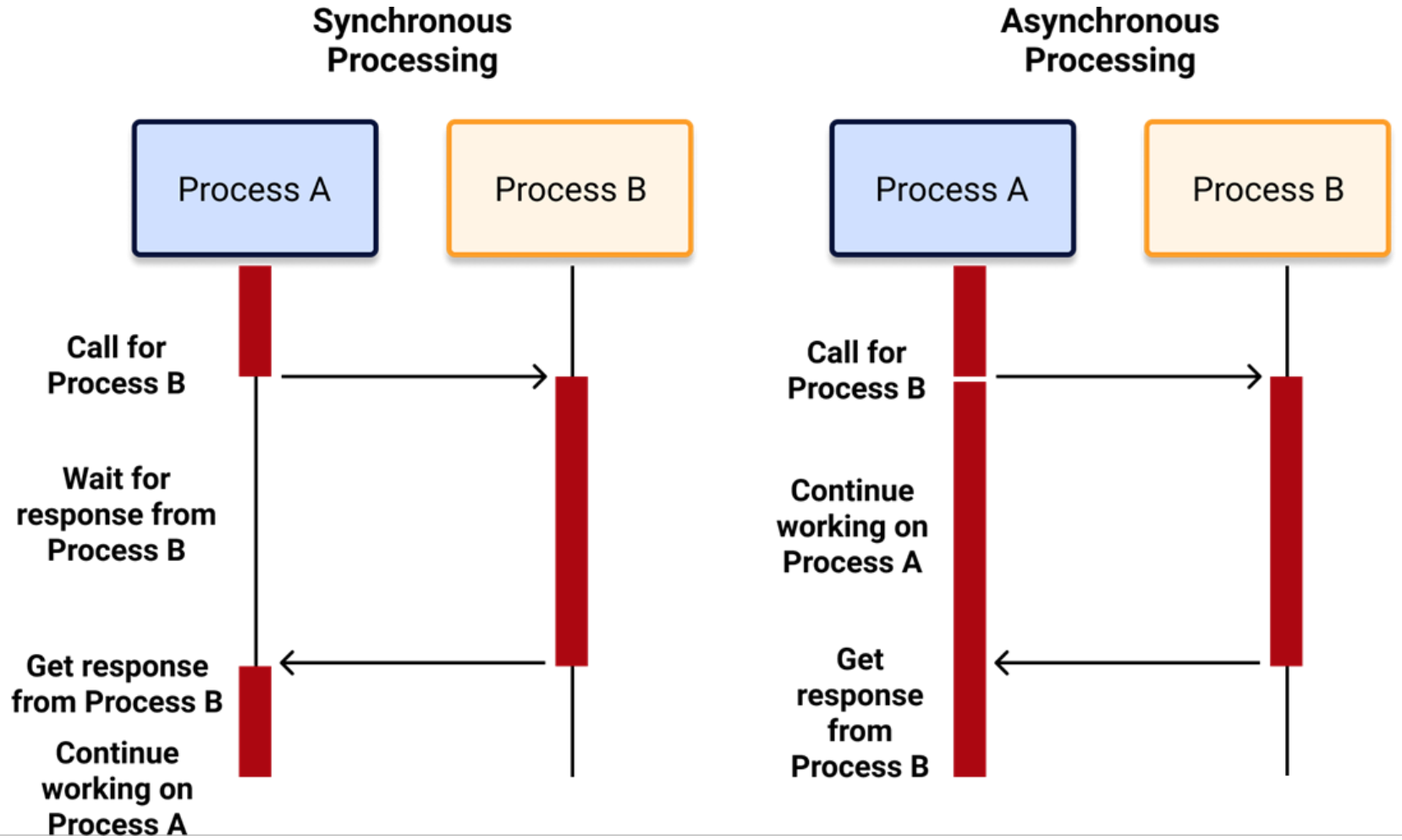
ASYNCHRONOUS PROGRAMMING

ASYNCHRONOUS PROGRAMMING

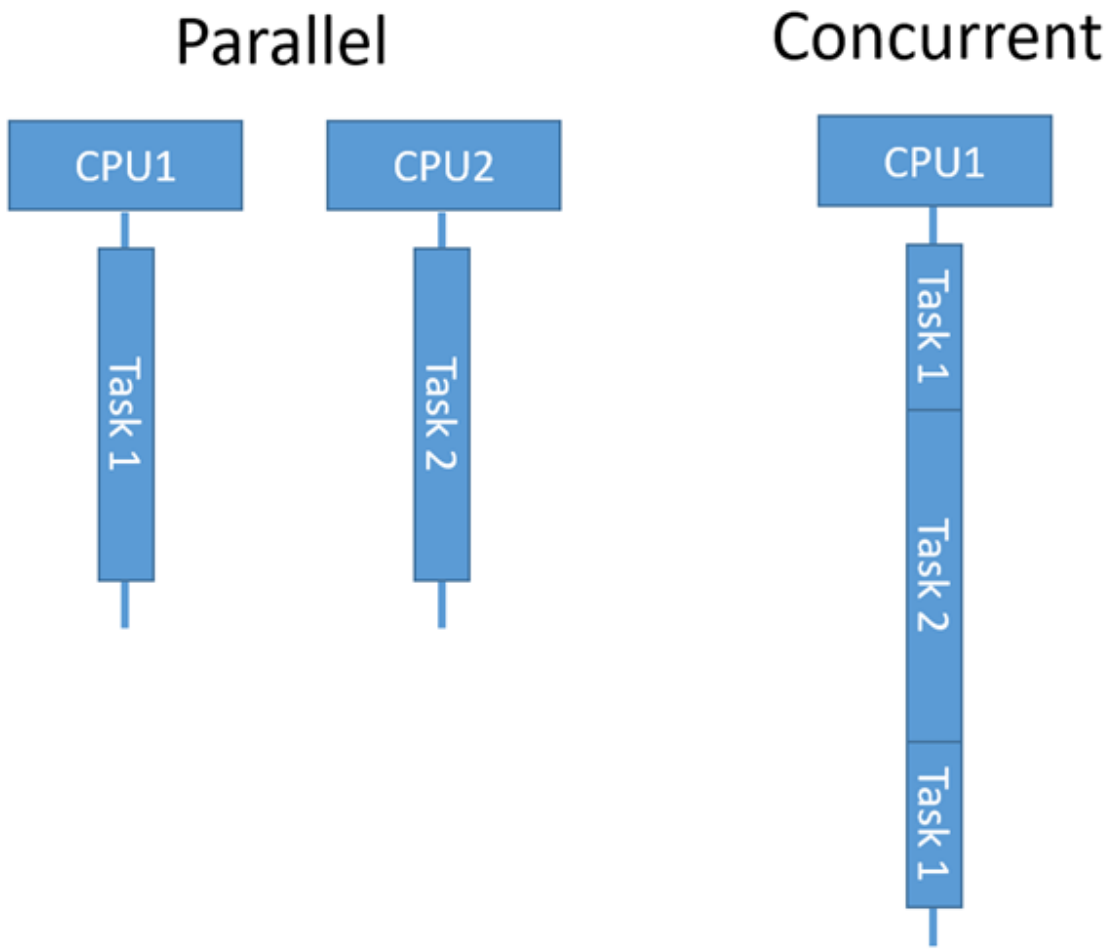
An asynchronous programming model allows a program to start a long-running* operation without waiting for it to complete, continuing to execute other tasks concurrently. This "non-blocking" approach keeps applications responsive by enabling other parts of the program to run while waiting for slow operations like network requests or file I/O. Instead of freezing, the program receives a notification when the asynchronous task finishes, allowing it to retrieve the result or handle the completion.

- When a long-running task is initiated (like fetching data from a server or reading/writing files), the program doesn't halt and wait. Instead, it can start executing other instructions.
- Tasks are not necessarily running at the exact same time in parallel, but rather the program can switch between them, managing multiple operations without waiting for each one to finish before moving on to the next.
- Once the background task is finished, the program is informed, and it can then access the results or perform actions based on the completion of that task.

ASYNCHRONOUS PROGRAMMING



PARALLEL VS CONCURRENT



ASYNCHRONOUS PROGRAMMING

- Cooking breakfast is a good example of asynchronous work that isn't parallel. One person (or thread) can handle all the tasks. One person can make breakfast asynchronously by starting the next task before the previous task completes. Each cooking task progresses regardless of whether someone is actively watching the process. As soon as you start warming the pan for the eggs, you can begin cooking the hash browns. After the hash browns start to cook, you can put the bread in the toaster.
- For a parallel algorithm, you need multiple people who cook (or multiple threads). One person cooks the eggs, another cooks the hash browns, and so on. Each person focuses on their one specific task. Each person who is cooking (or each thread) is blocked synchronously waiting for the current task to complete: Hash browns ready to flip, bread ready to pop up in toaster, and so on.

WHEN TO USE ASYNCHRONOUS

- If you want the computer to execute instructions asynchronously, you must write asynchronous code. When you write client programs, you want the UI to be responsive to user input. Your application shouldn't freeze all interaction while downloading data from the web. When you write server programs, you don't want to block threads that might be serving other requests. Using synchronous code when asynchronous alternatives exist hurts your ability to scale out less expensively. You pay for blocked threads.
- Successful modern apps require asynchronous code. Without language support, writing asynchronous code requires callbacks, completion events, or other means that obscure the original intent of the code. The advantage of synchronous code is the step-by-step action that makes it easy to scan and understand. Traditional asynchronous models force you to focus on the asynchronous nature of the code, not on the fundamental actions of the code.

ASYNCHRONOUS PROGRAMMING

```
static void Main(string[] args)
{
    Coffee cup = PourCoffee();
    Console.WriteLine("coffee is ready");

    Egg eggs = FryEggs(2);
    Console.WriteLine("eggs are ready");

    HashBrown hashBrown = FryHashBrowns(3);
    Console.WriteLine("hash browns are ready");

    Toast toast = ToastBread(2);
    ApplyButter(toast);
    ApplyJam(toast);
    Console.WriteLine("toast is ready");

    Juice oj = PourOJ();
    Console.WriteLine("oj is ready");
    Console.WriteLine("Breakfast is ready!");
}
```

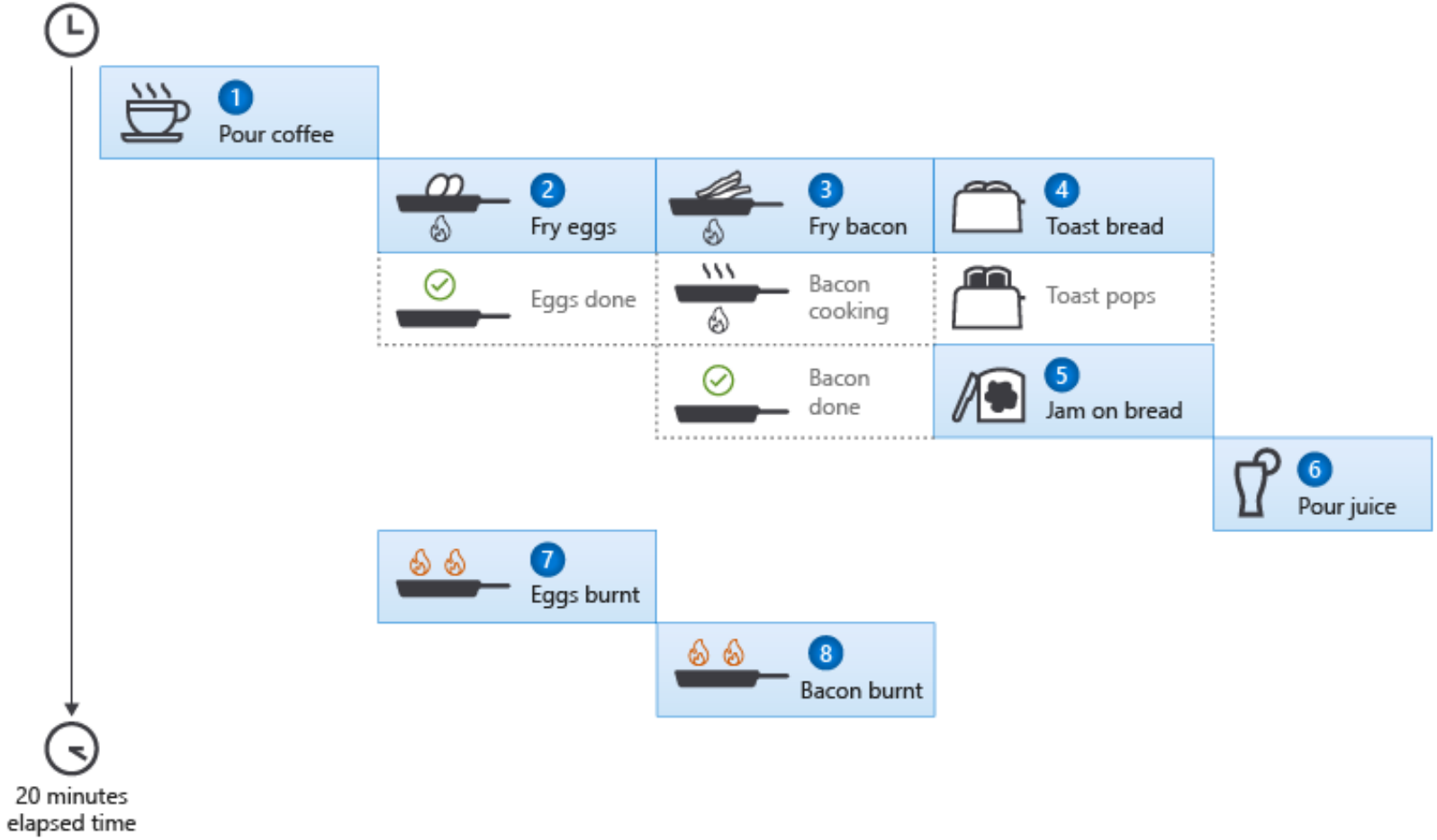
ASYNCHRONOUS PROGRAMMING

```
static async Task Main(string[] args)
{
    Coffee cup = PourCoffee();
    Console.WriteLine("coffee is ready");

    Egg eggs = await FryEggsAsync(2);
    Console.WriteLine("eggs are ready");

    HashBrown hashBrown = await FryHashBrownsAsync(3);
    Console.WriteLine("hash browns are ready");

    Toast toast = await ToastBreadAsync(2);
    ApplyButter(toast);
    ApplyJam(toast);
    Console.WriteLine("toast is ready");

    Juice oj = PourOJ();
    Console.WriteLine("oj is ready");
    Console.WriteLine("Breakfast is ready!");
}
```

ASYNCHRONOUS PROGRAMMING

```
static async Task Main(string[] args)
{
    Coffee cup = PourCoffee();
    Console.WriteLine("Coffee is ready");

    Task<Egg> eggsTask = FryEggsAsync(2);
    Task<HashBrown> hashBrownTask = FryHashBrownsAsync(3);
    Task<Toast> toastTask = ToastBreadAsync(2);

    Toast toast = await toastTask;
    ApplyButter(toast);
    ApplyJam(toast);
    Console.WriteLine("Toast is ready");
    Juice oj = PourOJ();
    Console.WriteLine("Oj is ready");

    Egg eggs = await eggsTask;
    Console.WriteLine("Eggs are ready");
    HashBrown hashBrown = await hashBrownTask;
    Console.WriteLine("Hash browns are ready");

    Console.WriteLine("Breakfast is ready!");
}
```

ASYNCHRONOUS PROGRAMMING



APPLY AWAIT EXPRESSIONS TO TASKS EFFICIENTLY

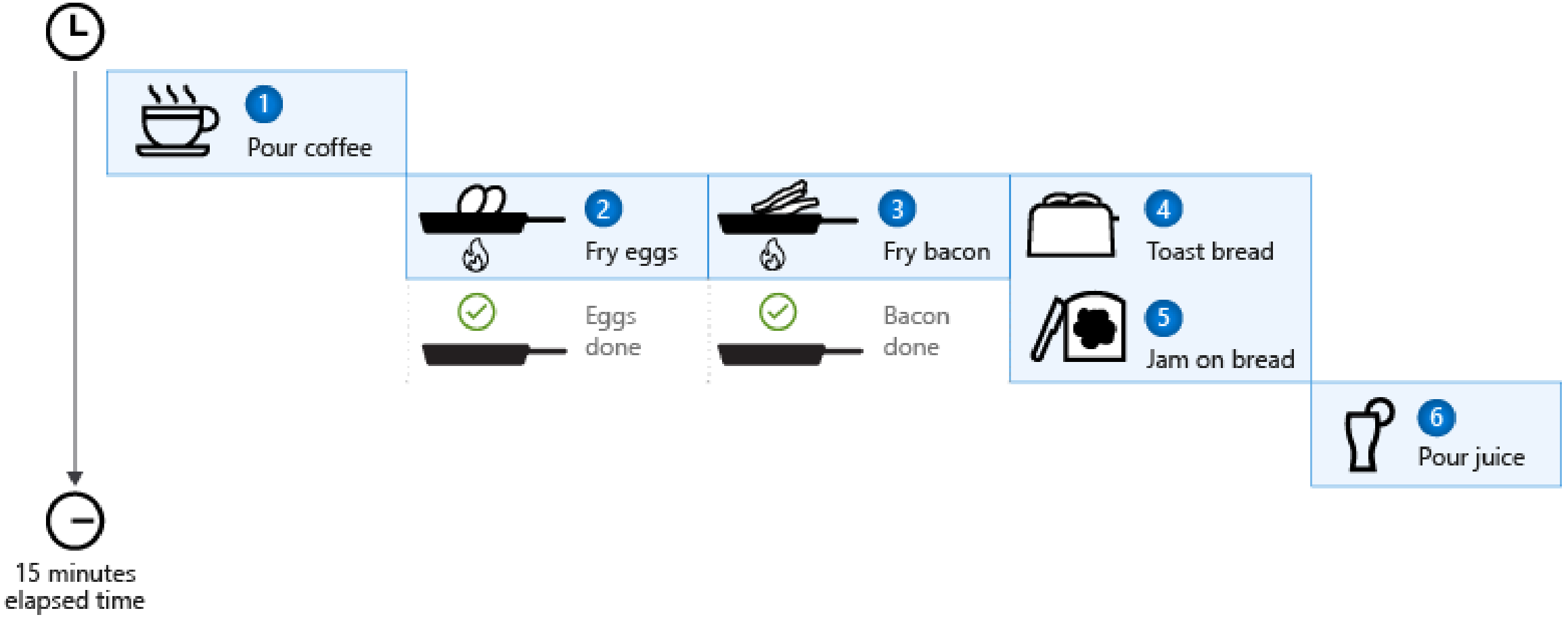
- You can improve the series of await expressions at the end of the previous code by using methods of the Task class. One API is the WhenAll method, which returns a Task object that completes when all the tasks in its argument list are complete.
- Another option is to use the WhenAny method, which returns a Task<Task> object that completes when any of its arguments complete.

```
await Task.WhenAll(eggsTask, hashBrownTask, toastTask);
Console.WriteLine("Eggs are ready");
Console.WriteLine("Hash browns are ready");
Console.WriteLine("Toast is ready");
Console.WriteLine("Breakfast is ready!");
```

APPLY AWAIT EXPRESSIONS TO TASKS EFFICIENTLY

```
var breakfastTasks = new List<Task> { eggsTask, hashBrownTask, toastTask };
while (breakfastTasks.Count > 0)
{
    Task finishedTask = await Task.WhenAny(breakfastTasks);
    if (finishedTask == eggsTask)
    {
        Console.WriteLine("Eggs are ready");
    }
    else if (finishedTask == hashBrownTask)
    {
        Console.WriteLine("Hash browns are ready");
    }
    else if (finishedTask == toastTask)
    {
        Console.WriteLine("Toast is ready");
    }
    await finishedTask;
    breakfastTasks.Remove(finishedTask);
}
```

THE FINAL RESULT



ASYNCHRONOUS PROGRAMMING PATTERNS

.NET provides three patterns for performing asynchronous operations:

- **Task-based Asynchronous Pattern (TAP)**, which uses a single method to represent the initiation and completion of an asynchronous operation. TAP was introduced in .NET Framework 4. It's the recommended approach to asynchronous programming in .NET. The async and await keywords in C# and the Async and Await operators in Visual Basic add language support for TAP. For more information, see Task-based Asynchronous Pattern (TAP).
- **Event-based Asynchronous Pattern (EAP)**, which is the event-based legacy model for providing asynchronous behavior. It requires a method that has the Async suffix and one or more events, event handler delegate types, and EventArg-derived types. EAP was introduced in .NET Framework 2.0. It's no longer recommended for new development. For more information, see Event-based Asynchronous Pattern (EAP).
- **Asynchronous Programming Model (APM)** pattern (*also called the IAsyncResult pattern*), which is the legacy model that uses the IAsyncResult interface to provide asynchronous behavior. In this pattern, asynchronous operations require Begin and End methods (for example, BeginWrite and EndWrite to implement an asynchronous write operation). This pattern is no longer recommended for new development. For more information, see Asynchronous Programming Model (APM).

TASK-BASED ASYNCHRONOUS PATTERN (TAP)

```
public class MyClass
{
    public int Read(byte [] buffer, int offset, int count);
}
```

```
public class MyClass
{
    public Task<int> ReadAsync(byte [] buffer, int offset, int count);
}
```

EVENT-BASED ASYNCHRONOUS PATTERN (EAP)

```
public class MyClass
{
    public int Read(byte [] buffer, int offset, int count);
}
```

```
public class MyClass
{
    public void ReadAsync(byte [] buffer, int offset, int count);
    public event ReadCompletedEventHandler ReadCompleted;
}
```

ASYNCHRONOUS PROGRAMMING MODEL (APM)

```
public class MyClass
{
    public int Read(byte [] buffer, int offset, int count);
}
```

```
public class MyClass
{
    public IAsyncResult BeginRead(
        byte [] buffer, int offset, int count,
        AsyncCallback callback, object state);
    public int EndRead(IAsyncResult asyncResult);
}
```

QUESTIONS TIME

TASK-BASED ASYNCHRONOUS PATTERN (TAP) IN .NET

236

TASK-BASED ASYNCHRONOUS PATTERN

- In .NET, The task-based asynchronous pattern is the recommended asynchronous design pattern for new development. It is based on the **Task** and **Task<TResult>** types in the *System.Threading.Tasks* namespace, which are used to represent asynchronous operations.
- A TAP method returns either a **System.Threading.Tasks.Task** or a **System.Threading.Tasks.Task<TResult>**, based on whether the corresponding synchronous method returns *void* or a type *TResult*.
- At its heart, a Task is just a data structure that represents the eventual completion of some asynchronous operation (other frameworks call a similar type a “promise” or a “future”). A Task is created to represent some operation, and then when the operation it logically represents completes, the results are stored into that.
- How to convert a synchronous method in its asynchronous counterpart:
  - The parameters of a TAP method should match the parameters of its synchronous counterpart and should be provided in the same order. However, **out** and **ref** parameters are exempt from this rule and should be avoided entirely. Any data that would have been returned through an out or ref parameter should instead be returned as part of the TResult returned by Task<TResult> and should use a tuple or a custom data structure to accommodate multiple values. Also, consider adding a **CancellationToken** parameter even if the TAP method's synchronous counterpart does not offer one.
- The **Task** class provides a life cycle for asynchronous operations, and that cycle is represented by the **TaskStatus** enumeration.

TASK-BASED ASYNCHRONOUS PATTERN

You may implement both compute-bound and I/O-bound asynchronous operations as TAP methods. However, when TAP methods are exposed publicly from a library, they should be provided only for workloads that involve I/O-bound operations (they may also involve computation but should not be purely computational). If a method is purely compute-bound, it should be exposed only as a synchronous implementation. The code that consumes it may then choose whether to wrap an invocation of that synchronous method into a task to offload the work to another thread or to achieve parallelism. And if a method is I/O-bound, it should be exposed only as an asynchronous implementation.

ASYNC/AWAIT

The async and await keywords are part of the Task-based Asynchronous Pattern (TAP) introduced in .NET 4.5. They simplify asynchronous programming by allowing you to write code that looks synchronous but runs asynchronously.

- **async** marks a method as asynchronous.
- **await** pauses the execution of the method until the awaited Task completes, without blocking the thread.

This is especially useful for I/O-bound operations like file access, web requests, or database queries.

ASYNC/AWAIT

```
public async Task<string> GetDataAsync()
{
    // Simulate an asynchronous operation
    await Task.Delay(1000); // Waits 1 second asynchronously
    return "Data retrieved";
}
```

```
public async Task MainAsync()
{
    string result = await GetDataAsync();
    Console.WriteLine(result);
}
```

ASYNC/AWAIT EXAMPLES

```
public async Task<string> FetchWebContentAsync(string url)
{
    using HttpClient client = new HttpClient();
    string content = await client.GetStringAsync(url);
    return content;
}
```

ASYNC/AWAIT EXAMPLES

```
public async Task RunMultipleTasksAsync()
{
    Task<string> task1 = GetDataAsync();
    Task<string> task2 = GetDataAsync();

    string[] results = await Task.WhenAll(task1, task2);
    Console.WriteLine(string.Join(", ", results));
}
```

ASYNC/AWAIT EXAMPLES

```
public async Task SafeAsync()
{
    try
    {
        string result = await GetDataAsync();
        Console.WriteLine(result);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}
```

QUESTIONS TIME

TASK-BASED ASYNCHRONOUS PATTERN: DEMO

245

EXERCISE: TAP

EXERCISE: TAP

Download Two Web Pages Sequentially vs Concurrently

- Create two async methods that download a web page.
- First, call them sequentially.
- Then, call them concurrently using Task.WhenAll.

Tips: Use the HttpClient type and invoke the GetStringAsync(string url) method to download the content.

ASYNC/AWAIT – HOW IT REALLY WORKS

One of the most powerful aspects of async/await in C# is that it allows developers to write asynchronous code in a linear, readable style, as if it were synchronous. This dramatically improves maintainability and clarity, especially compared to older patterns like callbacks or BeginInvoke/EndInvoke.

When the C# compiler sees an async method, it:

- Generates a hidden class that implements IAsyncStateMachine.
- Splits the method into multiple states—each representing a point before or after an await.
- Stores local variables and the current state so that execution can resume correctly.
- Uses a builder (like AsyncTaskMethodBuilder) to manage the Task returned to the caller.

This transformation allows the method to pause and resume execution without blocking the thread, while preserving the illusion of synchronous flow.

ASYNC/AWAIT – HOW IT REALLY WORKS

```
public static async Task<string> DownloadHtmlAsyncTask(string url)
{
    HttpClient httpClient = new HttpClient();
    Debug.WriteLine("before await");
    string result = await httpClient.GetStringAsync(url);
    Debug.WriteLine("after await");
    return result;
}
```

ASYNC/AWAIT – HOW IT REALLY WORKS

```
[DebuggerStepThrough, AsyncStateMachine(typeof(AsyncMethods.<DownloadHtmlAsyncTask>d__0))]
public static Task<string> DownloadHtmlAsyncTask(string url)
{
    AsyncMethods.<DownloadHtmlAsyncTask>d__0 <DownloadHtmlAsyncTask>d__;
    <DownloadHtmlAsyncTask>d__.url = url;
    <DownloadHtmlAsyncTask>d__.<>t__builder = AsyncTaskMethodBuilder<string>.Create();

    //set initial machine state to -1
    <DownloadHtmlAsyncTask>d__.<>1__state = -1;
    AsyncTaskMethodBuilder<string> <>t__builder = <DownloadHtmlAsyncTask>d__.<>t__builder;
    <>t__builder.Start<AsyncMethods.<DownloadHtmlAsyncTask>d__0>(ref <DownloadHtmlAsyncTask>d__);
    return <DownloadHtmlAsyncTask>d__.<>t__builder.get_Task();
}
```

ASYNC/AWAIT – HOW IT REALLY WORKS

```
[CompilerGenerated]
[StructLayout(LayoutKind.Auto)]
private struct <DownloadHtmlAsyncTask>d__0 : IAsyncStateMachine
{
    //initial state is set to -1 by the machine's creation code
    public int <>1__state;
    public AsyncTaskMethodBuilder<string> <>t__builder;
    public string url;
    public HttpClient <httpClient>5__1;
    public string <result>5__2;
    private TaskAwaiter<string> <>u__$awaiter3;
    private object <>t__stack;
    void IAsyncStateMachine.MoveNext()
    {
        string result;
        try
        {
            int num = this.<>1__state;

            //if (!Stop)
            if (num != -3)
            {
                TaskAwaiter<string> taskAwaiter;
                //machine starts with num=-1 so we enter
                if (num != 0)
                {
                    //first (+ initial) state code, run code before await is invoked
                    this.<httpClient>5__1 = new HttpClient();
                    Debug.WriteLine("before await");

                    //a task is invoked
                    taskAwaiter = this.<httpClient>5__1.GetStringAsync(this.url).GetAwaiter();
                    ...
                    ...
                    ...
```

EVENT-BASED ASYNCHRONOUS PATTERN (EAP) IN .NET

EVENT-BASED ASYNCHRONOUS PATTERN (EAP) IN .NET

The Event-based Asynchronous Pattern makes available the advantages of multithreaded applications while hiding many of the complex issues inherent in multithreaded design. Using a class that supports this pattern can allow you to:

- Perform time-consuming tasks, such as downloads and database operations, "in the background," without interrupting your application.
- Execute multiple operations simultaneously, receiving notifications when each completes.
- Wait for resources to become available without stopping ("blocking") your application.
- Communicate with pending asynchronous operations using the familiar events-and-delegates model. For more information on using event handlers and delegates.

A class that supports the Event-based Asynchronous Pattern will have one or more methods named *MethodName**Async***. These methods may mirror synchronous versions, which perform the same operation on the current thread. The class may also have a *MethodName**Completed*** event, and it may have a *MethodName**AsyncCancel*** (or simply ***CancelAsync***) method.

EVENT-BASED ASYNCHRONOUS PATTERN

A class that adheres to the Event-based Asynchronous Pattern may optionally provide an event for tracking progress and incremental results. This will typically be named ***ProgressChanged*** or *MethodName****ProgressChanged***, and its corresponding event handler will take a **ProgressChangedEventArgs** parameter.

The event handler for the ProgressChanged event can examine the *ProgressChangedEventArgs.ProgressPercentage* property to determine what percentage of an asynchronous task has been completed. This property will range from 0 to 100, and it can be used to update the Value property of a ProgressBar. If multiple asynchronous operations are pending, you can use the **ProgressChangedEventArgs.UserState** property to distinguish which operation is reporting progress.

EVENT-BASED ASYNCHRONOUS PATTERN (EAP) IN .NET

```
public class AsyncExample
{
    // Synchronous methods.
    public int Method1(string param);

    // Asynchronous methods.
    public void Method1Async(string param);
    public void Method1Async(string param, object userState);
    public event Method1CompletedEventHandler Method1Completed;
    public event ProgressChangedEventHandler Method1ProgressChanged;
}
```

EVENT-BASED ASYNCHRONOUS PATTERN (EAP): DEMO

EXERCISE: EAP

EXERCISE: EAP

Implement a class which expose a method to download a web page by using the EAP approach

- Downloads **one web page**.
- Measures **elapsed time** and **page size in bytes**.
- Emits an **event** with Url, ElapsedMilliseconds, and SizeInBytes.

Tips: Use the HttpClient type and invoke the GetStringAsync(string url) method to download the content.

TASK VS THREADS (VS PROCESSES)

A Thread is an OS resource (Kernel thread) that executes code. You schedule work manually. While a Task is a higher-level .NET abstraction that represents an asynchronous operation (which may or may not use a thread). It is about logically composing work, not owning a thread.

Threads:

- Low-level primitive. You decide when to create/start/join. Expensive to create (stack allocation, kernel transition).
- Always an OS thread actively running your code.
- No built-in chaining; you must implement signaling (ManualResetEventSlim, Join, etc.).

Tasks:

- Logical unit of work with built-in state machine support, continuations, composition (Task.WhenAll, WhenAny), cancellation, exception propagation.
- Usually queued to the ThreadPool (reusing existing threads). For I/O-bound async (await socket/file/db), no thread is occupied while waiting—kernel notifies completion, and a continuation is queued later.
- Continuations (ContinueWith), async/await, cancellation tokens, error aggregation, TaskCompletionSource for bridging events.

QUESTIONS TIME

TASK PARALLEL LIBRARY (TPL)

TASK PARALLEL LIBRARY (TPL)

- The **Task Parallel Library (TPL)** is a set of public types and APIs in the *System.Threading* and *System.Threading.Tasks* namespaces. The purpose of the TPL is to make developers more productive by simplifying the process of adding parallelism and concurrency to applications. The TPL dynamically scales the degree of concurrency to use all the available processors most efficiently. In addition, the TPL handles the partitioning of the work, the scheduling of threads on the *ThreadPool*, cancellation support, state management, and other low-level details. By using TPL, you can maximize the performance of your code while focusing on the work that your program is designed to accomplish.
- In .NET Framework 4, the TPL is the preferred way to write multithreaded and parallel code. However, not all code is suitable for parallelization. For example, if a loop performs only a small amount of work on each iteration, or it doesn't run for many iterations, then the overhead of parallelization can cause the code to run more slowly. Furthermore, parallelization, like any multithreaded code, adds complexity to your program execution. Although the TPL simplifies multithreaded scenarios, we recommend that you have a basic understanding of threading concepts, for example, locks, deadlocks, and race conditions, so that you can use the TPL effectively.

TASK PARALLEL LIBRARY (TPL)

Method class	Purpose	Notes
Parallel.For(from, to, body)	Index-based parallel loop	Use local init/Finally overloads for reduction
Parallel.ForEach(source, body)	Enumerate in parallel	Partitioning behind the scenes
Parallel.ForEachAsync(source, ct, async (item, ct) => …)	Async-aware enumeration (.NET 6+)	Great for I/O + throttling with degree of parallelism
Parallel.Invoke(params Action[])	Fire multiple actions	Simpler composition; limited flexibility
ParallelLoopState	Break(), Stop(), IsStopped	Break = partial range optimization
ParallelOptions	CancellationToken, MaxDegreeOfParallelism, TaskScheduler	Control behavior

TASK PARALLEL LIBRARY (TPL): DEMO

THREADING BEST PRACTICES

THREADING BEST PRACTICES

Multithreading requires careful programming. For most tasks, you can reduce complexity by queuing requests for execution by thread pool threads. This topic addresses more difficult situations, such as coordinating the work of multiple threads, or handling threads that block

Multithreading solves problems with throughput and responsiveness, but in doing so it introduces new problems: deadlocks and race conditions.

**Throughput* refers to the amount of work, data, or material that is processed or transmitted through a system in a given period of time.

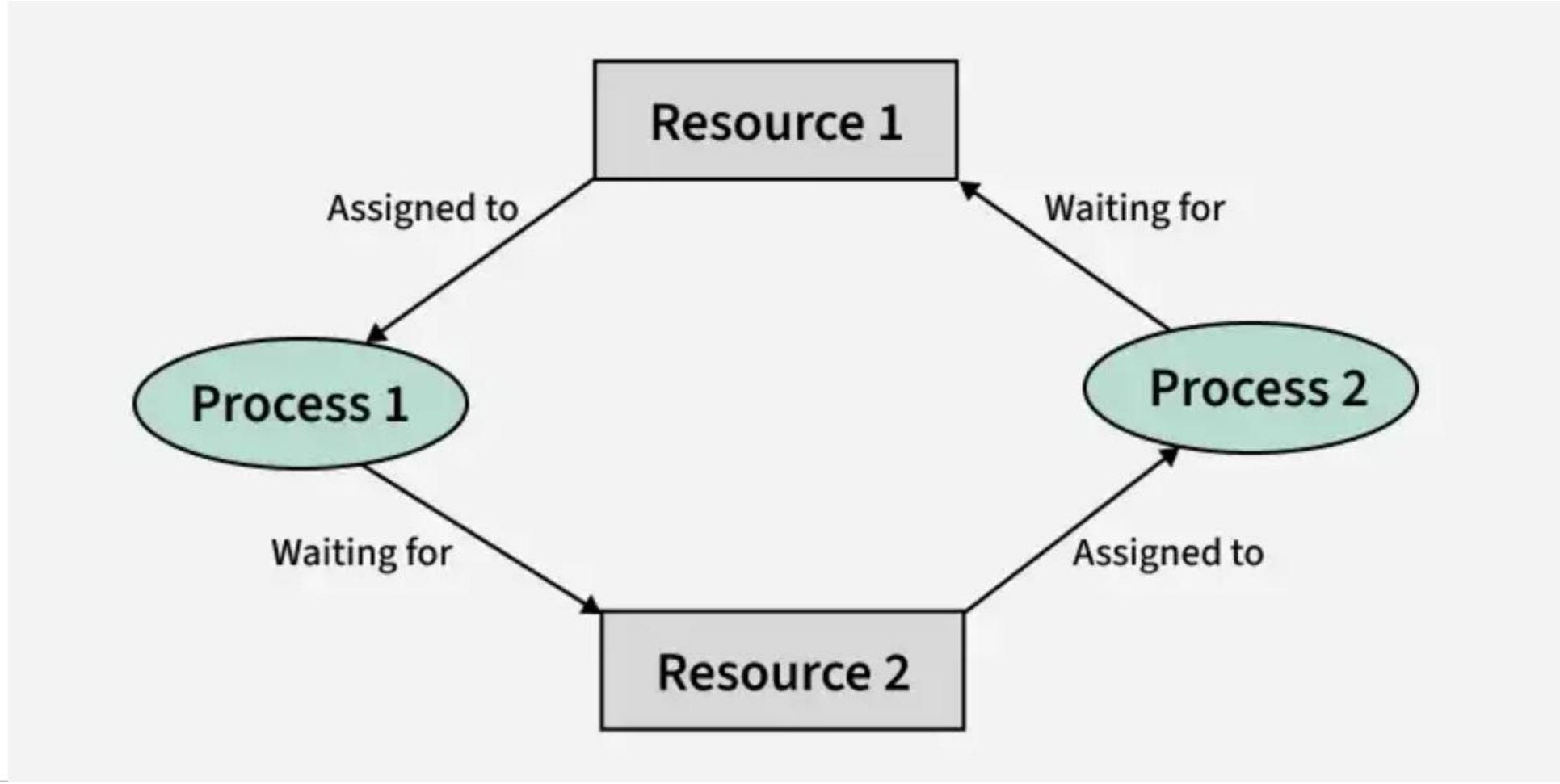
**Responsiveness* is the speed at which a system or interface reacts to user interactions, such as clicks, taps, or commands. A responsive system provides immediate feedback, making the experience smooth and intuitive.

Multithreading solves problems with throughput and responsiveness, but in doing so it introduces new problems: deadlocks and race conditions.

DEADLOCKS

- In computer science, a deadlock is a situation where a group of processes are unable to proceed because each one is waiting for a resource that is being held by another process in the group. As a result, none of the processes can continue execution, and the system gets stuck.
- Key characteristics of a deadlock:
  - Mutual exclusion – At least one resource is held in a non-shareable mode.
  - Hold and wait – A process holding at least one resource is waiting to acquire additional resources held by other processes.
  - No preemption – Resources cannot be forcibly taken away from a process holding them.
  - Circular wait – A set of processes are waiting for each other in a circular chain.

DEADLOCKS



RACE CONDITIONS

In computer science, a race condition occurs when two or more processes or threads access shared resources concurrently, and the final outcome depends on the timing or order of their execution. This can lead to unpredictable behavior, bugs, or security vulnerabilities.

Race conditions typically arise in multi-threaded or parallel systems when:

- Multiple threads/processes read and write shared data.
- There is no proper synchronization (e.g., locks, semaphores).

Imagine an online banking system where a user has €100 in their account. The user initiates two simultaneous transactions:

- Transaction A: Transfer €100 to Account X.
- Transaction B: Transfer €100 to Account Y.

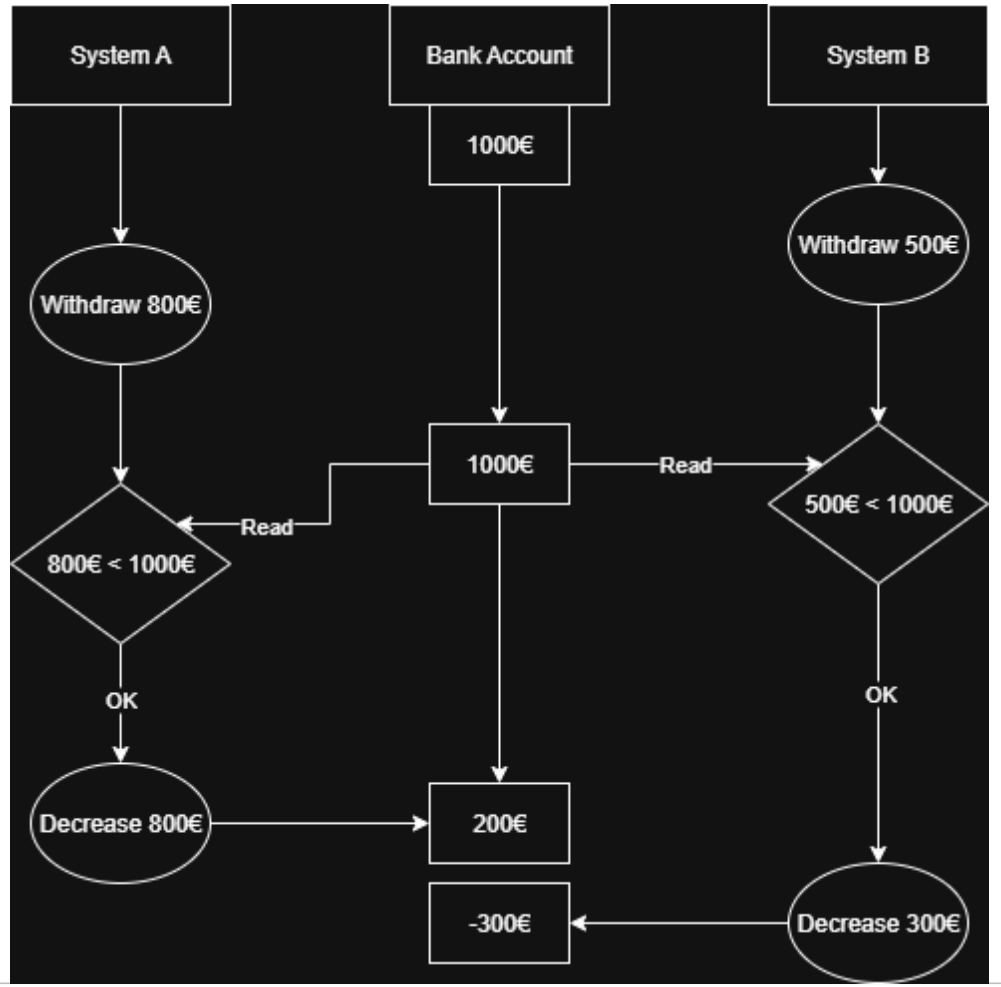
What should happen? Only one transaction should succeed, because the account only has €100. The system should check the balance before allowing the transfer.

Both transactions check the balance at the same time and see €100 available. Then:

- Transaction A proceeds and deducts €100.
- Transaction B, unaware of the deduction, also proceeds and deducts €100.

Now the account has -€100, which is incorrect and potentially dangerous.

RACE CONDITIONS



QUESTIONS TIME

THREADING OBJECTS AND FEATURES

LOCK - EXCLUSIVE ACCESS TO A SHARED RESOURCE

The **lock** statement acquires the mutual-exclusion lock for a given object, executes a statement block, and then releases the lock. While a lock is held, the thread that holds the lock can again acquire and release the lock. Any other thread is blocked from acquiring the lock and waits until the lock is released. The lock statement ensures that at maximum only one thread executes its body at any moment in time.

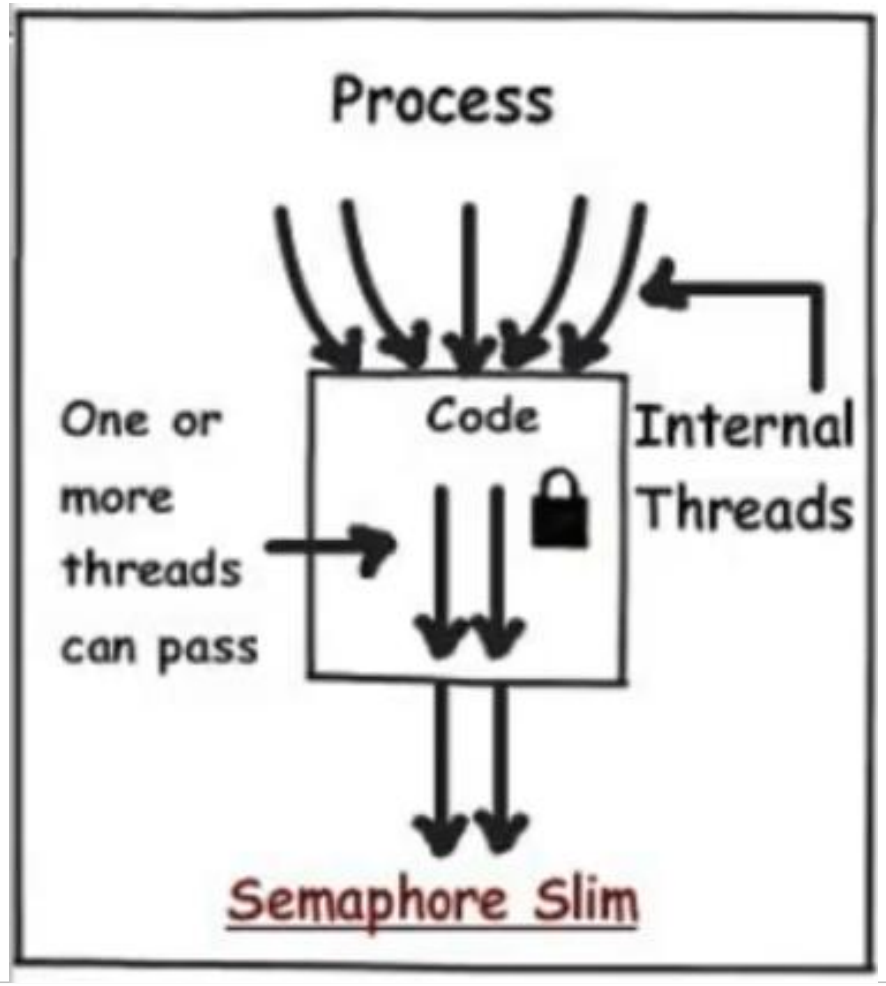
You can't use the **await** expression in the body of a lock statement.

```
object x = new object();

lock (x)
{
    // Your code...
}
```

SEMAPHORE AND SEMAPHORESLIM

- The **System.Threading.Semaphore** class represents a named (systemwide) or local semaphore. It is a thin wrapper around the Win32 semaphore object. Win32 semaphores are counting semaphores, which can be used to control access to a pool of resources.
- The **SemaphoreSlim** class represents a lightweight, fast semaphore that can be used for waiting within a single process when wait times are expected to be very short. **SemaphoreSlim** relies as much as possible on synchronization primitives provided by the common language runtime (CLR). However, it also provides lazily initialized, kernel-based wait handles as necessary to support waiting on multiple semaphores. **SemaphoreSlim** also supports the use of cancellation tokens, but it does not support named semaphores or the use of a wait handle for synchronization.
- Threads enter the semaphore by calling the **WaitOne** method or the **SemaphoreSlim.Wait** or **SemaphoreSlim.WaitAsync** method, in the case of a **SemaphoreSlim** object. When the call returns, the count on the semaphore is decremented. When a thread requests entry and the count is zero, the thread blocks. As threads release the semaphore by calling the **Semaphore.Release** or **SemaphoreSlim.Release** method, blocked threads are allowed to enter. There is no guaranteed order, such as first-in, first-out (FIFO) or last-in, first-out (LIFO), for blocked threads to enter the semaphore.

SEMAPHORE AND SEMAPHORESLIM



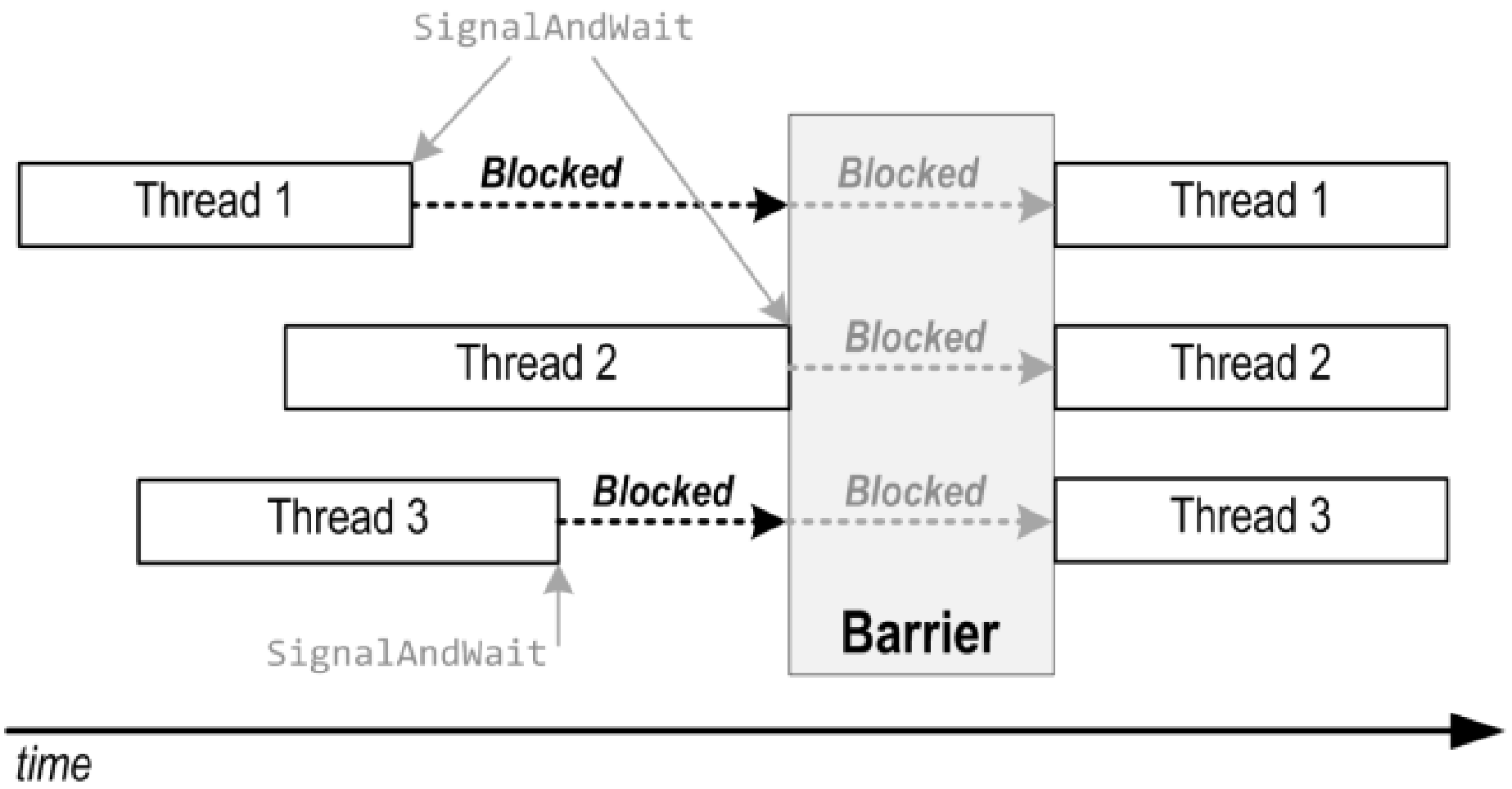
BARRIER

- A **System.Threading.Barrier** is a synchronization primitive that enables multiple threads (known as participants) to work concurrently on an algorithm in phases. Each participant executes until it reaches the barrier point in the code. The barrier represents the end of one phase of work. When a participant reaches the barrier, it blocks until all participants have reached the same barrier. After all participants have reached the barrier, you can optionally invoke a post-phase action. This post-phase action can be used to perform actions by a single thread while all other threads are still blocked. After the action has been executed, the participants are all unblocked.
- When you create a **Barrier** instance, specify the number of participants. You can also add or remove participants dynamically at any time. For example, if one participant solves its part of the problem, you can store the result, stop execution on that thread, and call **Barrier.RemoveParticipant** to decrement the number of participants in the barrier. When you add a participant by calling **Barrier.AddParticipant**, the return value specifies the current phase number, which may be useful in order to initialize the work of the new participant.

BARRIER

```
// Create the Barrier object, and supply a post-phase delegate
// to be invoked at the end of each phase.
Barrier barrier = new Barrier(2, (bar) =>
    {
        // Examine results from all threads, determine
        // whether to continue, create inputs for next phase, etc.
        if (someCondition)
            success = true;
    });

// Define the work that each thread will perform. (Threads do not
// have to all execute the same method.)
void CrunchNumbers(int partitionNum)
{
    // Up to System.Int64.MaxValue phases are supported. We assume
    // in this code that the problem will be solved before that.
    while (success == false)
    {
        // Begin phase:
        // Process data here on each thread, and optionally
        // store results, for example:
        results[partitionNum] = ProcessData(data[partitionNum]);

        // End phase:
        // After all threads arrive,post-phase delegate
        // is invoked, then threads are unblocked. Overloads
        // accept a timeout value and/or CancellationToken.
        barrier.SignalAndWait();
    }
}

// Perform n tasks to run in parallel. For simplicity
// all threads execute the same method in this example.
static void Main()
{
    var app = new BarrierDemo();
    Thread t1 = new Thread(() => app.CrunchNumbers(0));
    Thread t2 = new Thread(() => app.CrunchNumbers(1));
    t1.Start();
    t2.Start();
}
```

BARRIER



SEMAPHORE: DEMO

279

THREAD-SAFE COLLECTIONS

THREAD-SAFE COLLECTIONS

- The ***System.Collections.Concurrent*** namespace includes several collection classes that are both thread-safe and scalable. Multiple threads can safely and efficiently add or remove items from these collections, without requiring additional synchronization in user code. When you write new code, use the concurrent collection classes to write multiple threads to the collection concurrently. If you're only reading from a shared collection, then you can use the classes in the *System.Collections.Generic* namespace.

Type	Description
BlockingCollection<T>	Provides bounding and blocking functionality for any type that implements IProducerConsumerCollection<T>.
ConcurrentDictionary<TKey,TValue>	Thread-safe implementation of a dictionary of key-value pairs.
ConcurrentQueue<T>	Thread-safe implementation of a FIFO (first-in, first-out) queue.
ConcurrentStack<T>	Thread-safe implementation of a LIFO (last-in, first-out) stack.
ConcurrentBag<T>	Thread-safe implementation of an unordered collection of elements.

WHEN TO USE CONCURRENT COLLECTIONS

When multiple threads (or Tasks) access and modify shared data, you must ensure thread-safety. You can do that by:

- Manually locking (lock, semaphore).
- Using concurrent collections, which are designed for multi-threaded access.

Advantages over manual locking:

- **Correctness, baked-in**: Avoid subtle race conditions (e.g., check-then-add).
- **Performance**: Internally use fine-grained locks or lock-free/Interlocked algorithms → less contention than a single lock around a standard collection.
- **Ease of use**: Common compound operations are provided atomically (e.g., GetOrAdd, AddOrUpdate).
- **Progress under load**: Scales better with many cores and many threads.

CONCURRENT COLLECTIONS - EXAMPLE

```
using System;
using System.Collections.Concurrent;
using System.Text.RegularExpressions;

public static class RegexCache
{
    // Thread-safe cache of compiled regular expressions
    private static readonly ConcurrentDictionary<string, Regex> _cache =
        new ConcurrentDictionary<string, Regex>(StringComparer.OrdinalIgnoreCase);

    public static Regex Get(string pattern) =>
        _cache.GetOrAdd(pattern, p => new Regex(p, RegexOptions.Compiled | RegexOptions.CultureInvariant));
}

// Usage:
var emailRegex = RegexCache.Get(@"^[^@\s]+@[^@\s]+\.[^@\s]+$");
Console.WriteLine(emailRegex.IsMatch("alessandro@example.com"));
```

CONCURRENT COLLECTIONS - EXAMPLE

```
using System;
using System.Collections.Concurrent;
using System.Threading.Tasks;

public class EventCounter
{
    private readonly ConcurrentDictionary<string, int> _counts = new();

    public void Increment(string key)
    {
        _counts.AddOrUpdate(key, 1, (_, current) => current + 1);
    }

    public int GetCount(string key) => _counts.TryGetValue(key, out var v) ? v : 0;

    public void Dump()
    {
        foreach (var kv in _counts)
            Console.WriteLine($"{kv.Key}: {kv.Value}");
    }
}

// Demo
var counter = new EventCounter();
Parallel.For(0, 1_000_000, i => counter.Increment("requests"));
Console.WriteLine(counter.GetCount("requests"));
```

QUESTIONS TIME

TAKEAWAYS

- Confidence in using Collections
- Confidence in using LINQ
- When and how to use Nuget
- Principles of the Dependency Injection
- Principles of Asynchronous Programming
- Manage multitasking applications
- ... Have fun programming!

THE END

GOODBYE!

msc.com